

GAI-2109 Lesson Guide

Practical Development of Agentic AI Applications



Part of
Accenture

© 2026 Ascendient, LLC

Revision 1.0.0 published on 2026-05-20

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC

1 California Street Suite 2900

San Francisco, CA 94111

<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com

Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Table of Contents

1. GAI-2109: Practical Development of Agentic AI Applications	14
1.1. Outline	15
1.2. Agenda.....	15
1.3. Course Overview.....	16
1.4. What Is Agentic AI?	16
1.4.1. What You Will Build	17
1.4.2. About This Course	18
1.5. Prerequisites.....	18
1.5.1. What You Should Already Know	18
1.5.2. What You Do Not Need	18
1.6. Environment Setup	19
1.6.1. Pre-Configured Lab Environment	19
1.7. Reflection Questions.....	19
1.8. Additional Resources	19
2. Anatomy of an AI Agent.....	20
2.1. Module Overview	21
2.2. What is an AI Agent?	21
2.3. Characteristics of AI Agents.....	21
2.4. Core Components of an Agent.....	22
2.5. Supporting Infrastructure.....	22
2.6. Agent Design Principles	22
2.6.1. Clear Role Definition.....	22
2.6.2. Step-by-Step Reasoning	23
2.6.3. Reliability Through Structure	23
2.7. Practical Use Cases for AI Agents	23
2.7.1. Data Analysis Assistants.....	23
2.7.2. Customer Support Bots.....	24
2.7.3. Content Generation and Editing.....	24
2.7.4. Workflow Automation.....	24
2.8. Why Type Safety Matters	25

2.9. Pydantic AI Overview	25
2.10. Type Safety in Action	25
2.11. Creating a Basic Agent	26
2.11.1. Setting Up Your Environment	27
2.11.2. Defining a Simple Agent	27
2.11.3. Running the Agent Synchronously	28
2.11.4. Agent Configuration	28
2.12. System Prompts	28
2.12.1. Static System Prompts	28
2.12.2. Dynamic System Prompts	29
2.13. Model Configuration	30
2.13.1. Supported LLM Providers	30
2.13.2. Model Parameters	31
2.13.3. Key Model Parameters	32
2.14. Sync vs. Async Execution	32
2.14.1. Synchronous Execution	32
2.14.2. When to Use Sync	32
2.14.3. Asynchronous Execution	32
2.14.4. When to Use Async	33
2.14.5. Streaming Outputs	33
2.15. Summary	34
2.16. Reflection Questions	34
2.17. Additional Resources	34
3. Structured Outputs and Result Validation	35
3.1. Module Overview	36
3.2. Structured Outputs in AI Agents	36
3.2.1. The Problem with Free-Form Text	36
3.2.2. Benefits of Structured Outputs	37
3.2.3. Common Structured Formats	37
3.3. Defining Output Schemas with Pydantic Models	37
3.3.1. Basic Pydantic Model	37
3.3.2. Using the Model with an Agent	38
3.3.3. Choosing Appropriate Fields	39

3.3.4. Enumerations and Optional Fields.....	39
3.3.5. Nested Models	40
3.4. Result Validation with Pydantic AI	41
3.4.1. Automatic Validation.....	41
3.4.2. Validation Errors.....	42
3.4.3. Field Validation.....	43
3.5. Using Validators to Constrain Responses.....	43
3.5.1. Built-in Pydantic Validators	43
3.5.2. Custom Validators	44
3.6. Handling Validation Failures and Retries.....	46
3.6.1. Why Retries Matter	46
3.6.2. Basic Retry Logic.....	46
3.6.3. Retry with Adjusted Prompts	47
3.6.4. Fallback Responses.....	48
3.7. Union Types for Multi-Format Responses.....	49
3.7.1. Defining Union Types	49
3.7.2. Handling Union Responses	50
3.7.3. Use Cases for Union Types.....	51
3.8. Practical Data Extraction Patterns	51
3.8.1. Extracting Structured Data from Text.....	51
3.9. Summary.....	54
3.10. Reflection Questions.....	54
3.11. Additional Resources.....	54
4. Tool Integration and Design.....	55
4.1. Module Overview	56
4.2. Integrating Tools with AI Agents.....	56
4.3. Why Tools Matter	57
4.4. Tool Use Cases.....	57
4.5. How Tool Calling Works	58
4.6. Tool Design and Lifecycle	58
4.6.1. Phase 1: Definition.....	58
4.6.2. Phase 2: Validation.....	59
4.6.3. Phase 3: Execution.....	60

4.7. Tool Registration and Discovery.....	60
4.7.1. Registering Tools with Agents	61
4.7.2. Tool Discovery by the Agent.....	62
4.7.3. Tool Signature Shown to the LLM.....	63
4.7.4. Registering Multiple Tools.....	63
4.8. Passing Runtime Context to Tools.....	64
4.8.1. Using RunContext.....	65
4.8.2. Injecting Configuration	66
4.8.3. Sharing Resources Across Tools	67
4.9. Error Handling and Graceful Degradation.....	69
4.9.1. Handling Errors Inside Tools.....	69
4.9.2. Returning Structured Error Info.....	69
4.9.3. Fallback Strategies	71
4.9.4. Choosing an Error-Handling Strategy	72
4.10. Common Tool Integration Patterns	72
4.10.1. Calculator Pattern.....	73
4.10.2. Database Query Pattern	74
4.10.3. API Integration Pattern.....	75
4.10.4. Search Pattern.....	77
4.11. Summary.....	78
4.12. Reflection Questions	78
4.13. Additional Resources	79
5. Dependencies and Runtime Context	80
5.1. Module Overview	81
5.2. Managing External Dependencies.....	81
5.2.1. What Are Dependencies?	81
5.2.2. Problems with Hard-Coded Dependencies	81
5.2.3. Issues with Hard-Coded Dependencies	82
5.2.4. Benefits of Dependency Injection	82
5.3. Dependency Injection for Agents	82
5.3.1. Basic Dependency Injection	83
5.3.2. Typed Dependencies.....	84
5.3.3. Dependency Classes.....	85

5.4. Resource Management and State Scoping	87
5.4.1. Resource Lifecycle.....	87
5.4.2. Per-Request Scoping.....	88
5.4.3. Shared State Across Tools	90
5.5. Separating Business Logic from Agent Logic	91
5.5.1. Clean Architecture Principles.....	91
5.5.2. Bad Practice: Business Logic in Prompts	92
5.5.3. Good Practice: Business Logic in Functions.....	93
5.5.4. Layered Architecture.....	94
5.5.5. Benefits of Separation	96
5.6. Advanced Patterns.....	96
5.6.1. Factory Pattern for Dependencies.....	96
5.6.2. Dependency Injection Container.....	97
5.7. Summary	98
5.8. Reflection Questions.....	99
5.9. Additional Resources.....	99
6. Conversations, Memory, and State	100
6.1. Module Overview	101
6.2. Conversations, Memory, and State.....	101
6.2.1. The Challenge of Context Management	101
6.2.2. Why State Management Matters.....	101
6.3. Managing Multi-Turn Conversations	102
6.3.1. Role of Message History	102
6.3.2. Context Window Considerations.....	102
6.4. Memory Patterns Overview.....	103
6.5. Choosing a Memory Pattern.....	103
6.6. Memory Pattern: Sliding Window.....	104
6.6.1. Sliding Window Implementation.....	104
6.7. Memory Pattern: Summary Compression	105
6.7.1. Summary Compression Implementation.....	105
6.8. Memory Pattern: Hybrid	106
6.8.1. Hybrid Implementation.....	107

6.9. State Persistence Strategies	108
6.9.1. In-Memory State	109
6.10. Persistent Storage.....	109
6.11. Streaming Responses.....	110
6.11.1. Why Stream?	110
6.11.2. Streaming Implementation	110
6.12. Message History API	111
6.13. Building Interactive Loops	112
6.14. Human-in-the-Loop Confirmation	114
6.14.1. When to Use HITL Patterns.....	115
6.15. Asking Clarifying Questions.....	115
6.16. Confirmation Before Actions.....	116
6.17. Best Practices for Conversational Agents	117
6.17.1. Design Considerations	117
6.17.2. Testing Multi-Turn Conversations	117
6.17.3. Monitoring Metrics	118
6.18. Summary	118
6.19. Reflection Questions	118
6.20. Additional Resources.....	119
7. Evaluation and Guardrails	120
7.1. Module Overview.....	121
7.2. Defining Success Criteria	121
7.2.1. Why Evaluation Matters	121
7.2.2. Establishing Quality Metrics	121
7.2.3. Creating an Evaluation Rubric	122
7.3. Automated Evaluation Patterns	123
7.3.1. Unit Testing for Agent Logic	123
7.3.2. Testing with Mock Models.....	124
7.3.3. LLM-as-Judge Pattern.....	125
7.3.4. Building Test Datasets	126
7.4. Input and Output Guardrails.....	127
7.4.1. Why Guardrails Are Critical	128
7.4.2. Input Validation and Filtering	128

7.4.3. PII Detection and Redaction	130
7.4.4. Output Constraints and Safety Checks	130
7.4.5. Content Moderation Integration.....	131
7.5. Detecting Hallucinations and Drift	133
7.5.1. What Are Hallucinations?	134
7.5.2. Fact-Checking with Retrieval.....	134
7.5.3. Consistency Checking	135
7.5.4. Confidence Scoring.....	136
7.5.5. Mitigation: Abstain, Ground, or Escalate	137
7.6. Cost-Quality Trade-offs	138
7.6.1. Model Selection Matrix	139
7.6.2. Cascading Model Approach	140
7.6.3. Token Budget Management.....	142
7.7. Continuous Monitoring and Feedback Loops	143
7.7.1. Production Metrics Dashboard.....	143
7.7.2. Logging and Observability	143
7.7.3. Feedback Collection Patterns.....	144
7.7.4. A/B Testing for Improvements	146
7.8. Summary	147
7.9. Reflection Questions	148
7.10. Additional Resources.....	148
8. Workflow Orchestration.....	149
8.1. Module Overview	150
8.2. Introduction to Workflow Orchestration	150
8.2.1. Why Simple Agents Aren't Enough	150
8.2.2. What Is Workflow Orchestration?	150
8.2.3. Real-World Workflow Examples.....	151
8.3. Sequential, Parallel, and Conditional Patterns	151
8.3.1. Sequential Execution Pattern	152
8.3.2. Parallel Execution Pattern.....	153
8.3.3. Conditional Execution Pattern.....	154
8.3.4. Combining Patterns.....	155

8.4. Graphs and Directed Workflows.....	156
8.4.1. Directed Acyclic Graphs (DAGs).....	156
8.4.2. Implementing Graph-Based Workflows.....	157
8.4.3. Workflow Execution Engines.....	160
8.5. Passing Context Between Steps.....	161
8.5.1. The Context Object Pattern.....	161
8.5.2. Dependency Injection for Context.....	162
8.5.3. Immutable vs. Mutable State.....	164
8.6. Error Propagation and Recovery.....	164
8.6.1. Handling Individual Step Failures.....	164
8.6.2. Partial Success and Graceful Degradation.....	166
8.6.3. Error Recovery Strategies.....	167
8.7. Checkpointing and Resumption.....	168
8.7.1. Why Checkpointing Matters.....	168
8.7.2. Implementing Checkpoints.....	168
8.7.3. Resumable Workflow Pattern.....	169
8.7.4. Idempotent Operations.....	171
8.8. Retry and Fallback Strategies.....	171
8.8.1. Exponential Backoff for Retries.....	171
8.8.2. Circuit Breaker Pattern.....	172
8.8.3. Fallback Actions and Alternate Paths.....	175
8.8.4. Designing Fault-Tolerant Workflows.....	176
8.9. Summary.....	177
8.10. Reflection Questions.....	177
8.11. Additional Resources.....	177
9. Multi-Agent Systems.....	179
9.1. Module Overview.....	180
9.2. Introduction to Multi-Agent Systems.....	180
9.2.1. What Are Multi-Agent Systems?.....	180
9.2.2. Example: Customer Support System.....	180
9.2.3. Benefits of Multi-Agent Systems.....	181
9.2.4. Challenges and Trade-offs.....	181

9.3. Single Agent vs. Multi-Agent Considerations.....	181
9.3.1. When a Single Agent Is Better.....	181
9.3.2. When Multiple Agents Add Value	182
9.3.3. Decision Framework.....	182
9.4. Agent Delegation and Escalation.....	183
9.4.1. Basic Delegation Pattern.....	183
9.4.2. Hierarchical Escalation	185
9.4.3. Chaining Multiple Specialists	186
9.5. Shared vs. Isolated Context	188
9.5.1. Context Isolation Strategy.....	189
9.5.2. Shared Context Strategy.....	189
9.5.3. Hybrid Approach	190
9.6. Coordination and Handoff Protocols	191
9.6.1. Structured Handoff Messages	191
9.6.2. Request-Response Protocol.....	193
9.6.3. Broadcasting to Multiple Agents.....	194
9.7. Designing Agent Roles & Teamwork	195
9.7.1. Defining Clear Agent Roles.....	195
9.7.2. Role Definition Example	196
9.7.3. Complementary Agent Skills	196
9.7.4. Sequential Pipeline	197
9.7.5. Hub-and-Spoke.....	197
9.7.6. Peer-to-Peer	197
9.8. Debugging Multi-Agent Interactions.....	198
9.8.1. Common Multi-Agent Issues	198
9.8.2. Tracing Multi-Agent Execution.....	198
9.8.3. Debugging Techniques	199
9.9. Summary	199
9.10. Reflection Questions	200
9.11. Additional Resources.....	200
10. Testing and Production Deployment.....	201
10.1. Module Overview	202

10.2. Introduction to Testing & Deployment	202
10.2.1. Unique Challenges: Non-Determinism.....	202
10.2.2. Unique Challenges: External Dependencies	202
10.2.3. Unique Challenges: Complex Behavior.....	202
10.2.4. Testing Strategy: Unit Testing.....	203
10.2.5. Testing Strategy: Integration Testing.....	203
10.2.6. Testing Strategy: Production Monitoring	203
10.2.7. Production Concerns: Performance & Reliability.....	203
10.2.8. Production Concerns: Cost & Security.....	204
10.3. Testing Agents with Simulated Backends	204
10.3.1. Why Mock LLM Responses?	204
10.3.2. Benefits of Mocking the LLM	205
10.3.3. Using TestModel from Pydantic AI	205
10.3.4. Testing Tool Calls	206
10.4. Deterministic Testing for Non-Deterministic Systems	208
10.4.1. Testing Pattern: Pattern Matching.....	208
10.4.2. Testing Pattern: Property-Based.....	208
10.4.3. Testing Agent Logic in Isolation	209
10.4.4. Snapshot Testing for Prompts	210
10.5. Capturing & Inspecting Agent Runs	211
10.5.1. Recording Execution Traces	211
10.5.2. Step-by-Step Replay and Analysis.....	213
10.5.3. Using Pydantic AI's Built-in Tracing.....	215
10.6. Observability and Tracing Integration	216
10.6.1. Structured Logging for Agents.....	217
10.6.2. Metrics and Monitoring	217
10.6.3. OpenTelemetry Integration.....	218
10.7. Serving Agents via FastAPI.....	220
10.7.1. Creating Agent Web Service Endpoints.....	220
10.7.2. Handling Concurrency and Errors.....	221
10.7.3. Streaming Responses.....	222
10.8. Usage Tracking and Cost Management.....	223
10.8.1. Request and Token Tracking.....	224

10.8.2. Cost Control Measures.....	225
10.9. Security & Deployment Best Practices	227
10.9.1. Securing Agent Endpoints	227
10.9.2. Input Validation and Sanitization	228
10.9.3. Scaling Strategies Overview	229
10.9.4. Scaling Strategies: Caching	230
10.10. Summary	232
10.11. Reflection Questions.....	232
10.12. Additional Resources.....	233
11. Course Conclusion.....	234
11.1. Outline.....	235
11.2. Summary.....	235
11.3. Resources.....	235
11.4. Next Steps	236
11.5. Reflection Questions	237
11.6. Key Takeaways.....	237

GAI-2109: Practical Development of Agentic AI Applications

LESSON 1

- ✓ Identify the core technologies, frameworks, and tools used throughout this course
- ✓ Describe the structure of the three-day program and what each module covers
- ✓ Explain the role of agentic AI in modern software development
- ✓ Identify the prerequisites and environment requirements for hands-on labs

1.1. Outline

- Course agenda and module overview
- Course overview and what you will build
- Prerequisites and prior knowledge
- Environment setup and verification

1.2. Agenda

1

Anatomy of an AI Agent

Agent architecture, design principles, type safety, and the Pydantic AI framework.

2

Structured Outputs and Result Validation

Defining output schemas with Pydantic models and handling validation failures.

3

Tool Integration and Design

Extending agent capabilities with custom tools and integrating external APIs.

4

Dependencies and Runtime Context

Dependency injection, resource management, and clean architecture for agents.

5

Conversations, Memory, and State

Message history, sliding window memory, streaming responses, and human-in-the-loop patterns.

6

Evaluation and Guardrails

Testing agent quality, cost management, and enforcing output guardrails.

7

8

Workflow Orchestration

Sequential, parallel, and conditional multi-step agent pipelines with fault tolerance.

Multi-Agent Systems

Manager-worker patterns, task delegation, and coordination between specialized agents.

9**Testing and Production****Deployment**

Unit testing with simulated backends, FastAPI integration, and production monitoring.

1.3. Course Overview

This course teaches Python developers and data scientists how to build production-grade AI agents using the **Pydantic AI** framework.

1.4. What Is Agentic AI?

An **AI agent** is a large language model paired with the surrounding machinery that lets it act, not just respond.

- **LLM** — produces text and reasoning
- **Tools** — Python functions the model can invoke (data lookups, API calls, calculations)
- **Memory** — conversation history and persisted state
- **Validation** — typed schemas (Pydantic models) that constrain the model's output
- **Control loop** — code that runs the model, dispatches tool calls, and decides when the task is done

A direct LLM call answers a question; an agent decomposes a task, calls tools to gather information or take action, validates the result against a schema, and continues until the goal is met.

The same LLM (e.g., GPT-5.4-mini) can answer a single prompt directly **or** sit at the center of an agent that runs through several rounds of tool calls before responding. The difference is the surrounding code, not the model — Pydantic AI is the framework that supplies that surrounding code in a typed, testable form. The course builds that picture incrementally: this opening chapter frames the category, then the first three modules introduce agents, structured outputs, and tools; later modules add memory, evaluation, orchestration, and multi-agent coordination.

1.4.1. What You Will Build

Across three days of instruction and hands-on labs, you will progress from foundational agent concepts to deploying fully functional multi-agent systems.

- **Day 1** — Agent foundations, structured outputs, and tool integration
- **Day 2** — Dependencies, memory, conversation state, and evaluation
- **Day 3** — Workflow orchestration, multi-agent patterns, and production deployment

1.4.2. About This Course

Framework

- Pydantic AI — type-safe, production-ready agent development
- Model-agnostic — works with OpenAI, Anthropic, Google, and others
- FastAPI for agent deployment and serving

Approach

- Every module paired with hands-on lab exercises
- Real-world business scenarios throughout
- Emphasis on type safety, testability, and clean architecture

1.5. Prerequisites

What this course assumes coming in, and what it does not.

1.5.1. What You Should Already Know

- Python 3.10+ — comfortable writing classes, async functions, and type hints
- REST APIs — experience calling external APIs and handling JSON responses
- Basic LLM concepts — familiarity with prompts, completions, and API keys

1.5.2. What You Do Not Need

- Prior experience with AI agent frameworks
- Knowledge of specific LLM providers beyond basic API usage
- Machine learning or model training background

1.6. Environment Setup

Your lab environment is provisioned in advance — here's what's installed and how each lab activates it.

1.6.1. Pre-Configured Lab Environment

Your lab environment has been configured in advance on virtual machines. The following are already installed and ready to use:

- Python 3.12.10 (pre-installed)
- Astral **uv** for project-scoped dependency management
- VS Code with recommended extensions
- Each lab's dependencies are pinned in its `pyproject.toml` and installed by running `uv sync` from the lab's directory

Your instructor will provide the necessary API keys and credentials prior to the first lab.

1.7. Reflection Questions

- What types of tasks in your current work could benefit from agentic AI applications?
- How does the model-agnostic approach of Pydantic AI compare to other frameworks you may have encountered?
- What challenges do you anticipate when moving from simple LLM prompts to full agent architectures?

1.8. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- Pydantic documentation: <https://docs.pydantic.dev>
- Python documentation: <https://docs.python.org/3/>
- FastAPI documentation: <https://fastapi.tiangolo.com>

Anatomy of an AI Agent

LESSON 2

- ✓ Define the concept of an AI agent and describe the core components that constitute the anatomy of an AI agent
- ✓ Explain key agent design principles and identify practical use cases where agentic AI applications are beneficial
- ✓ Explain the importance of type safety and how structured agent frameworks help enforce data validation
- ✓ Implement a basic AI agent using the Pydantic AI framework and run this agent to perform simple tasks
- ✓ Differentiate between synchronous and asynchronous execution patterns for agents and discuss when to use each mode

2.1. Module Overview

- What is an AI agent
- Agent design principles
- Practical use cases for AI agents
- Type safety with structured frameworks
- Creating and running a basic agent
- System prompts and model configuration
- Synchronous and asynchronous execution

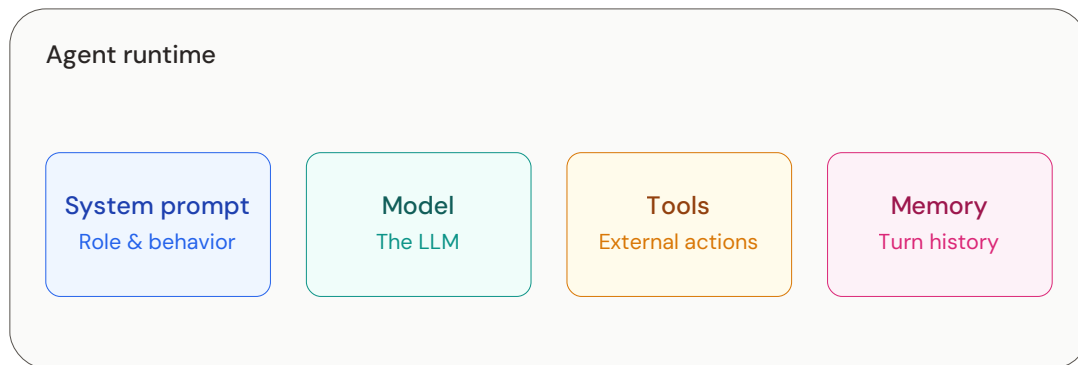
2.2. What is an AI Agent?

An autonomous system that uses LLMs to perceive its environment, make decisions, and act toward a goal.

2.3. Characteristics of AI Agents

- **Autonomy** – Operates with minimal human input
- **Goal-Oriented** – Works toward specific objectives
- **Adaptive** – Adjusts behavior to context
- **Interactive** – Supports multi-turn workflows

2.4. Core Components of an Agent



2.5. Supporting Infrastructure

- Input/output validation
- Error handling and recovery
- Observability and logging
- Security and guardrails

2.6. Agent Design Principles

Principles that keep agents reliable, maintainable, and predictable.

- Clear role definition
- Step-by-step reasoning
- Reliability through structure

2.6.1. Clear Role Definition

Define the agent's role and responsibilities in the system prompt.

- Specify expertise and limitations
- Establish tone and style

- Set behavior boundaries
- Define success criteria

2.6.2. Step-by-Step Reasoning

Guide the agent to work through problems methodically.

- Break complex tasks into steps
- Ask for explicit reasoning
- Use structured output formats
- Validate intermediate results

2.6.3. Reliability Through Structure

Use structured frameworks for consistent, validated outputs.

- Define input/output schemas
- Implement type checking
- Handle errors gracefully
- Test with known inputs

2.7. Practical Use Cases for AI Agents

Common patterns where agents deliver value today:

- Data analysis assistants
- Customer support bots
- Content generation and editing
- Workflow automation

2.7.1. Data Analysis Assistants

Agents that help users explore and understand data.

- Generate insights from datasets

- Create visualizations and reports
- Answer questions about trends
- Recommend next analysis steps

2.7.2. Customer Support Bots

Automated systems that handle customer inquiries.

- Answer frequently asked questions
- Troubleshoot common issues
- Escalate to humans when needed
- Provide 24/7 availability

2.7.3. Content Generation and Editing

Agents that assist with writing tasks.

- Draft documents and reports
- Summarize long texts
- Edit and improve content
- Maintain consistent style

2.7.4. Workflow Automation

Agents that orchestrate multi-step processes.

- Coordinate tools and services
- Handle conditional logic
- Monitor progress and errors
- Integrate with existing systems

2.8. Why Type Safety Matters

Type safety catches errors early and ensures reliable agent behavior.

- Catch errors at development time
- Ensure data integrity
- Improve maintainability
- Enable better IDE tooling

2.9. Pydantic AI Overview

A Python framework for type-safe, production-ready AI agents.

- Built on Pydantic models
- Supports OpenAI, Anthropic, Google
- Structured outputs and function calling
- Async-first, with sync support

2.10. Type Safety in Action

Pydantic AI ties the model's output to a schema. Invalid responses are rejected and retried.

```
class QueryResult(BaseModel):
    sentiment: str = Field(description="positive, negative, or neutral")
    confidence: float = Field(ge=0.0, le=1.0)
    summary: str = Field(max_length=200)

agent = Agent(model, output_type=QueryResult)
result = agent.run_sync("Summarize: '...')
print(result.output.sentiment, result.output.confidence)
```

The structured-output contract is the load-bearing teaching point of this module: agents become reliable when their output shape is a typed schema, not free-form text. Later chapters build on this — tool inputs and outputs are validated the same way, and the **Structured Outputs and Result Validation** module covers retry loops, union types, and validators in depth.

```
import os
from pydantic import BaseModel, Field
from pydantic_ai import Agent

class QueryResult(BaseModel):
    """Schema the agent must populate. Pydantic validates each field."""
    sentiment: str = Field(description="positive, negative, or neutral")
    confidence: float = Field(ge=0.0, le=1.0)
    summary: str = Field(max_length=200)

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=QueryResult, # constrain + validate the model's output
)

result = agent.run_sync("Summarize: 'The product works well but shipping was slow.'")
print(result.output.sentiment, result.output.confidence)
# Pydantic raises ValidationError if confidence > 1.0 or summary > 200 chars,
# and Pydantic AI re-prompts the model to fix the response.
```

2.11. Creating a Basic Agent

Let's build your first AI agent with Pydantic AI, starting from install to running a query.

2.11.1. Setting Up Your Environment

This course standardizes on **Python 3.12.10** and **Astral uv** for dependency management. Both are pre-installed in the lab environment; the labs run from each lab's `pyproject.toml` and `uv sync` provisions a venv automatically.

```
# Each lab carries its own pyproject.toml – uv sync installs
everything.
uv sync

# Add a dependency to the active project (rather than `pip
install`).
uv add pydantic-ai
uv add openai
```

The `pip install pydantic-ai` workflow used elsewhere on the web targets a system Python; in this course every lab is its own uv-managed project so dependencies stay scoped per lab and the lockfile pins exact versions across student machines. If you see `pip install` in older tutorials, the equivalent uv command is `uv add <package>` from the project root.

2.11.2. Defining a Simple Agent

Create an agent with a system prompt.

```
import os
from pydantic_ai import Agent

# Create an agent with a system prompt
agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    system_prompt='You are a helpful assistant that answers
questions clearly and concisely.'
)
```

2.11.3. Running the Agent Synchronously

Execute the agent with a user query using `run_sync`.

```
# Run the agent synchronously
result = agent.run_sync('What is the capital of France?')
print(result.output)
# Output: The capital of France is Paris.
```

2.11.4. Agent Configuration

Tune model behavior via `model_settings`.

```
import os

# Configure model settings
agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    system_prompt='You are a helpful assistant.',
    model_settings={
        'temperature': 0.7,
        'max_tokens': 500
    }
)
```

2.12. System Prompts

System prompts define agent behavior, personality, and constraints.

- Static: fixed across all runs
- Dynamic: adapt to runtime context

2.12.1. Static System Prompts

Fixed instructions for consistent behavior across all interactions.

```
import os

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    system_prompt='You are a professional data analyst. '
                  'Provide clear, accurate answers based on data.'
)
```

2.12.2. Dynamic System Prompts

Use the `@agent.system_prompt` decorator to build prompts from runtime context.

```
agent = Agent(model, deps_type=dict)

@agent.system_prompt
def generate_prompt(ctx: RunContext[dict]) -> str:
    user_name = ctx.deps.get('user_name', 'User')
    return f"You are a helpful assistant for {user_name}."

result = agent.run_sync('Hello', deps={'user_name': 'Alice'})
```

The `system_prompt=` constructor argument expects a string (or sequence of strings) — it does **not** accept a function. To build a prompt dynamically from `deps`, register a function with the `@agent.system_prompt` decorator as shown above. The Building a Basic AI Agent lab's Challenge 1 uses this same pattern.

The full version with imports and a more elaborate prompt that incorporates today's date:

```

import os
from datetime import datetime
from pydantic_ai import Agent, RunContext

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    deps_type=dict,
)

@agent.system_prompt
def generate_prompt(ctx: RunContext[dict]) -> str:
    user_name = ctx.deps.get('user_name', 'User')
    current_date = datetime.now().strftime('%Y-%m-%d')
    return (
        f"You are a helpful assistant for {user_name}. "
        f"Today's date is {current_date}. "
        "Provide information relevant to the current context."
    )

# Pass deps at run time so the prompt is built from runtime values.
result = agent.run_sync('What can you help me with?',
                        deps={'user_name': 'Alice'})

```

2.13. Model Configuration

Pydantic AI supports multiple LLM providers and configurable parameters for quality, cost, and performance.

2.13.1. Supported LLM Providers

A model-agnostic interface — swap providers without rewriting code. Each provider has its own prefix on the model string, and Pydantic AI's `KnownModelName` catalog is the authoritative list of currently supported identifiers (verify against your installed version with `from pydantic_ai.models import KnownModelName`).

```
import os
from pydantic_ai import Agent

agent_openai = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))
agent_anthropic = Agent('anthropic:claude-sonnet-4-5')
agent_google = Agent('google:gemini-2.5-flash')
```

Model identifiers move quickly. Pin a date-stamped variant (`anthropic:claude-sonnet-4-5-20250929` , for example) when you need deterministic test runs across deployments, and prefer the catalog's `-latest` aliases (`gemini-flash-latest` , `gemini-flash-lite-latest`) only when you've decided you want automatic upgrades. Pydantic AI 1.97 routes Google models through two prefixes — `google` for the consumer Generative Language API and `google-cloud` for Vertex AI. Earlier versions used `google-gla` / `google-vertex` ; those prefixes are deprecated in 1.97 and removed in v2.0, so verify the current names against `KnownModelName` for your installed version.

2.13.2. Model Parameters

Tune parameters to balance creativity, consistency, and length.

```
import os

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    model_settings={
        'temperature': 0.3, # Lower = more focused
        'max_tokens': 1000
    }
)
```

2.13.3. Key Model Parameters

- **Temperature** – Randomness (0.0 deterministic, 1.0 creative)
- **Max Tokens** – Limits response length
- **Top P** – Controls diversity via nucleus sampling

2.14. Sync vs. Async Execution

Choosing the right execution pattern affects performance and scalability.

- **Sync**: blocking, simple, sequential
- **Async**: non-blocking, concurrent

2.14.1. Synchronous Execution

Blocking calls that wait for the agent to complete.

```
# Synchronous execution
result = agent.run_sync('What is machine learning?')
print(result.output)
```

2.14.2. When to Use Sync

- Simple scripts and prototypes
- Single-user applications
- Debugging and testing
- When concurrency is not needed

2.14.3. Asynchronous Execution

Non-blocking calls that allow concurrent operations.

```
import asyncio

async def run_agent():
    result = await agent.run('Explain neural networks.')
    print(result.output)

# Run the async function
asyncio.run(run_agent())
```

2.14.4. When to Use Async

- Web applications and APIs
- Concurrent request processing
- Real-time streaming responses
- High-throughput scenarios

Key Differences:

Sync execution is blocking and sequential (one request at a time), while async is non-blocking and allows multiple concurrent requests. Use sync for simple scripts and prototypes; use async for production APIs and high-load scenarios.

2.14.5. Streaming Outputs

Stream responses token-by-token for real-time feedback.

```
async def stream_response():
    async with agent.run_stream('Tell me a story.') as stream:
        async for chunk in stream.stream():
            print(chunk, end='', flush=True)

asyncio.run(stream_response())
```

2.15. Summary

In this module we covered:

- The definition and core components of AI agents
- Agent design principles and practical use cases
- Type safety and structured frameworks like Pydantic AI
- Creating and running a basic agent with system prompts
- Configuring model backends and parameters
- Synchronous vs. asynchronous execution patterns

2.16. Reflection Questions

- When would you choose asynchronous execution over synchronous execution for an agent, and why?
- How does defining a clear system prompt improve an agent's reliability and usefulness?
- What are the trade-offs between using a lightweight model versus a frontier model for a basic agent?
- How might type safety prevent bugs that would be difficult to catch in a free-form text-based agent?

2.17. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- Pydantic documentation: <https://docs.pydantic.dev>
- Python asyncio documentation: <https://docs.python.org/3/library/asyncio.html>
- OpenAI API documentation: <https://platform.openai.com/docs>
- Anthropic API documentation: <https://docs.anthropic.com>

Structured Outputs and Result Validation

LESSON 3

- ✓ Explain the importance of structured outputs for AI agent responses and how they improve reliability
- ✓ Design Pydantic models to define the expected schema of an agent's output
- ✓ Implement output validation using Pydantic AI, ensuring agent responses conform to defined schemas
- ✓ Apply validation constraints to enforce business rules in agent outputs
- ✓ Design strategies to handle cases where agent output fails validation

3.1. Module Overview

- Structured outputs in AI agents
- Defining output schemas with Pydantic models
- Result validation with Pydantic AI
- Using validators to constrain responses
- Handling validation failures and retries
- Union types for multi-format responses
- Practical data extraction patterns

3.2. Structured Outputs in AI Agents

Free-form LLM text is hard to integrate with production systems.

- Structured outputs give reliable, type-safe data
- Downstream apps can process them consistently

3.2.1. The Problem with Free-Form Text

- Hard to parse programmatically
- Inconsistent formats across responses
- Hard to validate completeness
- Complex downstream integration

3.2.2. Benefits of Structured Outputs

Technical Benefits

- Easy parsing and validation
- Type safety and IDE support
- Consistent data formats
- Automated error detection

Business Benefits

- Reliable system integration
- Predictable app behavior
- Reduced debugging time
- Better user experience

3.2.3. Common Structured Formats

- **JSON** – Universal data interchange format
- **Pydantic Models** – Python objects with validation
- **Database Records** – Direct persistence
- **API Responses** – Standardized endpoint outputs

3.3. Defining Output Schemas with Pydantic Models

- Pydantic models describe an agent's output shape
- They act as documentation and runtime validation
- Schemas ensure data arrives in the expected format

3.3.1. Basic Pydantic Model

Define the expected structure of agent output.

```
from pydantic import BaseModel

class SummaryOutput(BaseModel):
    title: str
    key_points: list[str]
    word_count: int
    sentiment: str
```

3.3.2. Using the Model with an Agent

Pass the schema as `output_type=` and the agent returns a typed instance.

```
agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=SummaryOutput,
    system_prompt='Analyze the given text and provide a structured summary.',
)

result = agent.run_sync('Analyze this article: ...')
summary = result.output
print(summary.title, summary.key_points)
```

The full version with imports:

```
import os
from pydantic_ai import Agent

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=SummaryOutput,
    system_prompt='Analyze the given text and provide a structured
summary.',
)

result = agent.run_sync('Analyze this article: ...')
summary = result.output
print(summary.title)          # Type-safe access – IDE
                             # autocompletes .title
print(summary.key_points)
```

The IDE benefit is real and tangible — `summary.<TAB>` produces the schema's field list because `output_type=SummaryOutput` informs the static type of `result.output`. This is the load-bearing payoff of typed outputs, not just runtime validation.

3.3.3. Choosing Appropriate Fields

- Include only information needed for the task
- Use appropriate data types (str, int, float, bool)
- Consider optional fields for uncertain data
- Use enumerations for restricted choices

3.3.4. Enumerations and Optional Fields

Enums constrain choices; `Optional` (or `| None`) declares uncertain data.

```
class Sentiment(str, Enum):  
    POSITIVE = 'positive'  
    NEUTRAL = 'neutral'  
    NEGATIVE = 'negative'  
  
class AnalysisOutput(BaseModel):  
    sentiment: Sentiment  
    confidence_score: float  
    topics: list[str]  
    summary: Optional[str] = None
```

The full version with the necessary imports:

```
from enum import Enum  
from typing import Optional  
  
class Sentiment(str, Enum):  
    POSITIVE = 'positive'  
    NEUTRAL = 'neutral'  
    NEGATIVE = 'negative'  
  
class AnalysisOutput(BaseModel):  
    sentiment: Sentiment  
    confidence_score: float  
    topics: list[str]  
    summary: Optional[str] = None
```

In Python 3.10+ the modern equivalent is `summary: str | None = None` — same semantics as `Optional[str]`, no `typing` import needed. Both spellings are valid; pick one and use it consistently.

3.3.5. Nested Models

Create complex schemas with nested structures.

```
class Author(BaseModel):  
    name: str  
    email: str  
  
class Article(BaseModel):  
    title: str  
    author: Author  
    publication_date: str  
    tags: list[str]  
    word_count: int
```

3.4. Result Validation with Pydantic AI

- Pydantic AI validates outputs against your schema
- Invalid data raises clear validation errors
- Errors pinpoint exactly what went wrong

3.4.1. Automatic Validation

Constraints in the schema become validation rules the agent must satisfy.

```
class ProductReview(BaseModel):  
    rating: int = Field(ge=1, le=5) # between 1 and 5  
    review_text: str  
    recommend: bool  
  
agent = Agent(..., output_type=ProductReview)  
result = agent.run_sync('Review this product: ...')
```

The full pattern with imports — `Field(ge=1, le=5)` is the load-bearing line, declaring a constraint Pydantic enforces at parse time:

```
import os
from pydantic import BaseModel, Field
from pydantic_ai import Agent

class ProductReview(BaseModel):
    rating: int = Field(ge=1, le=5)
    review_text: str
    recommend: bool

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=ProductReview,
)

# The agent will ensure the output matches this schema
result = agent.run_sync('Review this product: ...')
review = result.output
```

Pydantic AI doesn't just **validate** — when the model returns invalid data, the framework re-prompts with the validation error so the model can correct itself, up to a configurable retry limit. This is what makes typed outputs **reliable**, not just type-safe.

3.4.2. Validation Errors

When the LLM produces invalid output, Pydantic raises an error.

```
from pydantic import ValidationError

try:
    result = agent.run_sync('Generate a product review')
    review = result.output
except ValidationError as e:
    print(f'Validation failed: {e}')
    # Handle the error appropriately
```

3.4.3. Field Validation

Pydantic validates field types and constraints automatically.

Validation Type	Example	Error Condition
Type checking	<code>rating: int</code>	LLM returns a string
Range constraints	<code>Field(ge=1, le=5)</code>	Value outside range
Required fields	<code>name: str</code>	Field is missing
String patterns	<code>Field(pattern=r'^\d{3}-\d{4}\$')</code>	Pattern doesn't match

The `pattern=` keyword is the Pydantic v2 spelling. Older tutorials and the v1 Pydantic series used `regex=` for the same constraint; that name was removed in v2 and raises `PydanticUserError` if you copy it forward. Whenever you see `Field(regex=...)` in older code, rename it to `Field(pattern=...)`.

3.5. Using Validators to Constrain Responses

- Validators enforce business rules beyond type checks
- Keep agent outputs aligned with domain requirements
- Protect data quality at the boundary

3.5.1. Built-in Pydantic Validators

Use `Field` constraints to enforce rules.

```
from pydantic import BaseModel, Field

class Report(BaseModel):
    title: str = Field(min_length=10, max_length=100)
    summary: str = Field(min_length=50, max_length=500)
    confidence: float = Field(ge=0.0, le=1.0)
    tags: list[str] = Field(min_length=1, max_length=5)
```

3.5.2. Custom Validators

Use `@field_validator` for rules that go beyond `Field` constraints.

```
class DataAnalysis(BaseModel):
    insights: list[str]
    recommendations: list[str]

    @field_validator('insights')
    @classmethod
    def insights_not_empty(cls, v):
        if not v:
            raise ValueError('At least one insight is required')
        return v
```

The full version with a second validator showing per-item iteration:

```
from pydantic import BaseModel, field_validator

class DataAnalysis(BaseModel):
    insights: list[str]
    recommendations: list[str]

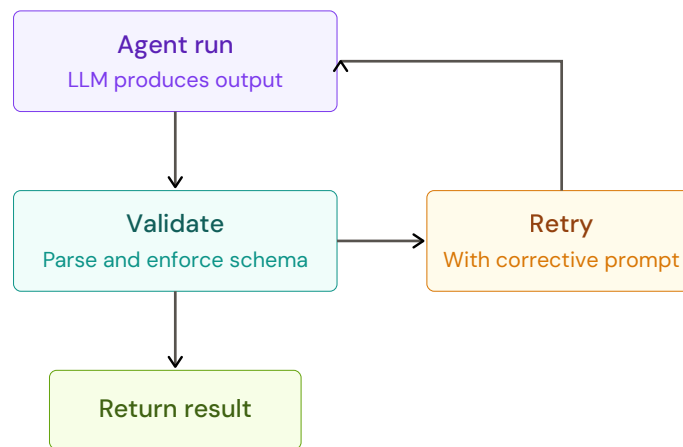
    @field_validator('insights')
    @classmethod
    def insights_not_empty(cls, v):
        if not v:
            raise ValueError('At least one insight is required')
        return v

    @field_validator('recommendations')
    @classmethod
    def check_recommendation_length(cls, v):
        for rec in v:
            if len(rec) < 10:
                raise ValueError('Recommendations must be at least
10 characters')
        return v
```

The `@classmethod` decorator below `@field_validator` is required by Pydantic v2 — leaving it off doesn't error at import time but does fire a deprecation warning, and v3 will reject it outright.

Business Rule Validation: You can also enforce domain-specific constraints like $\text{profit} = \text{revenue} - \text{expenses}$ using validators. Use the `info` parameter to access other field values for cross-field validation.

3.6. Handling Validation Failures and Retries



3.6.1. Why Retries Matter

- Validation failures are inevitable with LLMs
- Use retries and fallbacks to stay robust
- Keep apps running even when output is invalid

3.6.2. Basic Retry Logic

Wrap the agent call in a retry loop bounded by `max_retries`.

```
for attempt in range(max_retries):
    try:
        result = agent.run_sync('Generate analysis')
        break
    except ValidationError:
        if attempt == max_retries - 1:
            raise
```

The full version with the import and progress-printing on retry:

```
from pydantic import ValidationError

max_retries = 3
for attempt in range(max_retries):
    try:
        result = agent.run_sync('Generate analysis')
        analysis = result.output
        break # Success
    except ValidationError as e:
        if attempt == max_retries - 1:
            print('Max retries reached, validation failed')
            raise
        print(f'Attempt {attempt + 1} failed, retrying...')
```

Pydantic AI already does this internally for you — `Agent` accepts a `retries=` parameter, default 1, that controls how many re-prompts the framework attempts on validation failure before giving up. Reach for an outer retry loop only when you want to **re-roll the whole call** (e.g., switch models or providers between attempts) rather than just re-prompt the same agent.

3.6.3. Retry with Adjusted Prompts

Provide feedback to help the agent correct its output.

```
def run_with_retry(agent, prompt, max_retries=3):
    for attempt in range(max_retries):
        try:
            result = agent.run_sync(prompt)
            return result.output
        except ValidationError as e:
            if attempt == max_retries - 1:
                raise
            # Add validation error to prompt
            prompt = f'{prompt}\n\nPrevious attempt failed: {e}\nPlease correct the output.'
```

3.6.4. Fallback Responses

When validation keeps failing, return a safe default rather than raising.

```
def run_with_fallback(agent, prompt, fallback_data):  
    try:  
        return agent.run_sync(prompt).output  
    except ValidationError:  
        return fallback_data
```

The full pattern with an example fallback instance constructed against the schema:

```
def run_with_fallback(agent, prompt, fallback_data):  
    try:  
        result = agent.run_sync(prompt)  
        return result.output  
    except ValidationError:  
        print('Using fallback response')  
        return fallback_data  
  
# Usage  
fallback = AnalysisOutput(  
    sentiment=Sentiment.NEUTRAL,  
    confidence_score=0.0,  
    topics=[],  
    summary='Analysis unavailable',  
)  
  
analysis = run_with_fallback(agent, 'Analyze this text...',  
                             fallback)
```

The fallback must satisfy the schema itself — that's why it's constructed via `AnalysisOutput(...)` rather than as a raw dict. This keeps the return type of `run_with_fallback` consistent (always `AnalysisOutput`), so callers don't need to branch on success vs. failure to know what type they got back.

Partial Validation: For some use cases, you may want to accept partial results when full validation fails. You can design models with all optional fields and a `validation_errors` list to capture what failed. Discuss trade-offs between strict validation and graceful degradation.

3.7. Union Types for Multi-Format Responses

- Agents can return different shapes per request
- Union types keep flexibility with type safety
- Useful when inputs or needs vary

3.7.1. Defining Union Types

`output_type=Union[A, B]` lets the agent pick a shape per response.

```
class ShortAnswer(BaseModel):
    answer: str
    confidence: float

class DetailedReport(BaseModel):
    executive_summary: str
    detailed_analysis: str
    recommendations: list[str]
    confidence: float

agent = Agent(..., output_type=ShortAnswer | DetailedReport)
```

The full version with imports. Python 3.10+ accepts the `A | B` syntax in `output_type=`; older code uses `Union[A, B]` from `typing`.

```
import os
from typing import Union
from pydantic_ai import Agent

class ShortAnswer(BaseModel):
    answer: str
    confidence: float

class DetailedReport(BaseModel):
    executive_summary: str
    detailed_analysis: str
    recommendations: list[str]
    confidence: float

# Agent can return either format
agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=Union[ShortAnswer, DetailedReport],
)
```

This is exactly the same pattern the **Conversations, Memory, and State** module uses for clarification flows (`ClarificationNeeded | FinalResponse`) — the union output is what lets the agent decide between "I have an answer" and "I need to ask the user something" without resorting to free-form flags.

3.7.2. Handling Union Responses

Process different response types appropriately.

```
result = agent.run_sync('Analyze this situation...')

if isinstance(result.output, ShortAnswer):
    print(f'Quick answer: {result.output.answer}')
elif isinstance(result.output, DetailedReport):
    print(f'Summary: {result.output.executive_summary}')
    print(f'Recommendations: {result.output.recommendations}')
```

3.7.3. Use Cases for Union Types

- **Question complexity** – Short answers vs. detailed explanations
- **Data availability** – Full analysis vs. limited information
- **User preferences** – Brief vs. comprehensive responses
- **Error conditions** – Success vs. error response types

3.8. Practical Data Extraction Patterns

- Turn unstructured text into clean data
- Parse documents, emails, and notes
- Validate extracted fields automatically

3.8.1. Extracting Structured Data from Text

A typed schema turns "extract contact info" into a one-line agent call.

```
class ContactInfo(BaseModel):  
    name: str  
    email: str  
    phone: Optional[str] = None  
    company: Optional[str] = None  
  
agent = Agent(..., output_type=ContactInfo,  
               system_prompt='Extract contact information from the  
provided text.')  
  
contact = agent.run_sync(text).output
```

The full pattern with imports and a worked example showing what the agent extracts:

```
import os
from pydantic_ai import Agent

class ContactInfo(BaseModel):
    name: str
    email: str
    phone: Optional[str] = None
    company: Optional[str] = None

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=ContactInfo,
    system_prompt='Extract contact information from the provided
text.',
)

text = "John Doe works at Acme Corp. Reach him at john.doe@acme.com
or 555-1234."

contact = agent.run_sync(text).output
print(contact.name)      # 'John Doe'
print(contact.email)     # 'john.doe@acme.com'
print(contact.company)   # 'Acme Corp'
```

For higher-quality extraction, layer two patterns from earlier in the chapter on top:

- (a) a `@field_validator` to confirm the extracted email matches an email pattern;
- (b) `Optional[...]` defaults to `None` so the agent doesn't fabricate fields that aren't actually present in the source text.

Improving Extraction: Design clear prompts with extraction rules (e.g., "If a field is not present, leave it null", "Validate email format"). Add validators to ensure completeness and correctness of extracted data.

3.9. Summary

In this module we covered:

- The importance of structured outputs for reliable AI agents
- Defining output schemas using Pydantic models
- Automatic validation and error handling
- Custom validators for business rule enforcement
- Retry strategies and fallback mechanisms
- Union types for flexible response formats
- Practical data extraction patterns

3.10. Reflection Questions

- How does enforcing structured outputs change the way you design prompts for an agent?
- When would you use union types for agent responses instead of a single output schema?
- What is the right balance between strict validation and graceful degradation when handling agent outputs?
- How do custom validators help enforce business rules that built-in field constraints cannot cover?

3.11. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- Pydantic model documentation: <https://docs.pydantic.dev>
- Python typing module documentation: <https://docs.python.org/3/library/typing.html>
- OpenAI structured outputs documentation: <https://platform.openai.com/docs>

Tool Integration and Design

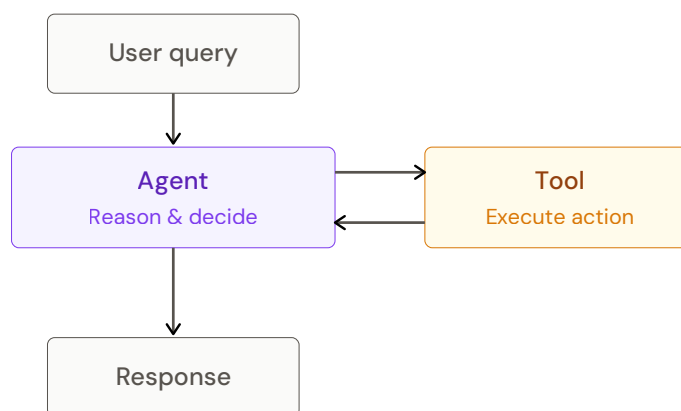
LESSON 4

- ✓ Describe how AI agents can use tools to extend their capabilities beyond text completion
- ✓ Implement a custom tool for an AI agent and explain the tool's lifecycle
- ✓ Configure tool registration and discovery so agents can identify and use available tools
- ✓ Apply patterns for passing runtime context into tools and implementing error handling

4.1. Module Overview

- Integrating tools with AI agents
- Tool design and lifecycle
- Tool registration and discovery
- Passing runtime context to tools
- Error handling and graceful degradation
- Common tool integration patterns

4.2. Integrating Tools with AI Agents



- LLMs cannot act or access live data
- Tools bridge that gap
- Agents invoke tools to complete tasks

4.3. Why Tools Matter

LLM Limitations

- No real-time data
- Unreliable calculations
- Cannot take actions
- Knowledge cutoffs

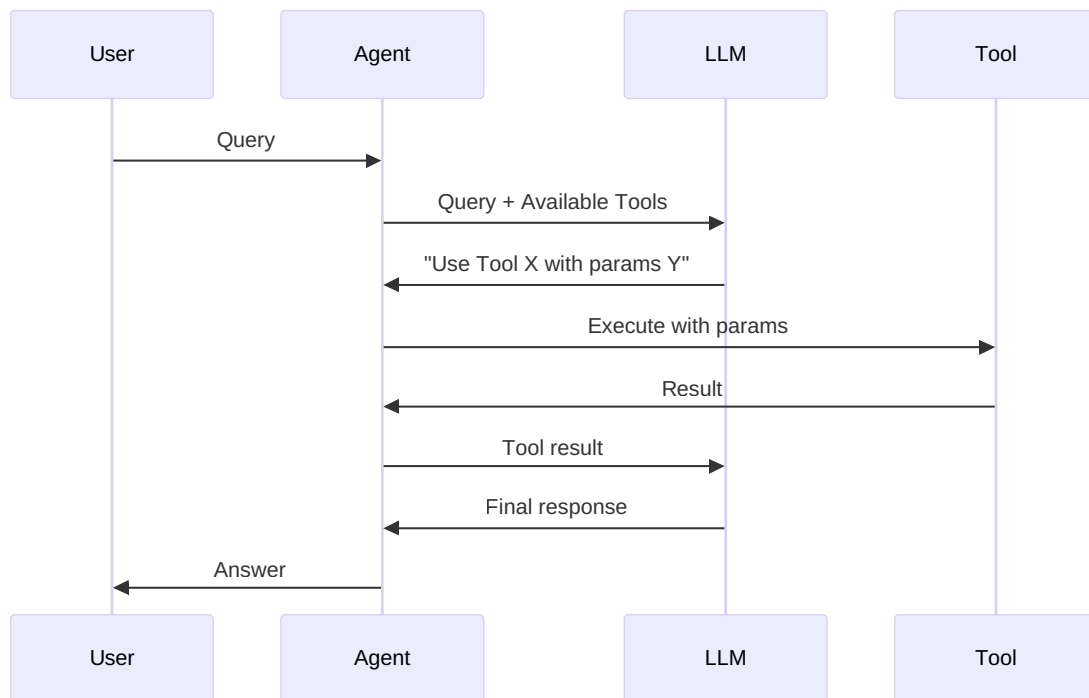
Tools Enable

- Real-time data access
- Accurate computations
- External API integration
- Action execution

4.4. Tool Use Cases

- **Calculations** – math, analysis
- **Data Lookup** – DB, API calls
- **File Operations** – read, write, process
- **External Services** – weather, stocks
- **Action Execution** – emails, records

4.5. How Tool Calling Works



4.6. Tool Design and Lifecycle

- Every tool has three phases
- Definition, validation, execution
- Clear lifecycle aids maintenance

4.6.1. Phase 1: Definition

Specify the tool as a Python function with clear inputs and outputs.

```

@agent.tool
def calculate_total(ctx: RunContext,
                    price: float,
                    quantity: int,
                    tax_rate: float = 0.08) -> float:
    """Calculate the total price including tax."""
    return round(price * quantity * (1 + tax_rate), 2)
  
```

The full pattern with imports and agent setup:

```
import os
from pydantic_ai import Agent, RunContext

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

@agent.tool
def calculate_total(ctx: RunContext,
                    price: float,
                    quantity: int,
                    tax_rate: float = 0.08) -> float:
    """Calculate the total price including tax."""
    subtotal = price * quantity
    total = subtotal * (1 + tax_rate)
    return round(total, 2)
```

The `@agent.tool` decorator inspects the function's signature and docstring at registration time. The signature becomes the tool's call schema (parameter names, types, defaults), and the docstring becomes the tool's **description** — what the LLM reads when deciding whether to call it. Both are load-bearing.

4.6.2. Phase 2: Validation

Ensure inputs and outputs meet expected formats.

```
from pydantic import BaseModel, Field

class CalculationInput(BaseModel):
    price: float = Field(gt=0)
    quantity: int = Field(gt=0)
    tax_rate: float = Field(ge=0, le=1)

@agent.tool
def validated_calculate(ctx: RunContext, params: CalculationInput) -> float:
    """Calculate total price with validated inputs."""
    subtotal = params.price * params.quantity
    return round(subtotal * (1 + params.tax_rate), 2)
```

4.6.3. Phase 3: Execution

The agent selects and invokes the tool during conversation.

```
result = agent.run_sync(
    'What is the total cost for 5 items at $29.99 each with 8% tax?'
)
print(result.output)
# The agent will call calculate_total(29.99, 5, 0.08)
# Output: "The total cost is $161.95"
```

4.7. Tool Registration and Discovery

- Agents must know what tools exist
- Registration exposes tools
- Discovery uses signatures and docstrings

4.7.1. Registering Tools with Agents

Two equivalent ways: the `@agent.tool` decorator or the `tools=[...]` constructor argument.

```
# Method 1: Decorator
@agent.tool
def search_database(ctx: RunContext, query: str) -> list[dict]:
    """Search the database for matching records."""
    ...

# Method 2: Constructor
agent_with_tools = Agent(model, tools=[another_tool])
```

The full version with both methods spelled out and the imports:

```
import os
from pydantic_ai import Agent

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

# Method 1: Decorator
@agent.tool
def search_database(ctx: RunContext, query: str) -> list[dict]:
    """Search the database for matching records."""
    # Implementation
    return results

# Method 2: Pass tools in the Agent constructor
def another_tool(ctx: RunContext, param: str) -> str:
    """Another tool function."""
    return result

agent_with_tools = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    tools=[another_tool],
)
```

Use the decorator when tools live in the same module as the agent. Use the constructor argument when you're composing tools from a separate registry — for example, when a multi-agent system shares a common tool library across several agents (the pattern the **Multi-Agent Systems** module's manager-worker layout uses).

4.7.2. Tool Discovery by the Agent

The LLM learns about tools from signatures and docstrings.

```
@agent.tool
def get_weather(ctx: RunContext, city: str, country_code: str =
'US') -> dict:
    """Get current weather information for a city.

    Args:
        city: Name of the city
        country_code: Two-letter country code (default: US)

    Returns:
        Dictionary with temperature, conditions, and humidity
    """
    # Implementation
    return weather_data
```

4.7.3. Tool Signature Shown to the LLM

The agent receives a description like:

```
get_weather(city: str, country_code: str = 'US') -> dict
Get current weather information for a city.
```

4.7.4. Registering Multiple Tools

Agents pick the right tool per query — the LLM uses each tool's docstring to decide.

```
@agent.tool
def add(ctx: RunContext, a: float, b: float) -> float:
    """Add two numbers."""
    return a + b

@agent.tool
def multiply(ctx: RunContext, a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

# The agent picks based on the user's query.
result = agent.run_sync('What is 15 times 23?') # → multiply(15, 23)
```

A third tool from a different domain shows that the LLM's selection is purely docstring-driven — the agent picks the right tool by **meaning**, not by name pattern:

```
@agent.tool
def search_products(ctx: RunContext, keyword: str) -> list[dict]:
    """Search for products matching a keyword."""
    return products
```

When tool selection misbehaves in production, the fix is almost always in the docstrings — make them describe **when to use this tool** rather than **what this tool does**. "Use to compute a*b for two numeric inputs" produces better selection than "Multiply two numbers".

4.8. Passing Runtime Context to Tools

- Tools need DBs, keys, shared state
- `RunContext` injects dependencies
- Avoids hard-coded connections

4.8.1. Using RunContext

Pass dependencies and shared state to tools via `ctx.deps`.

```
@agent.tool
def query_database(ctx: RunContext, sql: str) -> list[dict]:
    """Execute a SQL query against the database."""
    db = ctx.deps['database']
    return db.execute(sql)

result = agent.run_sync('Show me all customers in California',
                        deps={'database': db_connection})
```

This chapter uses a plain-dict `deps=` with dict-keyed access (`ctx.deps['database']`) for brevity. The **Dependencies and Runtime Context** module introduces the typed form — `RunContext[MyDeps]` over a `TypedDict`, `BaseModel`, or dataclass — that you should prefer in production code. The two forms are **not** interchangeable on the same object: a `BaseModel` `deps` class does not implement `getitem`, so the dict idiom raises `TypeError: '<DepsClass>' object is not subscriptable`. Be deliberate about which form your tool expects.

The full version with imports:

```
import os
from pydantic_ai import Agent, RunContext

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

@agent.tool
def query_database(ctx: RunContext, sql: str) -> list[dict]:
    """Execute a SQL query against the database."""
    # Access database connection from context
    db = ctx.deps['database']
    results = db.execute(sql)
    return results

# Provide dependencies when running the agent
result = agent.run_sync(
    'Show me all customers in California',
    deps={'database': db_connection},
)
```

4.8.2. Injecting Configuration

Configuration (URLs, API keys) flows in through `deps`, not module-level globals.

```
@agent.tool
def call_api(ctx: RunContext, endpoint: str) -> dict:
    """Call an external API endpoint."""
    base_url = ctx.deps['api_base_url']
    api_key = ctx.deps['api_key']
    response = requests.get(
        f'{base_url}/{endpoint}',
        headers={'Authorization': f'Bearer {api_key}'},
    )
    return response.json()
```

The full pattern with imports and an example call site that reads the API key from `os.getenv`:

```
import os
import requests

@agent.tool
def call_api(ctx: RunContext, endpoint: str) -> dict:
    """Call an external API endpoint."""
    base_url = ctx.deps['api_base_url']
    api_key = ctx.deps['api_key']

    response = requests.get(
        f'{base_url}/{endpoint}',
        headers={'Authorization': f'Bearer {api_key}'},
    )
    return response.json()

# Provide configuration
result = agent.run_sync(
    'Get user data for user ID 123',
    deps={
        'api_base_url': 'https://api.example.com',
        'api_key': os.getenv('API_KEY'),
    },
)
```

The reason to thread config through `deps` rather than reading `os.getenv` **inside** the tool is testability — the tool can be exercised in unit tests with a fake `deps` dict that points at a mock server, without monkey-patching environment variables.

4.8.3. Sharing Resources Across Tools

Multiple tools see the **same** `ctx.deps` — share connections, caches, or clients.

```

@agent.tool
def cache_data(ctx: RunContext, key: str, value: str) -> str:
    """Store data in the cache."""
    ctx.deps['cache'][key] = value
    return f'Cached {key}'

@agent.tool
def retrieve_data(ctx: RunContext, key: str) -> str:
    """Retrieve data from the cache."""
    return ctx.deps['cache'].get(key, 'Not found')

```

The full version with the call site that wires the cache through:

```

@agent.tool
def cache_data(ctx: RunContext, key: str, value: str) -> str:
    """Store data in the cache."""
    cache = ctx.deps['cache']
    cache[key] = value
    return f'Cached {key}'

@agent.tool
def retrieve_data(ctx: RunContext, key: str) -> str:
    """Retrieve data from the cache."""
    cache = ctx.deps['cache']
    return cache.get(key, 'Not found')

# Shared cache between tools
cache = {}
result = agent.run_sync(
    'Store "hello" as greeting, then retrieve it',
    deps={'cache': cache},
)

```

Resources that are **expensive to construct** (database connections, HTTP clients, embeddings models) belong in `deps` so the agent doesn't pay the construction

cost on every tool call. The plain dict shown here is fine for in-process state; for cross-process state, point `deps['cache']` at Redis instead.

4.9. Error Handling and Graceful Degradation

- External systems fail
- Tools must handle failure
- Agents stay useful when tools break

4.9.1. Handling Errors Inside Tools

Catch exceptions inside tools to prevent agent failures.

```
@agent.tool
def safe_divide(ctx: RunContext, a: float, b: float) -> str:
    """Divide two numbers safely."""
    try:
        result = a / b
        return f'Result: {result}'
    except ZeroDivisionError:
        return 'Error: Cannot divide by zero'
    except Exception as e:
        return f'Error: {str(e)}'
```

4.9.2. Returning Structured Error Info

A typed `ToolResult` lets the agent **reason about** failures rather than just narrate them.

```

class ToolResult(BaseModel):
    success: bool
    data: dict | None = None
    error: str | None = None

@agent.tool
def api_call(ctx: RunContext, endpoint: str) -> ToolResult:
    try:
        response = requests.get(endpoint, timeout=5)
        response.raise_for_status()
        return ToolResult(success=True, data=response.json())
    except Exception as e:
        return ToolResult(success=False, error=str(e))

```

The full version with the imports and a docstring on the tool:

```

import requests
from pydantic import BaseModel

class ToolResult(BaseModel):
    success: bool
    data: dict | None = None
    error: str | None = None

@agent.tool
def api_call(ctx: RunContext, endpoint: str) -> ToolResult:
    """Call an external API with error handling."""
    try:
        response = requests.get(endpoint, timeout=5)
        response.raise_for_status()
        return ToolResult(success=True, data=response.json())
    except Exception as e:
        return ToolResult(success=False, error=str(e))

```

For richer agent reasoning, replace the free-form `error: str | None` with a typed enum (`error_kind: Literal['timeout', 'auth', 'rate_limit',`

`'bad_request', 'unknown']`) so the agent can branch deterministically on failure mode rather than parsing the error string.

4.9.3. Fallback Strategies

When the upstream fails, return a stale answer from cache rather than nothing.

```
@agent.tool
def get_data_with_fallback(ctx: RunContext, query: str) -> dict:
    cache = ctx.deps['cache']
    try:
        data = fetch_from_api(query)
        cache[query] = data
        return data
    except Exception:
        if query in cache:
            return {'data': cache[query], 'source': 'cache'}
        return {'error': 'Data unavailable'}
```

The pattern shown here is **write-through** — every successful API call refreshes the cache, so the cache always holds the most recent successful answer. The fallback path returns `{'source': 'cache'}` so the agent knows the data is potentially stale and can communicate that to the user.

Three production refinements worth knowing:

1. **TTL on cache entries** — stale data eventually becomes wrong data. A 5–30 minute expiry depending on data volatility is a common starting point.
2. **Distinguish cache types in the response** — `'source': 'cache'` is one bit; production callers often want to know **how stale** the cached entry is.
3. **Graceful degradation tier** — pair the cache fallback with a default-value tier (the third branch above) so the tool never raises into the agent loop on first-run queries that miss the cache.

4.9.4. Choosing an Error-Handling Strategy

Three patterns, three failure profiles. Match the pattern to the failure mode.

Pattern	Use when...	Example
try/except returning a string	The tool can fail in obvious, recoverable ways and the agent only needs a short human-readable explanation.	Division by zero, malformed input, missing key.
Structured <code>ToolResult</code> (Pydantic)	The agent must reason about the failure (decide whether to retry, escalate, or pick a different tool) rather than just narrate it.	Upstream API returns a typed error (<code>auth_failed</code> , <code>rate_limit</code> , <code>bad_request</code>).
Fallback to cache or alternate source	The upstream has known availability issues and a recent stale answer is more useful than no answer at all.	External market-data API; first-party cache survives outages.

A useful selection rubric: ask yourself what the agent should **do** with the failure. If the right answer is "say sorry and stop," return a string. If the right answer is "try the same call again with a longer timeout, or pick a different tool," return a structured `ToolResult` so the agent can branch on the error type. If the right answer is "give the user any answer, even a slightly stale one," fall back to a cache. The patterns are layered — production tools often combine all three (cache fallback for transient API failures, structured `ToolResult` for terminal upstream errors, plain-string return for impossible inputs).

4.10. Common Tool Integration Patterns

- Recurring shapes show up often

- Four core patterns
- Calculator, DB, API, search

4.10.1. Calculator Pattern

Mathematical operations and computations.

```
@agent.tool
def calculate(ctx: RunContext, expression: str) -> float:
    """Evaluate a mathematical expression safely."""
    try:
        return eval(expression, {'__builtins__': {}}, OPS)
    except Exception as e:
        raise ValueError(f'Invalid expression: {e}')
```

The operator allowlist that makes the `eval` safe(r) — `builtins: {}` blocks Python's built-ins, and only the named keys in `OPS` are reachable inside the expression:

```
import operator

OPS = {
    'add': operator.add,
    'sub': operator.sub,
    'mul': operator.mul,
    'div': operator.truediv,
}

@agent.tool
def calculate(ctx: RunContext, expression: str) -> float:
    """Evaluate a mathematical expression safely."""
    try:
        return eval(expression, {'__builtins__': {}}, OPS)
    except Exception as e:
        raise ValueError(f'Invalid expression: {e}')
```

`eval` is a **teaching shortcut** — production calculators should use a proper parser like `ast.parse` with a custom node visitor that walks the AST and rejects anything outside the allowlist. The course example is safe enough to demo with, but isn't safe enough to ship.

4.10.2. Database Query Pattern

Parameterized SQL via `ctx.deps['database']` — never string-concatenate user input.

```
@agent.tool
def query_sales(ctx: RunContext, start_date: str,
                end_date: str, region: str = "all") -> list[dict]:
    """Query sales data for a date range and region."""
    db = ctx.deps["database"]
    query = "SELECT date, region, amount FROM sales WHERE date
    BETWEEN ? AND ?"
    params = [start_date, end_date]
    if region != "all":
        query += " AND region = ?"
        params.append(region)
    return [dict(row) for row in db.execute(query,
    params).fetchall()]
```

The full version with the multi-line SQL formatting and per-step assignments:

```
@agent.tool
def query_sales(
    ctx: RunContext, start_date: str, end_date: str, region: str = "
all"
) -> list[dict]:
    """Query sales data for a date range and region."""
    db = ctx.deps["database"] # sqlite3.Connection (or compatible)

    query = """
        SELECT date, region, amount
        FROM sales
        WHERE date BETWEEN ? AND ?
    """
    params = [start_date, end_date]

    if region != "all":
        query += " AND region = ?"
        params.append(region)

    cursor = db.execute(query, params)
    return [dict(row) for row in cursor.fetchall()]
```

The `?` placeholders + `params` list is the SQL-injection-safe pattern — the database driver substitutes parameters separately from query parsing, so no user-controlled string ever becomes SQL syntax. Never f-string or `.format` user-supplied values into a query string, even when the values look "safe" — agents don't reliably tell when a value is hostile.

4.10.3. API Integration Pattern

External service integration with timeout, auth header, and explicit status check.

```
@agent.tool
def get_stock_price(ctx: RunContext, symbol: str) -> dict:
    """Get current stock price for a symbol."""
    response = requests.get(
        f'https://api.stockdata.com/quote/{symbol}',
        headers={'X-API-Key': ctx.deps['stock_api_key']},
        timeout=10,
    )
    response.raise_for_status()
    return response.json()
```

The full version pulls only the fields the caller needs and explicitly checks the status code:

```
import requests

@agent.tool
def get_stock_price(ctx: RunContext, symbol: str) -> dict:
    """Get current stock price for a symbol."""
    api_key = ctx.deps['stock_api_key']

    response = requests.get(
        f'https://api.stockdata.com/quote/{symbol}',
        headers={'X-API-Key': api_key},
        timeout=10,
    )

    if response.status_code == 200:
        data = response.json()
        return {
            'symbol': symbol,
            'price': data['price'],
            'change': data['change'],
            'timestamp': data['timestamp'],
        }
    else:
        raise ValueError(f'Failed to fetch stock price: {response.status_code}')
```

The `timeout=10` argument is the load-bearing line — without it, a hung upstream API will block the agent loop indefinitely, blocking the entire request. Always set a timeout on outbound calls, even when "the API is reliable". Pair the timeout with one of the error-handling patterns from earlier in the chapter so the agent has a defined behavior when the timeout fires.

4.10.4. Search Pattern

Information retrieval and search operations.

```
@agent.tool
def search_documents(ctx: RunContext, query: str, max_results:
int = 10) -> list[dict]:
    """Search documents using keyword search."""
    index = ctx.deps["search_index"]

    results = index.search(query, limit=max_results)

    return [
        {"title": doc.title, "snippet": doc.content[:200],
"relevance": doc.score}
        for doc in results
    ]
```

4.11. Summary

In this module we covered:

- Why tools are essential for extending agent capabilities
- The tool lifecycle: definition, validation, and execution
- Tool registration and discovery mechanisms
- Passing runtime context and dependencies to tools
- Error handling and graceful degradation strategies
- Common tool integration patterns

4.12. Reflection Questions

- Why are clear docstrings critical for tool functions, and how do they influence tool selection by the LLM?
- How does passing dependencies through RunContext improve testability compared to hard-coding resources in tools?
- What strategies would you use to handle a tool that depends on an unreliable external API?

- When designing tools for an agent, how do you decide the right level of granularity for each tool?

4.13. Additional Resources

- Pydantic AI tool documentation: <https://ai.pydantic.dev>
- Pydantic documentation: <https://docs.pydantic.dev>
- OpenAI function calling documentation: <https://platform.openai.com/docs>
- Python typing documentation: <https://docs.python.org/3/library/typing.html>

Dependencies and Runtime Context

LESSON 5

- ✓ Demonstrate how to pass external context or data to an agent using dependency injection mechanisms
- ✓ Manage state and dependencies within agent runs, ensuring proper resource scoping
- ✓ Apply clean architecture principles by separating business logic from agent orchestration logic

5.1. Module Overview

- Managing external dependencies
- Dependency injection for agents
- Resource management and state scoping
- Separating business logic from agent logic
- Advanced patterns: factory and container

5.2. Managing External Dependencies

Agents rarely run in isolation.

- Databases and caches
- API clients
- Config and secrets
- File systems

5.2.1. What Are Dependencies?

External resources your agent needs.

- **Databases** – connection pools, ORM sessions
- **API Clients** – HTTP clients, connectors
- **Configuration** – settings, flags, secrets
- **Caches** – Redis, in-memory stores
- **File Systems** – data files, model weights

5.2.2. Problems with Hard-Coded Dependencies

Directly accessing resources creates issues.

```
# BAD: Hard-coded dependencies
@agent.tool
def query_database(ctx: RunContext, sql: str) -> list[dict]:
    # Hard-coded connection string
    db = connect('postgresql://localhost:5432/mydb')
    results = db.execute(sql)
    return results
```

5.2.3. Issues with Hard-Coded Dependencies

- Hard to test (needs real DB)
- Hard to reconfigure per environment
- Risk of resource leaks
- Tight coupling to infrastructure

5.2.4. Benefits of Dependency Injection

Injecting dependencies solves these problems.

- **Testability** – inject mocks or fakes
- **Flexibility** – swap per environment
- **Resource Management** – central lifecycle
- **Modularity** – loose coupling

5.3. Dependency Injection for Agents

Supply resources cleanly, without hard-coding.

- Improves testability
- Enables flexibility
- Separates concerns

5.3.1. Basic Dependency Injection

Pass dependencies via `deps=`; tools read them from `ctx.deps`.

```
@agent.tool
def get_user_data(ctx: RunContext, user_id: int) -> dict:
    """Retrieve user data from the database."""
    db = ctx.deps['database']
    return db.query('SELECT * FROM users WHERE id = ?', [user_id])

result = agent.run_sync('Get user 123', deps={'database':
db_connection})
```

The full version with imports and the agent declaration:

```
import os
from pydantic_ai import Agent, RunContext

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

@agent.tool
def get_user_data(ctx: RunContext, user_id: int) -> dict:
    """Retrieve user data from the database."""
    # Access injected database connection
    db = ctx.deps['database']
    user = db.query('SELECT * FROM users WHERE id = ?', [user_id])
    return user

# Inject dependencies when running
db_connection = create_database_connection()
result = agent.run_sync(
    'Get information for user 123',
    deps={'database': db_connection},
)
```

5.3.2. Typed Dependencies

`TypedDict` + `RunContext[Dependencies]` gives the IDE full autocomplete on `ctx.deps`.

```
class Dependencies(TypedDict):
    database: Database
    config: Config

@agent.tool
def query_data(ctx: RunContext[Dependencies], query: str) ->
    list[dict]:
    db = ctx.deps['database']
    max_results = ctx.deps['config'].max_query_results
    return db.execute(query, limit=max_results)
```

The full version with imports and the call site:

```
import os
from typing import TypedDict
from pydantic_ai import Agent, RunContext

class Dependencies(TypedDict):
    database: Database
    config: Config

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

@agent.tool
def query_data(ctx: RunContext[Dependencies], query: str) -> list[dict]:
    """Query data using injected dependencies."""
    db = ctx.deps['database']
    max_results = ctx.deps['config'].max_query_results
    return db.execute(query, limit=max_results)

# Type-safe dependency injection
result = agent.run_sync(
    'Find all active users',
    deps={'database': db, 'config': config},
)
```

`TypedDict` keeps dict-keyed access (`ctx.deps['database']`) the same as the **Tool Integration and Design** module's untyped pattern, while adding type-checker awareness. The next slide introduces the alternative — `BaseModel`, which uses **attribute** access — and explains when to pick one over the other.

5.3.3. Dependency Classes

`BaseModel` deps use **attribute** access (`ctx.deps.cache`) — not dict-keyed.

```

class AppDependencies(BaseModel):
    database: Database
    cache: RedisClient
    api_key: str
    timeout: int = 30

@agent.tool
def fetch_with_cache(ctx: RunContext[AppDependencies], key: str) -> dict:
    if cached := ctx.deps.cache.get(key):
        return cached
    data = call_api(key, ctx.deps.api_key, ctx.deps.timeout)
    ctx.deps.cache.set(key, data, expire=3600)
    return data

```

Pattern migration from the Tool Integration and Design module: That module's tools used dict-keyed access (`ctx.deps['database']`) against a plain-dict `deps=` . With a `BaseModel` `deps` class like `AppDependencies` above, that idiom raises:

```

TypeError: 'AppDependencies' object is not subscriptable

```

Selection rule:

- `TypedDict` or untyped (plain-dict) `deps` → dict-keyed access (`ctx.deps['x']`)
- `BaseModel` or `dataclass` `deps` → attribute access (`ctx.deps.x`)

The contrast between the `Typed Dependencies` slide above (which uses `TypedDict` + dict access) and this slide (which uses `BaseModel` + attribute access) illustrates the rule. Pick one shape per agent and stay consistent — your IDE and your type-checker will both reward you.

5.4. Resource Management and State Scoping

Control lifecycle and state visibility.

- Prevent leaks
- Ensure cleanup
- Isolate per-run state

5.4.1. Resource Lifecycle

A `@contextmanager`-wrapped factory creates and cleans up dependencies in one place.

```
@contextmanager
def get_dependencies():
    db = Database.connect()
    cache = RedisClient.connect()
    try:
        yield {'database': db, 'cache': cache}
    finally:
        db.close()
        cache.close()

with get_dependencies() as deps:
    result = agent.run_sync('Process data', deps=deps)
```

The full version with imports and a docstring:

```
from contextlib import contextmanager

@contextmanager
def get_dependencies():
    """Create and manage dependencies with proper cleanup."""
    db = Database.connect()
    cache = RedisClient.connect()

    deps = {'database': db, 'cache': cache}

    try:
        yield deps
    finally:
        db.close()
        cache.close()

# Usage with automatic cleanup
with get_dependencies() as deps:
    result = agent.run_sync('Process data', deps=deps)
```

The `try/finally` is load-bearing: `db.close()` runs whether the agent succeeded, raised, or was cancelled. Resource leaks in async agents are a common production failure mode — typically connection-pool exhaustion under load — and a context manager is the cleanest way to prevent them.

5.4.2. Per-Request Scoping

Each agent run gets its own isolated `RequestContext` — no cross-request bleeding.

```
class RequestContext(BaseModel):
    request_id: str
    user_id: str
    database: Database

ctx = RequestContext(request_id=str(uuid.uuid4()), user_id='user1',
                     database=db)
result = agent.run_sync('Query for user1', deps=ctx)
```

The full version showing two side-by-side requests with distinct contexts:

```
import uuid

class RequestContext(BaseModel):
    request_id: str
    user_id: str
    database: Database

# Each agent run gets its own isolated context
context1 = RequestContext(
    request_id=str(uuid.uuid4()),
    user_id='user1',
    database=db,
)
result1 = agent.run_sync('Query for user1', deps=context1)

context2 = RequestContext(
    request_id=str(uuid.uuid4()),
    user_id='user2',
    database=db,
)
result2 = agent.run_sync('Query for user2', deps=context2)
```

Per-request scoping is what makes multi-tenant agent services safe. If `user_id` were a module-level global instead of a `RequestContext` field, two concurrent

requests could overwrite each other's user context mid-call — the kind of bug that shows up in production but never in single-user testing.

5.4.3. Shared State Across Tools

State on the `deps` object persists across every tool call within a single run.

```
class SharedState(BaseModel):
    query_count: int = 0
    cached_results: dict = {}

@agent.tool
def query_1(ctx: RunContext[SharedState], param: str) -> dict:
    ctx.deps.query_count += 1
    ctx.deps.cached_results[param] = {'data': f'Result for {param}'}
    return ctx.deps.cached_results[param]
```

The companion tool that **reads** the shared cache, plus the run that exercises both:

```
@agent.tool
def query_2(ctx: RunContext[SharedState], param: str) -> dict:
    ctx.deps.query_count += 1
    if param in ctx.deps.cached_results:
        return ctx.deps.cached_results[param]
    return {'data': f'New result for {param}'}

# Shared state persists across all tool calls
state = SharedState()
result = agent.run_sync('Run both queries', deps=state)
print(f'Total queries: {state.query_count}')
```

Shared mutable state across tools is convenient but risky if the agent is run concurrently — see the **per-request scoping** slide above. The combination — a shared `SharedState` **within** a run, isolated per-request — is the production-safe pattern.

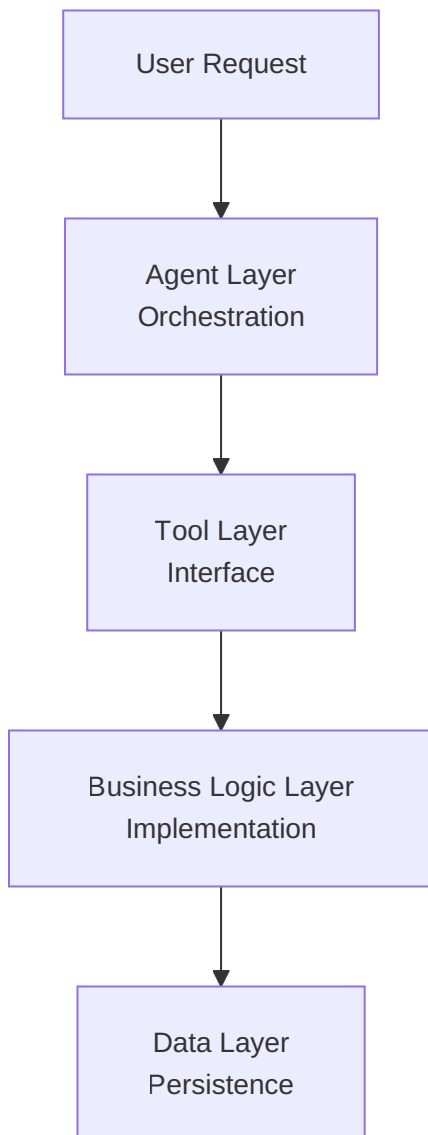
5.5. Separating Business Logic from Agent Logic

Keep agents focused on orchestration.

- Easier to test
- Easier to modify
- Easier to understand

5.5.1. Clean Architecture Principles

Keep agent code focused on orchestration, not implementation.



5.5.2. Bad Practice: Business Logic in Prompts

Tax-rate tables, rounding rules, and surcharge thresholds belong in **code**, not in a prompt.

```
# BAD: Complex business logic in prompt
agent = Agent(
    model,
    system_prompt="""
When calculating sales tax:
1. If region is CA, use 7.25% rate
2. If region is TX, use 6.25% rate
...
""",
)
```

The complete bad-practice example with all five rules:

```
import os

agent = Agent(
    os.getenv("AI_MODEL", "openai:gpt-5.4-mini"),
    system_prompt="""
When calculating sales tax:
1. If region is CA, use 7.25% rate
2. If region is TX, use 6.25% rate
3. If region is NY, use 8.875% rate
4. Round to 2 decimal places
5. Add processing fee of $2.50 if total > $100
...
""",
)
```

Two failure modes follow from this anti-pattern: (1) the LLM sometimes gets the arithmetic wrong (especially under prompt complexity), and (2) updating the rates

means rewriting and re-testing prompts instead of editing a constant. The next slide shows the right alternative.

5.5.3. Good Practice: Business Logic in Functions

Move arithmetic into pure Python functions; the agent calls them as tools.

```
def calculate_sales_tax(amount: float, region: str) -> float:
    tax_rates = {'CA': 0.0725, 'TX': 0.0625, 'NY': 0.08875}
    return round(amount * tax_rates.get(region, 0.0), 2)

@agent.tool
def get_order_total(ctx: RunContext, amount: float, region: str) -> float:
    return calculate_total(amount, region)
```

The full version with the composing function and docstrings:

```
def calculate_sales_tax(amount: float, region: str) -> float:
    """Calculate sales tax based on region."""
    tax_rates = {'CA': 0.0725, 'TX': 0.0625, 'NY': 0.08875}
    return round(amount * tax_rates.get(region, 0.0), 2)

def calculate_total(amount: float, region: str) -> float:
    """Calculate total including tax and fees."""
    subtotal = amount + calculate_sales_tax(amount, region)
    if subtotal > 100:
        subtotal += 2.50
    return round(subtotal, 2)

@agent.tool
def get_order_total(ctx: RunContext, amount: float, region: str) -> float:
    """Get total order cost."""
    return calculate_total(amount, region)
```

The pure-Python helpers (`calculate_sales_tax`, `calculate_total`) can be unit-tested deterministically with no LLM in the loop — exactly the kind of code that should carry the bulk of an agent system's test coverage.

5.5.4. Layered Architecture

Three layers — data access, business logic, tool — each with one responsibility.

```
# Layer 3: Tool Integration
@agent.tool
def analyze_sales(ctx: RunContext, region: str, start: str, end:
str) -> dict:
    """Analyze regional sales."""
    return ctx.deps['sales_service'].calculate_revenue(region,
start, end)
```

The full three-layer stack with all class definitions:

```
# Layer 1: Data Access
class SalesRepository:
    def __init__(self, db: Database):
        self.db = db

    def get_sales(self, region: str, start: str, end: str):
        return self.db.query(
            'SELECT * FROM sales WHERE...',
            [region, start, end],
        )

# Layer 2: Business Logic
class SalesService:
    def __init__(self, repo: SalesRepository):
        self.repo = repo

    def calculate_revenue(self, region: str, start: str, end: str) -
> dict:
        sales = self.repo.get_sales(region, start, end)
        total = sum(s['amount'] for s in sales)
        return {'region': region, 'total': total, 'count':
len(sales)}

# Layer 3: Tool Integration
@agent.tool
def analyze_sales(ctx: RunContext, region: str, start: str, end:
str) -> dict:
    """Analyze regional sales."""
    return ctx.deps['sales_service'].calculate_revenue(region,
start, end)
```

The layers are independently testable: `SalesRepository` against an in-memory database, `SalesService` against a fake repository, the agent tool against a fake service. No layer needs the layer below to be the **real** one to be exercised.

5.5.5. Benefits of Separation

- **Testability** – unit test logic directly
- **Reusability** – share across the app
- **Maintainability** – no prompt rewrites
- **Performance** – save LLM tokens

5.6. Advanced Patterns

Manage complexity as systems grow.

- Factory pattern
- DI container
- Production-ready config

5.6.1. Factory Pattern for Dependencies

A factory swaps real vs. mock dependencies based on environment.

```
class DependencyFactory:
    @staticmethod
    def create_for_environment(env: str) -> dict:
        if env == 'production':
            return {'database': ProductionDatabase(),
                    'api_key': os.getenv('PROD_API_KEY')}
        return {'database': MockDatabase(), 'api_key': 'test-key'}

deps =
DependencyFactory.create_for_environment(os.getenv('ENVIRONMENT'))
```

The full version with explicit branches and a docstring:

```
class DependencyFactory:
    @staticmethod
    def create_for_environment(env: str) -> dict:
        """Create dependencies for specific environment."""
        if env == 'production':
            return {
                'database': ProductionDatabase(),
                'api_key': os.getenv('PROD_API_KEY'),
            }
        else: # testing/development
            return {
                'database': MockDatabase(),
                'api_key': 'test-key',
            }

# Usage
env = os.getenv('ENVIRONMENT', 'development')
deps = DependencyFactory.create_for_environment(env)
result = agent.run_sync('Query data', deps=deps)
```

The factory's value scales with environment count — for 2 environments it's overkill, but for `dev / staging / prod / on-call-incident-replay`, a centralized factory beats four scattered conditionals throughout the codebase.

5.6.2. Dependency Injection Container

A container resolves multi-layer graphs through registered factories.

```
class Container:
    def __init__(self):
        self._services = {}

    def register(self, name: str, factory):
        self._services[name] = factory

    def get(self, name: str):
        return self._services[name]()
```

The full setup that wires the three-layer Sales architecture from earlier:

```
container = Container()
container.register('db', lambda: Database.connect())
container.register('repo', lambda:
    SalesRepository(container.get('db')))
container.register('service', lambda:
    SalesService(container.get('repo')))

deps = {'sales_service': container.get('service')}
result = agent.run_sync('Analyze sales', deps=deps)
```

For production, use a real DI library — `dependency-injector` and `pung` are the popular Python choices. They add features the hand-rolled `Container` lacks: singleton vs. transient lifetimes, async resolution, configuration injection, and integration with FastAPI's `Depends()`. The hand-rolled version is fine for teaching the concept; production typically reaches for the library.

5.7. Summary

- External dependencies and injection
- Resource lifecycle and scoping
- Business logic vs. orchestration
- Factory and container patterns

5.8. Reflection Questions

- Why is DI especially important for AI agents?
- How does separating logic aid testing?
- What breaks if concurrent runs share state?
- When is a factory better than a dict?

5.9. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- Pydantic documentation: <https://docs.pydantic.dev>
- Python contextlib documentation: <https://docs.python.org/3/library/contextlib.html>
- FastAPI dependency injection guide: <https://fastapi.tiangolo.com>

Conversations, Memory, and State

LESSON 6

- ✓ Describe how AI agents maintain context across multiple user turns in a conversation and why managing state is essential for multi-turn interactions
- ✓ Implement conversation memory in an agent to handle follow-up questions by utilizing message history or state persistence
- ✓ Compare different memory management patterns (such as sliding window, summary-based memory) and explain when to use each in agent design
- ✓ Demonstrate how an agent can produce streaming responses and continuous outputs, and enable basic interactive agent loops
- ✓ Apply human-in-the-loop strategies by designing agents that ask clarifying questions or require user confirmation when needed

6.1. Module Overview

- Managing multi-turn conversations
- Memory patterns: sliding window, summary, and hybrid
- State persistence strategies
- Streaming responses and interactive loops
- Human-in-the-loop confirmation
- Best practices for conversational agents

6.2. Conversations, Memory, and State

Multi-turn dialogues present a unique challenge: how do agents track what has been said to maintain context?

6.2.1. The Challenge of Context Management

- **Token Limitations** – finite context windows
- **Relevance** – not all history is equal
- **Cost** – longer contexts cost more
- **Coherence** – consistency across turns

6.2.2. Why State Management Matters

Without proper state management, agents struggle with:

- Following up on previous questions
- Maintaining conversational coherence
- Understanding references ("it", "that")
- Avoiding repetitive responses

6.3. Managing Multi-Turn Conversations

Preserving conversation history enables agents to refer back to previous questions and answers, creating more natural and helpful interactions.

6.3.1. Role of Message History

The prompt history consists of alternating user and assistant messages that provide context for each new turn.

```
# Conceptual illustration only – see "Message History API" below for the
# Pydantic AI shape you actually pass to message_history=.
messages = [
    {"role": "system", "content": "You are a helpful data analyst."},
    {"role": "user", "content": "What are the top 5 products by revenue?"},
    {"role": "assistant", "content": "The top 5 products are..."},
    {"role": "user", "content": "Now show me the growth trend for those products."}
]
```

In the second user message, "those products" refers to the results from the first turn. Without the conversation history, the agent couldn't answer this follow-up question. The role/content dict above is conceptual — Pydantic AI models messages as typed `ModelMessage` objects accessed via `result.all_messages()` and `result.new_messages()`. The next slides cover the actual API.

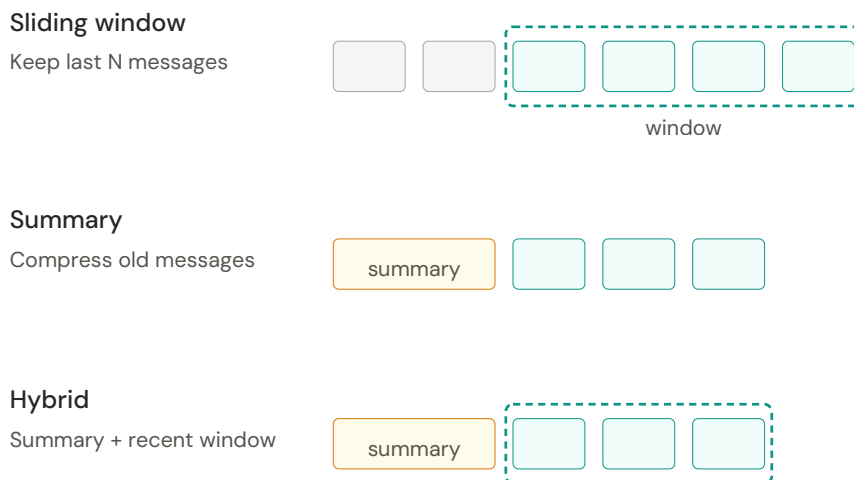
6.3.2. Context Window Considerations

Every LLM has a maximum context window (e.g., 128K–1M tokens depending on the model). As conversations grow, you must decide:

- Which messages to keep in context

- When to compress or summarize history
- How to balance detail with token efficiency

6.4. Memory Patterns Overview



6.5. Choosing a Memory Pattern

- **Sliding window** — when sessions are bounded in length and recent fidelity matters more than long-term coherence (most chatbots, single-task assistants)
- **Summary compression** — when conversations must survive arbitrary length and key facts from earlier turns must be preserved (long-running support sessions, multi-day analyst threads)
- **Hybrid** — when both recent fidelity **and** long-term coherence matter (production assistants that handle multi-hour sessions where the user references something said an hour ago)

The selection collapses to one question: "what does the agent need to remember from older turns?" If the answer is "very little — just the last few exchanges," sliding window is enough. If the answer is "specific facts decisions and preferences, even

from hours ago," you need summary compression so those facts survive token-limit pressure. Hybrid layers summary on top of sliding window so the recent turns are verbatim **and** the older context is compressed — the production-grade default for assistants that combine short-task and long-task interactions.

6.6. Memory Pattern: Sliding Window

Keep only the most recent N interactions in context, dropping older turns to stay within token limits.

Advantages

- Simple to implement
- Predictable token usage
- Maintains recent context verbatim

Disadvantages

- Loses older context completely
- May drop important information
- Not suitable for long conversations

6.6.1. Sliding Window Implementation

```
def sliding_window_memory(messages: list, window_size: int = 10):  
    """Keep only the most recent messages."""  
    system_message = messages[0] # Preserve system prompt  
    recent_messages = messages[-window_size:]  
    return [system_message] + recent_messages
```

6.7. Memory Pattern: Summary Compression

Replace older dialogue with a summarized version to free up space while retaining key points.

Advantages

- Retains key information
- Balances detail with efficiency
- Enables long conversations

Disadvantages

- Summarization can lose nuance
- Requires extra LLM calls
- More complex to implement

6.7.1. Summary Compression Implementation

```
async def summarize_conversation(messages: list) -> str:
    old_messages = messages[:-10] # all but recent 10
    summary_prompt = format_messages(old_messages)
    result = await summarizer_agent.run(summary_prompt)
    return result.output
```

This block illustrates the **pattern** against the conceptual `{role, content}` message shape used at the top of the chapter. The **Implementing Conversational Memory in an AI Agent** lab implements the same pattern against Pydantic AI's `ModelMessage` shape using a real `summarizer_agent` and the `Message History API` shown later in this chapter.

The full pattern: a dedicated summarizer agent kept separate from the main chat agent so its output type and prompt stay stable, plus the formatter that renders the message list into plain text the summarizer can read.

```

import os
from pydantic_ai import Agent

# Dedicated agent for summarisation – kept separate from the main
# chat agent
# so its output_type and prompt remain stable across the
# conversation.
summarizer_agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=str,
    system_prompt='Summarize conversation history concisely,
preserving key facts, decisions, and context.',
)

def format_messages(messages: list) -> str:
    """Render a conceptual {role, content} message list as plain
    text."""
    return '\n'.join(f"{m['role']}: {m['content']}" for m in
messages)

async def summarize_conversation(messages: list) -> str:
    """Create a summary of older conversation turns."""
    old_messages = messages[:-10] # All but recent 10

    summary_prompt = (
        'Summarize the following conversation history, '
        'preserving key facts, decisions, and context:\n\n'
        + format_messages(old_messages)
    )

    result = await summarizer_agent.run(summary_prompt)
    return result.output

```

6.8. Memory Pattern: Hybrid

Combine both strategies: summarize older turns, keep recent details verbatim.

Benefits:

- Recent turns preserved exactly
- Older context compressed into summaries
- Token usage remains manageable

6.8.1. Hybrid Implementation

```
def hybrid_memory(messages, recent_window=10, summary_threshold=20):  
    if len(messages) <= summary_threshold:  
        return messages  
    split = len(messages) - recent_window  
    summary = create_summary(messages[1:split])  
    summary_msg = {"role": "system", "content": f"Previous context:  
{summary}"}  
    return [messages[0], summary_msg] + messages[split:]
```

Pattern illustration against the conceptual `{role, content}` message shape — `create_summary` is a synchronous wrapper around the `summarizer_agent` defined in the previous slide. The **Implementing Conversational Memory in an AI Agent** lab's `HybridMemory.apply()` implements the same logic against `list[ModelMessage]`.

```
def create_summary(messages: list) -> str:
    """Synchronous wrapper around summarizer_agent for use in non-
    async code."""
    summary_prompt = (
        'Summarize the following conversation history concisely:
\n\n'
        + format_messages(messages)
    )
    return summarizer_agent.run_sync(summary_prompt).output

def hybrid_memory(messages: list, recent_window: int = 10,
                  summary_threshold: int = 20):
    system_message = messages[0]

    if len(messages) <= summary_threshold:
        return messages

    split_point = len(messages) - recent_window
    summary = create_summary(messages[1:split_point])
    summary_msg = {"role": "system", "content": f"Previous context:
{summary}"}

    return [system_message, summary_msg] + messages[split_point:]
```

The two parameters — `recent_window` and `summary_threshold` — are the tuning knobs. A common starting point is `recent_window=10, summary_threshold=20`: keep the last 10 turns verbatim, only start compressing once the conversation passes 20 turns. Production tuning depends on average turn length and the model's context window.

6.9. State Persistence Strategies

For conversations that span multiple sessions or need to be paused and resumed, state persistence is essential.

6.9.1. In-Memory State

Simplest approach: store conversation state in application memory.

```
# Session-based storage
sessions = {}

def get_conversation(session_id: str):
    return sessions.get(session_id, [])

def save_conversation(session_id: str, messages: list):
    sessions[session_id] = messages
```

⚠ Warning

In-memory state is lost when the application restarts. Use this only for short-lived sessions or development.

6.10. Persistent Storage

Persist conversations to a database for durability and multi-session support.

```
class Conversation(Base):
    id = Column(String, primary_key=True)
    messages = Column(JSON)
    updated_at = Column(DateTime)
```

The upsert helper that writes a conversation back to the database. Note

`datetime.utcnow()` was deprecated in Python 3.12; use a timezone-aware UTC instant via `datetime.now(timezone.utc)`.

```
from datetime import datetime, timezone

def save_conversation(db: Session, conv_id: str, messages: list):
    conv = db.query(Conversation).filter_by(id=conv_id).first()
    if conv:
        conv.messages = messages
        conv.updated_at = datetime.now(timezone.utc)
    else:
        db.add(Conversation(id=conv_id, messages=messages))
    db.commit()
```

You can also use file-based storage (JSON files) for simple persistence or Redis for high-performance caching of conversation state.

6.11. Streaming Responses

Streaming enables agents to produce token-by-token outputs for real-time user feedback.

6.11.1. Why Stream?

- Reduced latency perception
- Better UX with progress indicators
- Allows cancellation mid-generation

6.11.2. Streaming Implementation

```
async def stream_response(user_input: str):
    async with agent.run_stream(user_input) as response:
        async for chunk in response.stream_text():
            print(chunk, end='', flush=True)
```

6.12. Message History API

Capture messages from each run; pass them back via `message_history=` on the next call.

```
result1 = agent.run_sync('My name is Alice.')
history = result1.all_messages()

result2 = agent.run_sync(
    'What did I tell you my name was?',
    message_history=history,
)
```

- `result.all_messages()` — full conversation; pass as `message_history=` on the next call
- `result.new_messages()` — only messages added by this turn; useful for incremental persistence
- Plain `{"role": ..., "content": ...}` dicts are **not** accepted — pydantic-ai expects typed `ModelMessage` objects

Pydantic AI represents conversation history as typed `ModelMessage` objects, not plain dicts. The full pattern with imports and the type annotations on `history`:

```

import os
from pydantic_ai import Agent
from pydantic_ai.messages import ModelMessage

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

# Turn 1 – no prior history
result1 = agent.run_sync('My name is Alice.')
history: list[ModelMessage] = result1.all_messages()

# Turn 2 – pass the captured history back so the agent has context
result2 = agent.run_sync(
    'What did I tell you my name was?',
    message_history=history,
)
print(result2.output) # references "Alice" from the prior turn

# After this run, all_messages() returns the full 4-message
conversation;
# new_messages() returns only the messages added by this turn.
history = result2.all_messages()

```

Passing a plain dict to `message_history=` raises `AttributeError` on the first follow-up call — the framework tries to access typed attributes (`.parts`, `.kind`, etc.) that dicts don't have. If you persist conversations to a database, persist them as the result of `model_dump_json()` on the `ModelMessage` instances and reconstruct them with `ModelMessagesTypeAdapter.validate_python(...)` on read.

6.13. Building Interactive Loops

Combine streaming with the message-history API. Capture

`response.all_messages()` after each streamed turn and pass it back on the next call.

```
async def interactive_loop():
    history = []
    while True:
        user_input = input("You: ").strip()
        if user_input.lower() in ['exit', 'quit']:
            break
        async with agent.run_stream(user_input,
message_history=history) as response:
            async for chunk in response.stream_text():
                print(chunk, end='', flush=True)
            history = response.all_messages()
```

The full version with the typed `history: list[ModelMessage]` annotation, a trailing newline after streaming, and a placeholder for capping context size with one of the memory strategies from earlier in the chapter:

```

from pydantic_ai.messages import ModelMessage

async def interactive_loop():
    history: list[ModelMessage] = []

    while True:
        user_input = input("You: ").strip()
        if user_input.lower() in ['exit', 'quit']:
            break

        async with agent.run_stream(user_input,
message_history=history) as response:
            async for chunk in response.stream_text():
                print(chunk, end='', flush=True)
            print() # newline after the streamed response

            # Capture the full message list once streaming has
finished.
            history = response.all_messages()

            # Optionally cap context size with a sliding-window or
summary
            # strategy here – see the memory-pattern slides above. The
strategy
            # function should accept and return list[ModelMessage].

```

The point at which `history = response.all_messages()` runs is **after** the inner `async for` finishes — `run_stream` only finalizes the message list when the streaming context manager exits. Calling `all_messages()` while still inside the stream loop returns an incomplete history.

6.14. Human-in-the-Loop Confirmation

Patterns where an agent defers to a human or asks the user for clarification/confirmation during a conversation.

6.14.1. When to Use HITL Patterns

Human-in-the-loop is essential when:

- User instructions are ambiguous
- Actions have significant consequences
- Agent confidence is low
- Compliance requires human oversight

6.15. Asking Clarifying Questions

Use a **union** output type — the agent can return either a clarification request or a final answer.

```
class ClarificationNeeded(BaseModel):
    question: str

class FinalResponse(BaseModel):
    answer: str

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=ClarificationNeeded | FinalResponse,
)
```

The handler that branches on which member of the union the agent returned:

```

import os
from pydantic_ai import Agent
from pydantic import BaseModel

async def handle_request(user_input: str):
    result = await agent.run(user_input)

    if isinstance(result.output, ClarificationNeeded):
        print(f"Agent: {result.output.question}")
        clarification = input("You: ")
        result = await agent.run(f"{user_input}\n{clarification}")

    return result.output

```

Because `output_type` is a typed union, `isinstance(result.output, ClarificationNeeded)` is the right discriminator — Pydantic AI populates one of the two branches based on what the model decided to produce. This is significantly safer than asking the model to set a free-form `"needs_clarification": true` flag and trusting the boolean.

6.16. Confirmation Before Actions

Gate sensitive operations behind an explicit human confirmation.

```

SENSITIVE_ACTIONS = {"delete_file", "make_payment", "send_email"}

async def execute_with_confirmation(action: str, details: dict):
    if action in SENSITIVE_ACTIONS:
        if input(f"Confirm {action}? (yes/no): ").lower() != 'yes':
            return {"status": "cancelled"}
    return await perform_action(action, details)

```

The full version with verbose pre-confirmation output so the user sees what's about to happen before they decide:

```
async def execute_with_confirmation(action: str, details: dict):
    if action in SENSITIVE_ACTIONS:
        print(f"\nAgent wants to: {action}")
        print(f"Details: {details}")
        if input("Confirm? (yes/no): ").lower() != 'yes':
            return {"status": "cancelled"}

    return await perform_action(action, details)
```

Agents should also practice graceful degradation: when they can't complete a task, they should explain why, offer alternatives, and never fabricate capabilities. The confirmation gate is one of the few places where blocking on user input is the **right** default — a silent agent that performs irreversible actions on its own is a worse failure mode than a slow one that asks first.

6.17. Best Practices for Conversational Agents

Effective agents require thoughtful UX, memory management, and error handling.

6.17.1. Design Considerations

- **Clear Session Boundaries** – signal new vs. continuing
- **Context Indicators** – show what the agent remembers
- **Memory Transparency** – let users view or clear history
- **Token Monitoring** – warn near context limits

6.17.2. Testing Multi-Turn Conversations

- Test follow-up questions referencing prior context
- Verify pronoun resolution ("it", "those")
- Test memory strategies with long conversations
- Validate compression preserves critical info

6.17.3. Monitoring Metrics

- Average conversation length (turns)
- Context window utilization
- Failed reference resolution rate
- User satisfaction by length
- Memory strategy effectiveness

6.18. Summary

In this module, you learned how to:

- Manage multi-turn conversations with message history
- Implement different memory patterns (sliding window, summary, hybrid)
- Persist conversation state across sessions
- Build streaming responses for better user experience
- Design human-in-the-loop patterns for ambiguity and confirmation

These techniques enable you to build agents that handle complex, extended conversations while maintaining context and providing responsive, interactive experiences.

6.19. Reflection Questions

- How would you decide between a sliding window and a summary-based memory strategy for a customer support agent?
- What risks arise when agents lose context from earlier in a conversation, and how can you mitigate them?
- In what scenarios is human-in-the-loop confirmation essential, and when might it create unnecessary friction?
- How does streaming improve the user experience compared to waiting for a complete response?

6.20. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- Python asyncio documentation: <https://docs.python.org/3/library/asyncio.html>
- OpenAI API documentation on chat completions: <https://platform.openai.com/docs>
- Anthropic API documentation: <https://docs.anthropic.com>

Evaluation and Guardrails

LESSON 7

- ✓ Identify key success criteria for evaluating an AI agent's output (accuracy, relevance, completeness, format, compliance)
- ✓ Implement basic guardrails to ensure agent outputs meet quality and safety standards
- ✓ Utilize automated methods to evaluate an agent's performance using test cases or secondary AI evaluators
- ✓ Analyze strategies for detecting AI errors like hallucinations or inconsistent answers and describe mitigation techniques
- ✓ Evaluate the trade-offs between response quality and cost/performance, and determine appropriate continuous monitoring strategies

7.1. Module Overview

- Defining success criteria and quality metrics
- Automated evaluation patterns
- Input and output guardrails
- Detecting hallucinations and drift
- Cost-quality trade-offs
- Continuous monitoring and feedback loops

7.2. Defining Success Criteria

- Establish clear metrics before building
- Measurable goals enable objective assessment
- Criteria guide improvements over time

7.2.1. Why Evaluation Matters

- Is the agent accurate?
- Does it handle edge cases?
- Is it safe and compliant?
- Does it meet user expectations?
- Is it cost-effective?

7.2.2. Establishing Quality Metrics

- **Accuracy** – Correctness of information
- **Relevance** – Matches the question asked
- **Completeness** – Required information included
- **Format Compliance** – Matches expected schema
- **Safety** – No harmful or biased content
- **Latency** – Acceptable response time
- **Cost** – Token usage within budget

Domain-specific metrics might include task success rate, user satisfaction scores, or business KPIs like conversion rate for sales agents.

7.2.3. Creating an Evaluation Rubric

Design a scoring system for agent outputs.

```
class EvaluationResult(BaseModel):  
    accuracy: float          # 0.0 to 1.0  
    relevance: float  
    completeness: float  
    safety_score: float  
    passed: bool
```

The full rubric class adds an `explanation: str` field for the judge's reasoning and an `overall_score` computed property that averages the four scalar fields. Both are useful in production but distract from the load-bearing teaching point on this slide — the **shape of the contract** the agent must satisfy.

```
from pydantic import BaseModel  
  
class EvaluationResult(BaseModel):  
    accuracy: float          # 0.0 to 1.0  
    relevance: float  
    completeness: float  
    safety_score: float  
    explanation: str  
    passed: bool  
  
    @property  
    def overall_score(self) -> float:  
        return (self.accuracy + self.relevance +  
                self.completeness + self.safety_score) / 4
```

7.3. Automated Evaluation Patterns

- Consistent, repeatable validation
- Enables rapid iteration
- Catches regressions before production

7.3.1. Unit Testing for Agent Logic

Test agent behavior with known inputs and expected outputs.

```
@pytest.mark.asyncio
async def test_basic_math():
    result = await agent.run('What is 2 + 2?')
    assert '4' in result.output
```

A second test exercises the **edge-case** side of the same agent. Pair the happy-path test with at least one boundary or failure-mode test so regressions on either side surface in CI.

```
import os
import pytest
from pydantic_ai import Agent

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    system_prompt='You are a math tutor.',
)

@pytest.mark.asyncio
async def test_basic_math():
    result = await agent.run('What is 2 + 2?')
    assert '4' in result.output

@pytest.mark.asyncio
async def test_invalid_input():
    result = await agent.run('Calculate the square root of -1')
    assert 'imaginary' in result.output.lower() or 'complex' in
result.output.lower()
```

7.3.2. Testing with Mock Models

Replace the real LLM with a deterministic stand-in for fast, free, repeatable tests.

- Pydantic AI ships `TestModel` for predetermined outputs — no API key, no network
- Tests run on CI without consuming a token budget
- Covered in depth in the **Testing and Production Deployment** module

The minimal `TestModel` recipe lets a unit test assert behavior without any LLM call. The **Testing and Production Deployment** module covers `TestModel`, `FunctionModel`, scripted tool-call replay, and `Agent.override(model=...)` patterns end-to-end.

```
from pydantic_ai import Agent
from pydantic_ai.models.test import TestModel

# Configure TestModel with a predetermined response.
test_model = TestModel(custom_output_text='The capital of France is Paris.')
agent = Agent(test_model)

result = agent.run_sync('What is the capital of France?')
assert 'Paris' in result.output
```

7.3.3. LLM-as-Judge Pattern

Use a secondary LLM to evaluate the primary agent's output.

```
class JudgeResponse(BaseModel):
    accuracy: float
    relevance: float
    reasoning: str
    passed: bool

judge_agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=JudgeResponse,
    system_prompt='Evaluate the AI response for accuracy and relevance.',
)
```

The pattern in production: format `Question: ... / Answer: ...` into a single prompt and run the judge agent over it. Because the judge's output is typed (`JudgeResponse`), downstream code can branch on `passed` deterministically.

```
async def evaluate_response(question: str, answer: str) ->
    JudgeResponse:
        prompt = f"Question: {question}\nAnswer: {answer}\n\nEvaluate
        this response."
        return (await judge_agent.run(prompt)).output
```

A common refinement is to use a **cheaper** model for the judge than for the primary agent — judges only need to detect quality drift, not produce frontier-quality reasoning.

7.3.4. Building Test Datasets

Create comprehensive test cases covering common and edge scenarios.

```
test_cases = [
    {"input": "What is Python?", "expected_keywords":
    ["programming", "language"]},
    {"input": "How do I sort a list?", "expected_keywords":
    ["sort()", "sorted()"]},
    {"input": "Explain recursion", "expected_keywords":
    ["function", "itself"]},
]
```

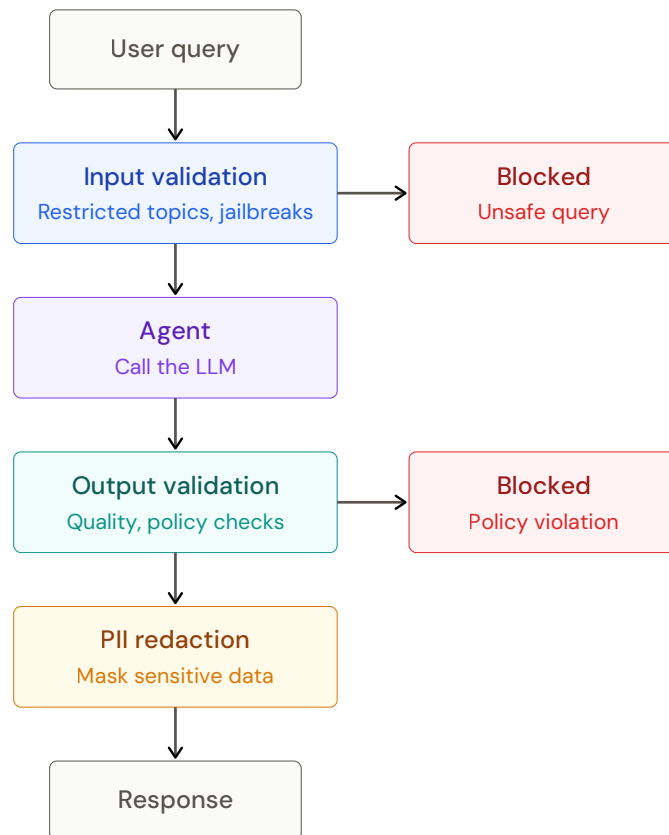
Run the dataset against the agent and collect pass/fail per case. Keep the harness itself as small and inspectable as possible — the value is in the **cases**, not the loop.

```
async def run_evaluation_suite(agent, test_cases):
    results = []
    for case in test_cases:
        response = await agent.run(case["input"])
        passed = all(
            kw in response.output.lower() for kw in
case["expected_keywords"]
        )
        results.append({"case": case, "passed": passed})
    return results
```

In a real project the dataset lives in a versioned file (JSON, YAML, or a dedicated eval framework like `pydantic-evals`) so changes to the test set are reviewable in PRs.

7.4. Input and Output Guardrails

Guardrails enforce safety boundaries at every stage of the agent pipeline.



7.4.1. Why Guardrails Are Critical

Common Risks:

- Prompt injection attacks
- PII leakage
- Toxic or biased content
- Exceeding resource limits
- Generating prohibited content

7.4.2. Input Validation and Filtering

Check and sanitize user inputs before processing.

```
def validate_input(user_input: str) -> tuple[bool, str]:
    if len(user_input) > 5000:
        return False, "Input exceeds maximum length"
    for pattern in INJECTION_PATTERNS:
        if re.search(pattern, user_input, re.IGNORECASE):
            return False, "Potentially malicious input detected"
    return True, "Input validated"
```

The full pattern with module-level constants and the call-site usage:

```
import re

INJECTION_PATTERNS = [
    r"ignore previous instructions",
    r"system prompt",
]

def validate_input(user_input: str) -> tuple[bool, str]:
    if len(user_input) > 5000:
        return False, "Input exceeds maximum length"
    for pattern in INJECTION_PATTERNS:
        if re.search(pattern, user_input, re.IGNORECASE):
            return False, "Potentially malicious input detected"
    return True, "Input validated"

# Caller
valid, message = validate_input(user_input)
if not valid:
    raise ValueError(message)
```

Regex pattern lists are a **first-line** filter — fast, free, and easy to reason about. Treat them as a complement to (not a substitute for) the LLM-based moderation patterns shown later in the chapter.

7.4.3. PII Detection and Redaction

Identify and remove sensitive information.

```
class PIIFilter(BaseModel):
    contains_pii: bool
    pii_types: list[str]
    redacted_text: str

pii_detector = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=PIIFilter,

    system_prompt='Detect and redact PII (emails, phone, SSN, credit
cards) with [REDACTED].',
)
```

The wrapper that pipes user text through the detector and returns the redacted version:

```
async def filter_pii(text: str) -> str:
    result = await pii_detector.run(f"Check this text: {text}")
    if result.output.contains_pii:
        return result.output.redacted_text
    return text
```

For high-volume systems, pair the LLM-based detector with a fast regex pass for the obvious cases (email, phone, SSN) — the regex covers 80% of inputs without an API call, and the LLM handles the long tail.

7.4.4. Output Constraints and Safety Checks

Validate agent outputs meet safety and format requirements.

```
BANNED_TOPICS = ["violence", "illegal activities", "hate speech"]

async def safe_agent_run(agent, user_input: str):
    result = await agent.run(user_input)
    for topic in BANNED_TOPICS:
        if topic in result.output.lower():
            return "I cannot provide information on that topic."
    return result.output
```

The full version also caps output length to prevent runaway responses from consuming downstream resources:

```
async def safe_agent_run(agent, user_input: str):
    result = await agent.run(user_input)
    output = result.output

    # Banned-content check
    for topic in BANNED_TOPICS:
        if topic in output.lower():
            return "I cannot provide information on that topic."

    # Length cap
    if len(output) > 10000:
        return output[:10000] + "... [truncated]"

    return output
```

Substring matching against a banned-words list is a blunt instrument — it both over-blocks (legitimate uses of "violence" in news summaries) and under-blocks (typos, paraphrases, multi-language inputs). Use it as a fast pre-filter, then defer the **judgment call** to a moderation API or LLM-based classifier.

7.4.5. Content Moderation Integration

Use external moderation APIs for additional safety.

```
moderation_client = AsyncOpenAI()

async def moderate_content(text: str) -> bool:
    """Returns True if safe, False if flagged."""
    response = await
moderation_client.moderations.create(input=text)
    return not response.results[0].flagged
```

The complete pattern wraps an agent so both the user input and the model output flow through `moderate_content`. The moderation client is **separate from** the LLM provider — even when the agent talks to Anthropic or Google, OpenAI's moderation endpoint is the cheap, fast classifier that's been the de facto industry baseline.

```
import os
from openai import AsyncOpenAI
from pydantic_ai import Agent

moderation_client = AsyncOpenAI()

async def moderate_content(text: str) -> bool:
    response = await
moderation_client.moderations.create(input=text)
    return not response.results[0].flagged

async def agent_with_moderation(agent: Agent, user_input: str):
    if not await moderate_content(user_input):
        return "Input contains inappropriate content."

    result = await agent.run(user_input)

    if not await moderate_content(result.output):
        return "Unable to generate appropriate response."

    return result.output

# Example caller
agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))
# response = await agent_with_moderation(agent, 'Hello, how are
you?')
```

7.5. Detecting Hallucinations and Drift

- Agents can fabricate plausible-sounding info
- Behavior can drift over time
- Detection enables mitigation

7.5.1. What Are Hallucinations?

Common Types:

- Inventing facts or statistics
- Fabricating citations or sources
- Contradicting known information
- Mixing unrelated concepts

7.5.2. Fact-Checking with Retrieval

Verify agent claims against trusted sources.

```
class FactCheck(BaseModel):
    claim: str
    is_factual: bool
    confidence: float
    source: str | None

fact_checker = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=FactCheck,
    system_prompt='Verify claims against provided context. Be
conservative.',
)
```

The verification helper composes the prompt and unwraps the typed output:

```
async def verify_claim(claim: str, context: str) -> FactCheck:
    prompt = f"Context: {context}\n\nClaim: {claim}\n\nIs this claim supported?"
    return (await fact_checker.run(prompt)).output
```

In production the `context` argument is the result of a retrieval step — typically a vector-store top-K query against a curated knowledge base. The fact-checker's job is **not** to know the truth, only to compare a claim against retrieved evidence.

7.5.3. Consistency Checking

Ensure agent responses don't contradict themselves.

- Run the same question multiple times
- Compare responses (semantic similarity in production)
- Flag divergence as a hallucination signal

The minimal exact-match implementation suffices for teaching the **shape** of the pattern. Production systems compare responses with embedding similarity rather than string equality so that paraphrases don't count as inconsistencies.

```
async def consistency_test(agent, question: str, iterations: int = 3):
    responses = []
    for _ in range(iterations):
        result = await agent.run(question)
        responses.append(result.output)

    # Exact-match check — replace with semantic similarity for
    # production.
    if len(set(responses)) > 1:
        return False, "Inconsistent responses detected"

    return True, "Responses are consistent"
```

Consistency checking is most useful at **evaluation time**, not request time — running the same query 3× per user request triples cost and latency. Sample a small fraction of production traffic into a consistency-check job instead.

7.5.4. Confidence Scoring

Ask agents to rate their own confidence.

```
class AnswerWithConfidence(BaseModel):
    answer: str
    confidence: float # 0.0 to 1.0
    reasoning: str

confident_agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=AnswerWithConfidence,
    system_prompt='Answer questions and rate your confidence honestly.',
)
```

The caller branches on the confidence threshold, attaching a low-confidence warning when the model is uncertain:

```
async def get_confident_answer(question: str):
    result = await confident_agent.run(question)

    if result.output.confidence < 0.7:
        return f"{result.output.answer}\n\n[Note: Low confidence response]"

    return result.output.answer
```

Self-rated confidence is **imperfect** — language models are systematically overconfident on plausible-sounding hallucinations. Use it as one signal among several (consistency, retrieval support, judge agreement), not as a sole arbiter.

7.5.5. Mitigation: Abstain, Ground, or Escalate

Detection alone is not enough — every detection signal needs a mitigation path.

- **Ground** — retrieve context from a knowledge base and inject it into the prompt
 - **Escalate** — when a cheap model is uncertain, hand the question to a more capable one
 - **Abstain** — refuse to answer with low confidence rather than hallucinate
-

The detection patterns above (fact-checking, consistency, confidence) name the **symptoms** of hallucination. Mitigation is what you **do** with the symptom. Three patterns, layered:

- **Ground** — retrieve relevant context from a vector store or knowledge base and inject it into the prompt. Most hallucinations come from gaps in training data; closing the gap removes the incentive to invent facts.
- **Escalate** — when a cheaper model is uncertain, hand the same question to a more capable one. Pairs naturally with the cascading-model pattern in this chapter's `Cost-Quality Trade-offs` section.
- **Abstain and escalate to a human** — refuse to answer with low confidence and surface the failure rather than hide it. The right last resort for production, especially in regulated domains.

The worked example below sequences all three: RAG first, then cascading-model retry, then explicit abstention. Production code rarely uses all three on every query — pick the layers that match your domain's risk tolerance.

```

async def answered_or_abstain(question: str, knowledge_base) -> str:
    """Three-stage mitigation: ground first, then check confidence,
    then abstain."""
    # Stage 1: ground the answer in retrieved context (RAG fallback)
    context = await knowledge_base.search(question, top_k=3)
    grounded = await confident_agent.run(
        f"Context:\n{context}\n\nQuestion: {question}"
    )

    # Stage 2: low-confidence answers escalate rather than guess
    if grounded.output.confidence < 0.7:
        # Escalate to a more capable model OR abstain explicitly.
        capable_agent = Agent(
            os.getenv('AI_MODEL_CAPABLE', 'openai:gpt-5.4'),
            output_type=AnswerWithConfidence,

system_prompt='Answer questions and rate your confidence honestly.',
        )
        retry = await capable_agent.run(
            f"Context:\n{context}\n\nQuestion: {question}"
        )
        if retry.output.confidence < 0.7:
            # Stage 3: abstain. Better to refuse than to
            hallucinate.
            return (
                "I don't have enough information to answer that
confidently. "
                "Please rephrase or supply more context."
            )
        return retry.output.answer

    return grounded.output.answer

```

7.6. Cost-Quality Trade-offs

- Balance quality against operational cost

- Strategic choices: model, caching, compute
- Trade-offs enable efficient systems

7.6.1. Model Selection Matrix

Different models offer different trade-offs between quality, speed, and cost.

High Quality, High Cost

- Frontier models (e.g., GPT-5.4, Claude Opus)
- Best for complex reasoning
- ~10–30x cost vs. small models
- Use for critical decisions

Balanced

- Mid-tier models (e.g., GPT-5.4-mini, Claude Sonnet)
- Good for most tasks
- ~3–5x cost vs. small models
- Use for general workflows

Fast & Cheap

- Lightweight models (e.g., GPT-5.4-nano, Claude Haiku)
- Quick responses
- Low cost
- Use for simple tasks

Specialized

- Fine-tuned models
- Optimized for specific domains
- Variable cost
- Use for repeated patterns

7.6.2. Cascading Model Approach

Start with cheaper models, escalate only when quality drops.

```
async def cascading_agent_run(question: str):
    result = await cheap_agent.run(question)
    quality_score = await evaluate_response_quality(result.output)
    if quality_score > 0.8:
        return result.output          # Good enough – keep the
cheap answer.
    result = await expensive_agent.run(question) # Escalate.
    return result.output
```

The full pattern: a `QualityScore` schema, a small judge agent that rates output quality, and the cascading caller that escalates only on a low score. The judge reuses the LLM-as-judge pattern from earlier in this module.

```
import os
from pydantic import BaseModel
from pydantic_ai import Agent

class QualityScore(BaseModel):
    score: float # 0.0 to 1.0

quality_judge = Agent(
    os.getenv('AI_MODEL_FAST', 'openai:gpt-5.4-nano'),
    output_type=QualityScore,
    system_prompt=(
        'Rate the response quality on a 0.0-1.0 scale. '
        'Consider accuracy, completeness, and relevance.'
    ),
)

async def evaluate_response_quality(response: str) -> float:
    result = await quality_judge.run(f"Rate this response:
\n\n{response}")
    return result.output.score

async def cascading_agent_run(question: str):
    cheap_agent = Agent(os.getenv('AI_MODEL_FAST', 'openai:gpt-5.4-
nano'))
    result = await cheap_agent.run(question)

    quality_score = await evaluate_response_quality(result.output)
    if quality_score > 0.8:
        return result.output

    expensive_agent = Agent(os.getenv('AI_MODEL_CAPABLE',
'openai:gpt-5.4'))
    result = await expensive_agent.run(question)
    return result.output
```

The economics only work when the cheap model handles the **majority** of traffic on its own — measure your escalation rate. If 90% of queries escalate, you're paying both model costs plus the judge for no quality gain.

7.6.3. Token Budget Management

Monitor and limit token usage to control costs.

```
class TokenBudget:
    def __init__(self, max_tokens: int):
        self.max_tokens = max_tokens
        self.used_tokens = 0

    def check_budget(self, tokens: int):
        if self.used_tokens + tokens > self.max_tokens:
            raise ValueError("Token budget exceeded")
        self.used_tokens += tokens
```

The wrapper that estimates input tokens before the call and accounts them against the budget:

```
budget = TokenBudget(max_tokens=100_000)

async def budget_aware_run(agent, prompt: str):
    estimated = len(prompt.split()) * 2 # rough words→tokens
    budget.check_budget(estimated)
    result = await agent.run(prompt)
    return result
```

The `* 2` heuristic is a teaching shortcut — for production accuracy use a real tokenizer (`tiktoken` for OpenAI, `anthropic.tokenizer` for Claude) and account **actual** usage from `result.usage()` after the call rather than an estimate before it.

7.7. Continuous Monitoring and Feedback Loops

- Real-time visibility into performance
- Rapid response to emerging issues
- Feedback loops drive improvement

7.7.1. Production Metrics Dashboard

Essential Metrics:

- Request volume and latency
- Success/failure rate
- Token usage and cost
- User satisfaction scores
- Error types and frequency
- Model performance trends

7.7.2. Logging and Observability

Comprehensive logging enables debugging and improvement.

```
async def instrumented_agent_run(agent, user_input: str, user_id: str):
    try:
        logger.info(f"Agent request from {user_id}: {user_input[:100]}")
        result = await agent.run(user_input)
        logger.info(f"Success, tokens: {result.usage()}")
        return result.output
    except Exception as e:
        logger.error(f"Agent error for {user_id}: {e}")
        raise
```

The full version also captures duration, using a timezone-aware UTC clock (Python 3.12 deprecated naive `datetime.utcnow()`):

```
import logging
from datetime import datetime, timezone

logger = logging.getLogger(__name__)

async def instrumented_agent_run(agent, user_input: str, user_id:
str):
    start_time = datetime.now(timezone.utc)
    try:
        logger.info(f"Agent request from {user_id}: {user_input[:
100]}")
        result = await agent.run(user_input)

        duration = (datetime.now(timezone.utc) -
start_time).total_seconds()
        logger.info(f"Success in {duration}s, tokens:
{result.usage()}")

        return result.output
    except Exception as e:
        logger.error(f"Agent error for {user_id}: {str(e)}")
        raise
```

In production, ship logs to a structured backend (CloudWatch, Datadog, Honeycomb) so durations and token counts become queryable metrics rather than text-grep targets. The **Testing and Production Deployment** module covers OpenTelemetry instrumentation in depth.

7.7.3. Feedback Collection Patterns

Gather user feedback to improve agents over time.

```
class UserFeedback(BaseModel):
    response_id: str
    rating: int
    timestamp: datetime

def collect_feedback(response_id: str, rating: int):
    feedback_storage.append(UserFeedback(
        response_id=response_id,
        rating=rating,
        timestamp=datetime.now(timezone.utc),
    ))
```

The full version emits a warning on low ratings so operators can act on negative feedback in near-real-time:

```
feedback_storage = []

def collect_feedback(response_id: str, rating: int):
    feedback = UserFeedback(
        response_id=response_id,
        rating=rating,
        timestamp=datetime.now(timezone.utc),
    )
    feedback_storage.append(feedback)

    if rating < 3:
        logger.warning(f"Low rating for response {response_id}")
```

`feedback_storage` is shown as an in-memory list to keep the example focused; in production this is a database table or analytics store, joined back to the original `response_id` so every low rating is traceable to the request, prompt, and model output that produced it.

7.7.4. A/B Testing for Improvements

Test agent variations to measure improvements.

```
def ab_test_agent(user_input: str):  
    variant = random.choice(['A', 'B'])  
    prompt = prompt_v1 if variant == 'A' else prompt_v2  
    agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),  
system_prompt=prompt)  
    result = agent.run_sync(user_input)  
    return result.output
```

The full version captures the variant assignment for downstream analysis. In production, the assignment must be **sticky per user** (so the same user sees the same variant across sessions) and the metric — output quality, user rating, downstream conversion — must be wired back to the variant tag.

```
import os
import random
from pydantic_ai import Agent

prompt_v1 = 'You are a helpful assistant. Answer concisely.'
prompt_v2 = 'You are a helpful assistant. Answer thoroughly with
examples.'

def ab_test_agent(user_input: str):
    variant = random.choice(['A', 'B'])
    prompt = prompt_v1 if variant == 'A' else prompt_v2
    agent = Agent(
        os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
        system_prompt=prompt,
    )
    result = agent.run_sync(user_input)

    # Replace this print with a real analytics write (e.g., PostHog,
    # Statsig,
    # or the feedback_storage table from the previous slide).
    print(f'[A/B] variant={variant} input={user_input!r}
    output_len={len(result.output)}')

    return result.output
```

For statistically meaningful results you need both sample size (typically thousands of impressions per variant) and a clear primary metric chosen **before** the test starts. Don't pick the metric after looking at the data — that's how A/B testing produces false positives.

7.8. Summary

In this module, you learned how to:

- Define success criteria and quality metrics for agent evaluation

- Implement automated evaluation using unit tests and LLM-as-judge patterns
- Build input and output guardrails to ensure safety and compliance
- Detect and mitigate hallucinations through fact-checking and consistency checks
- Balance cost and quality through model selection and cascading approaches
- Set up continuous monitoring and feedback loops for production agents

These evaluation and guardrail techniques are essential for building reliable, safe, and cost-effective AI agents in production.

7.9. Reflection Questions

- How would you design an evaluation rubric for an agent in your own domain or industry?
- What are the limitations of the LLM-as-judge pattern, and how can you compensate for them?
- How do you balance strict guardrails with allowing agents enough flexibility to be useful?
- What metrics would you prioritize when monitoring an agent in production, and why?
- How might a cascading model approach reduce costs without significantly impacting response quality?

7.10. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- OpenAI moderation endpoint documentation: <https://platform.openai.com/docs>
- Anthropic safety documentation: <https://docs.anthropic.com>
- Pydantic validation documentation: <https://docs.pydantic.dev>

Workflow Orchestration

LESSON 8

- ✓ Design multi-step agent workflows incorporating sequential, parallel, and conditional execution patterns
- ✓ Implement context-passing mechanisms to maintain state between workflow steps in an agent pipeline
- ✓ Apply error propagation, recovery techniques, and checkpointing to make agent workflows fault-tolerant
- ✓ Optimize agent reliability using retry and fallback strategies for handling failures in a workflow

8.1. Module Overview

- Introduction to workflow orchestration
- Sequential, parallel, and conditional patterns
- Graphs and directed workflows
- Passing context between steps
- Error propagation and recovery
- Checkpointing and resumption
- Retry and fallback strategies

8.2. Introduction to Workflow Orchestration

Orchestration coordinates multiple steps and agents to accomplish complex tasks.

8.2.1. Why Simple Agents Aren't Enough

Single-agent prompts struggle with multi-step work.

- Hard to break down complex tasks
- State doesn't persist across operations
- No recovery from partial failures
- Limited parallelism

8.2.2. What Is Workflow Orchestration?

Coordinating agent steps to reach a complex goal.

- **Steps** – Agent or tool operations
- **Dependencies** – Ordering between steps
- **State** – Data shared across steps
- **Control Flow** – Sequential, parallel, or conditional
- **Error Handling** – Graceful recovery

8.2.3. Real-World Workflow Examples

Research and Writing:

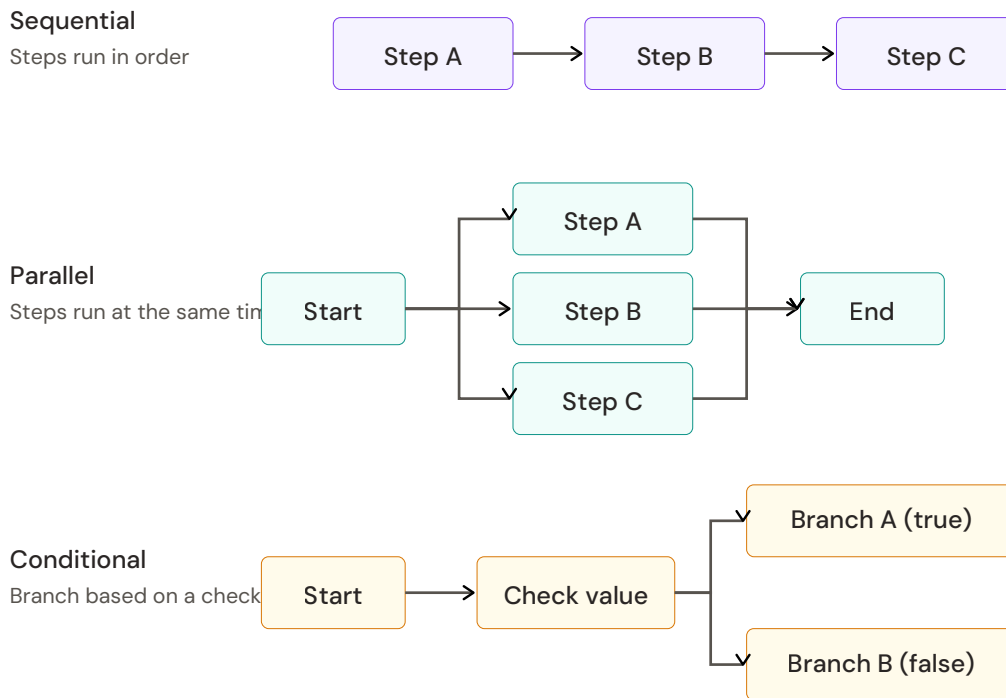
1. Research topic
2. Create outline
3. Write sections (parallel)
4. Review and publish

Data Analysis Pipeline:

1. Extract data
2. Clean and validate
3. Analyze (parallel)
4. Aggregate and report

8.3. Sequential, Parallel, and Conditional Patterns

These patterns are the building blocks of workflow orchestration.



8.3.1. Sequential Execution Pattern

Steps execute one after another; each uses the previous output.

```

async def sequential_workflow(topic: str):
    research = await research_agent.run(f"Research {topic}")
    outline = await outline_agent.run(f"Outline for {topic}.
    Research: {research.output}")
    draft = await writer_agent.run(f"Write article. Outline:
    {outline.output}")
    return draft.output
  
```

Use When: Each step depends on the previous step's output.

The full version with explanatory comments per step:

```
async def sequential_workflow(topic: str):
    # Step 1: Research
    research = await research_agent.run(f"Research {topic}")

    # Step 2: Outline (uses research results)
    outline = await outline_agent.run(
        f"Create outline for {topic}. Research: {research.output}"
    )

    # Step 3: Write (uses outline)
    draft = await writer_agent.run(
        f"Write article. Outline: {outline.output}"
    )

    return draft.output
```

8.3.2. Parallel Execution Pattern

Independent steps execute simultaneously for faster completion.

```
async def parallel_workflow(topics: list[str]):
    tasks = [research_agent.run(f"Research {topic}") for topic in
topics]
    results = await asyncio.gather(*tasks)
    return "\n\n".join(r.output for r in results)
```

Use When: Steps are independent and can run concurrently.

The full version with the import and per-stage comments:

```
import asyncio

async def parallel_workflow(topics: list[str]):
    # Create tasks for all topics
    tasks = [research_agent.run(f"Research {topic}") for topic in
topics]

    # Execute all in parallel
    results = await asyncio.gather(*tasks)

    # Combine results
    summaries = [r.output for r in results]
    return "\n\n".join(summaries)
```

`asyncio.gather` raises on the **first** failure and cancels the rest by default — call it with `return_exceptions=True` if you want partial-success behavior, exactly the way the **Multi-Agent Systems** module's `broadcast_task` pattern does.

8.3.3. Conditional Execution Pattern

Steps execute based on conditions or previous results.

```
async def conditional_workflow(user_request: str):
    classification = await classifier_agent.run(user_request)
    if classification.output == "technical":
        response = await technical_agent.run(user_request)
    elif classification.output == "sales":
        response = await sales_agent.run(user_request)
    else:
        response = await general_agent.run(user_request)
    return response.output
```

Use When: Execution path depends on runtime conditions.

A typed-route refinement: constrain `classification.output` to a `Literal['technical', 'sales', 'general']` (via the classifier's `output_type=`) so the dispatch can't drift out of sync with the routes the model is allowed to produce. Same pattern the **Multi-Agent Systems** module uses for the Basic Delegation Pattern.

8.3.4. Combining Patterns

Real workflows mix sequential, parallel, and conditional logic.

```
async def complex_workflow(document: str):
    info = await extract_agent.run(document)           #
sequential
    if not info.output.is_valid:                       #
conditional
        return await handle_invalid(document)
    sentiment, summary, keywords = await asyncio.gather( #
parallel
        sentiment_agent.run(document),
        summary_agent.run(document),
        keyword_agent.run(document),
    )
    return combine_results(sentiment, summary, keywords)
```

The same workflow with the per-stage comments spelled out — useful for the lecture book where the inline labels above might be too compact for first-time readers:

```
async def complex_workflow(document: str):
    # Sequential: Extract key info
    info = await extract_agent.run(document)

    # Conditional: Check validity
    if not info.output.is_valid:
        return await handle_invalid(document)

    # Parallel: Multiple analyses
    sentiment, summary, keywords = await asyncio.gather(
        sentiment_agent.run(document),
        summary_agent.run(document),
        keyword_agent.run(document),
    )

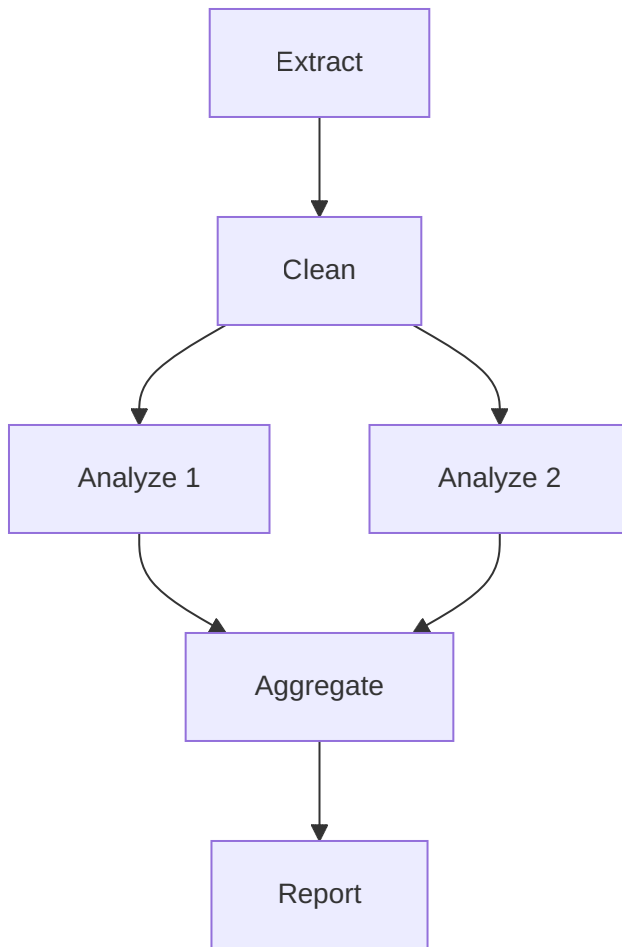
    # Sequential: Combine results
    return combine_results(sentiment, summary, keywords)
```

8.4. Graphs and Directed Workflows

DAGs model complex dependencies and branching logic.

8.4.1. Directed Acyclic Graphs (DAGs)

DAGs represent workflows with clear dependencies and no cycles.

**Benefits:**

- Visual representation of complex workflows
- Clear dependency tracking
- Parallel execution opportunities identified
- Easy to reason about and debug

8.4.2. Implementing Graph-Based Workflows

Define workflows as node dependencies; a topological sort respects them.

```
@dataclass
class WorkflowNode:
    name: str
    func: Callable[..., Any]
    depends_on: list[str]

async def execute_workflow(workflow, data):
    nodes_by_name = {n.name: n for n in workflow}
    graph = {n.name: set(n.depends_on) for n in workflow}
    results = {}
    for name in TopologicalSorter(graph).static_order():
        node = nodes_by_name[name]
        results[name] = await node.func(*[results[d] for d in
node.depends_on])
    return results["report"]
```

The full pattern with imports, an example workflow declaration, and inline notes about the typing fix and `graphlib`'s cycle detection:

```
from dataclasses import dataclass
from graphlib import TopologicalSorter
from typing import Any, Callable

@dataclass
class WorkflowNode:
    name: str
    # Lowercase `callable` is the built-in predicate, not a valid
    type
    # annotation – use Callable from typing instead.
    func: Callable[..., Any]
    depends_on: list[str]

workflow = [
    WorkflowNode("extract", extract_data, depends_on=[]),
    WorkflowNode("clean", clean_data, depends_on=["extract"]),
    WorkflowNode("analyze", analyze, depends_on=["clean"]),
    WorkflowNode("report", generate_report,
depends_on=["analyze"]),
]

async def execute_workflow(workflow, data):
    """Run nodes in dependency order, passing each predecessor's
    result downstream."""
    nodes_by_name = {node.name: node for node in workflow}
    graph = {node.name: set(node.depends_on) for node in workflow}

    results = {}
    # graphlib.TopologicalSorter handles cycle detection and yields
    names
    # in an order consistent with the declared dependencies.
    for name in TopologicalSorter(graph).static_order():
        node = nodes_by_name[name]
        deps = [results[d] for d in node.depends_on]
        results[name] = await node.func(*deps)
    return results["report"]
```

`graphlib.TopologicalSorter` raises `graphlib.CycleError` if you accidentally introduce a cycle (e.g., A depends on B, B depends on A) — useful free behavior worth pointing out in lecture, since cycles are easy to introduce when workflows are edited over time.

8.4.3. Workflow Execution Engines

Use libraries like Prefect or Temporal for complex workflows.

```
@task
async def research(topic: str) -> str:
    return (await research_agent.run(topic)).output

@workflow
async def content_pipeline(topic: str):
    return await write(await research(topic))
```

A conceptual example showing the typical decorator-based shape these engines use — the actual import (`from workflow_engine import task, workflow`) is a placeholder for whichever framework you adopt:

```
from workflow_engine import task, workflow

@task
async def research(topic: str) -> str:
    return (await research_agent.run(topic)).output

@task
async def write(outline: str) -> str:
    return (await writer_agent.run(outline)).output

@workflow
async def content_pipeline(topic: str):
    research_result = await research(topic)
    article = await write(research_result)
    return article
```

The value of a real engine like Prefect or Temporal isn't the decorator syntax — it's the operational features layered underneath: durable execution (workflows survive process restarts), retries with policies, observability dashboards, and human-in-the-loop pause/resume. Reach for one when those features start to matter; the hand-rolled DAG on the previous slide is enough until then.

8.5. Passing Context Between Steps

Workflow steps need to share state and results cleanly.

8.5.1. The Context Object Pattern

Maintain workflow state in a shared, typed context object.

```
class WorkflowContext(BaseModel):  
    user_id: str  
    request: str  
    research_data: str | None = None  
    draft: str | None = None
```

The driver function that fills in the context as each step completes:

```
from pydantic import BaseModel  
  
async def workflow_with_context(ctx: WorkflowContext):  
    # Research  
    result = await research_agent.run(ctx.request)  
    ctx.research_data = result.output  
  
    # Write  
    result = await writer_agent.run(  
        f"Write draft. Research: {ctx.research_data}"  
    )  
    ctx.draft = result.output  
  
    return ctx
```

The optional fields (`research_data: str | None = None`) are how the schema models in-flight workflow state — the field exists from the start of the workflow but is `None` until the step that produces it completes. An immutable variant would return a **new** `WorkflowContext` per step instead of mutating in place; see the next slide pair for the trade-off.

8.5.2. Dependency Injection for Context

Pass context through agent dependencies — tools read it via `ctx.deps`.

```
research_agent = Agent(  
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),  
    deps_type=WorkflowContext,  
)  
  
@research_agent.tool  
async def get_user_preferences(ctx: RunContext[WorkflowContext]) ->  
    str:  
    return f"User {ctx.deps.user_id} preferences"
```

The full pattern with imports and an example call site:

```
import os  
from pydantic_ai import Agent, RunContext  
  
research_agent = Agent(  
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),  
    deps_type=WorkflowContext,  
)  
  
@research_agent.tool  
async def get_user_preferences(ctx: RunContext[WorkflowContext]) ->  
    str:  
    # Access workflow context in tools  
    return f"User {ctx.deps.user_id} preferences"  
  
# Run with context  
ctx = WorkflowContext(user_id="123", request="Explain AI")  
result = await research_agent.run("Research this topic", deps=ctx)
```

This is the same `RunContext[T]` / `deps_type=T` pattern the **Dependencies and Runtime Context** module covered in depth — workflows simply extend it across multiple steps, with each step potentially mutating the shared `ctx.deps` instance.

8.5.3. Immutable vs. Mutable State

Consider trade-offs between immutable and mutable context.

Immutable (Functional)

- Each step returns new context
- Easier to reason about
- Better for retries/checkpoints
- More verbose

Mutable (Stateful)

- Context updated in place
- More ergonomic
- Easier to share state
- Requires careful tracking

8.6. Error Propagation and Recovery

Multi-step workflows need thoughtful error handling to fail gracefully.

8.6.1. Handling Individual Step Failures

Each step is tagged `required` or optional; the dispatcher decides what to do.

```
async def workflow_with_error_handling(steps: list):
    results, errors = [], []
    for step in steps:
        try:
            results.append(await execute_step(step))
        except Exception as e:
            errors.append({"step": step.name, "error": str(e)})
            if step.required:
                raise # fail fast for critical steps
            results.append(None)
    return results, errors
```

The full version with the original explanatory comments:

```
async def workflow_with_error_handling(steps: list):
    results = []
    errors = []

    for step in steps:
        try:
            result = await execute_step(step)
            results.append(result)
        except Exception as e:
            errors.append({"step": step.name, "error": str(e)})

            if step.required:
                raise # Fail fast for critical steps
            else:
                results.append(None) # Continue with None

    return results, errors
```

The `step.required` flag is the **policy** — make it explicit per step rather than burying it in try/except branches scattered through the workflow. A workflow

where every step is `required=True` degenerates to fail-fast; a workflow where every step is `required=False` is best-effort. The interesting workflows mix both.

8.6.2. Partial Success and Graceful Degradation

Return a structured `WorkflowResult` so callers can branch on partial success.

```
class WorkflowResult(BaseModel):  
    success: bool  
    completed_steps: list[str]  
    errors: list[str]
```

The driver that walks the task list and records both completions and errors into the structured result:

```
async def resilient_workflow(tasks):  
    result = WorkflowResult(success=True, completed_steps=[],  
errors=[])  
  
    for task in tasks:  
        try:  
            await execute_task(task)  
            result.completed_steps.append(task.name)  
        except Exception as e:  
            result.success = False  
            result.errors.append(f"{task.name}: {str(e)}")  
  
    return result
```

`success: bool` is a single coarse signal — augment with a per-step status enum (`completed`, `skipped`, `failed`) when the workflow has more than a handful of steps and you need fine-grained dashboarding.

8.6.3. Error Recovery Strategies

Different exceptions deserve different recovery paths.

```
async def step_with_recovery(step_func, input_data):
    try:
        return await step_func(input_data)
    except NetworkError:
        await asyncio.sleep(1)
        return await step_func(input_data)           # retry
    except ValidationError:
        return await fallback_step(input_data)        # alternate path
    except Exception:
        return get_default_value()                    # last resort
```

The full version with the explanatory comments and a logger call on the last-resort branch:

```
async def step_with_recovery(step_func, input_data):
    try:
        return await step_func(input_data)
    except NetworkError:
        # Retry on network issues
        await asyncio.sleep(1)
        return await step_func(input_data)
    except ValidationError as e:
        # Try alternate approach
        return await fallback_step(input_data)
    except Exception:
        # Log and return default
        logger.error(f"Step {step_func.__name__} failed")
        return get_default_value()
```

The order of `except` clauses is the **policy** — `NetworkError` first because retrying a network blip is cheap and likely to succeed; `ValidationError` next because validation failures usually need a different path, not a retry; the bare `Exception`

catch-all last as the failsafe. Reordering changes the semantics of the workflow, not just the code style.

8.7. Checkpointing and Resumption

Checkpointing lets long-running workflows resume from known states.

8.7.1. Why Checkpointing Matters

Long-running workflows must survive interruptions.

Use Cases:

- Server restarts or crashes
- Debugging specific steps
- Recovering from transient failures
- Avoiding rework on expensive steps

8.7.2. Implementing Checkpoints

Save workflow state at key points so the workflow can resume after a restart.

```
class WorkflowCheckpoint:
    def __init__(self, workflow_id: str):
        self.file = Path(f"checkpoints/{workflow_id}.json")

    def save(self, step_name: str, data: dict):
        self.file.parent.mkdir(exist_ok=True)
        self.file.write_text(json.dumps({"last_step": step_name,
"data": data}))
```

The full class with the matching `load` method:

```
import json
from pathlib import Path

class WorkflowCheckpoint:
    def __init__(self, workflow_id: str):
        self.file = Path(f"checkpoints/{workflow_id}.json")

    def save(self, step_name: str, data: dict):
        checkpoint = {"last_step": step_name, "data": data}
        self.file.parent.mkdir(exist_ok=True)
        with open(self.file, 'w') as f:
            json.dump(checkpoint, f)

    def load(self) -> dict | None:
        if not self.file.exists():
            return None
        with open(self.file) as f:
            return json.load(f)
```

File-per-workflow checkpointing is fine for prototypes. Production deployments should use a real durable store (Redis, Postgres, S3) so checkpoints survive container churn and aren't lost when the disk goes away.

8.7.3. Resumable Workflow Pattern

Skip already-completed steps on resume; checkpoint after each one succeeds.

```
async def resumable_workflow(workflow_id: str, data: dict):
    cp = WorkflowCheckpoint(workflow_id)
    all_steps = ["extract", "analyze", "summarize", "format"]
    saved = cp.load()
    ctx, steps = (saved['data'],
all_steps[all_steps.index(saved['last_step'])+1:]) \
                if saved else (data, all_steps)
    for step in steps:
        ctx = await execute_step(step, ctx)
        cp.save(step, ctx)
    return ctx
```

The full version with explicit branches and per-step assignment, which reads more clearly than the conditional expression on the slide:

```
async def resumable_workflow(workflow_id: str, data: dict):
    checkpoint = WorkflowCheckpoint(workflow_id)
    all_steps = ["extract", "analyze", "summarize", "format"]

    saved = checkpoint.load()
    if saved:
        ctx = saved['data']
        steps = all_steps[all_steps.index(saved['last_step']) + 1:]
    else:
        ctx = data
        steps = all_steps

    for step in steps:
        ctx = await execute_step(step, ctx)
        checkpoint.save(step, ctx)

    return ctx
```

The pattern only works correctly if `execute_step` is **idempotent** — re-running a step on resume must produce the same result as the original run. The next slide covers idempotence in more detail.

8.7.4. Idempotent Operations

Design steps to be safely re-executed.

Idempotent means running the same operation multiple times produces the same result. This is crucial for checkpointing. Example: "Set user status to active" is idempotent; "Increment counter" is not.

8.8. Retry and Fallback Strategies

Retries and fallbacks make workflows robust against transient failures.

8.8.1. Exponential Backoff for Retries

Retry with increasing delays to handle transient failures.

```
async def retry_with_backoff(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return await func()
        except Exception:
            if attempt == max_retries - 1:
                raise
            await asyncio.sleep(2 ** attempt)  # 1s, 2s, 4s
```

The full version with logging on each retry attempt:

```
import asyncio

async def retry_with_backoff(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return await func()
        except Exception as e:
            if attempt == max_retries - 1:
                raise # Final attempt failed

            wait_time = 2 ** attempt # 1s, 2s, 4s
            print(f"Retry {attempt + 1}/{max_retries} after {wait_time}s")
            await asyncio.sleep(wait_time)
```

For production systems, layer **jitter** on top of the exponential growth — `await asyncio.sleep(2 ** attempt) + random.uniform(0, 1)` — so simultaneous failures don't all retry at exactly the same moment and overwhelm the recovering service. Libraries like `tenacity` ship this and policy management out of the box.

8.8.2. Circuit Breaker Pattern

Three states — **closed** (calls flow), **open** (calls fast-fail), **half-open** (one trial call after a cooldown).

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5, cooldown_seconds=30.0):
        self.failure_threshold = failure_threshold
        self.cooldown_seconds = cooldown_seconds
        self.failure_count = 0
        self.is_open = False
        self.last_failure_time: float | None = None
```

The `call(...)` method is where the state machine lives — fast-fail when open and within cooldown, allow a half-open trial after the cooldown expires, reset to closed on success, count failures and trip on threshold:

```
import time

class CircuitBreaker:
    def __init__(self, failure_threshold: int = 5, cooldown_seconds: float = 30.0):
        self.failure_count = 0
        self.failure_threshold = failure_threshold
        self.cooldown_seconds = cooldown_seconds
        self.is_open = False
        self.last_failure_time: float | None = None

    async def call(self, func):
        # If open, check whether the cooldown has elapsed; if so,
        allow one
        # trial call (half-open). Otherwise fail fast.
        if self.is_open:
            elapsed = time.monotonic() - (self.last_failure_time or 0)

            if elapsed < self.cooldown_seconds:
                raise Exception("Circuit breaker is OPEN")
            # Half-open: let the next call through as a trial.

        try:
            result = await func()
            # Success – close the breaker and reset the counter.
            self.is_open = False
            self.failure_count = 0
            self.last_failure_time = None
            return result
        except Exception:
            self.failure_count += 1
            self.last_failure_time = time.monotonic()
            if self.failure_count >= self.failure_threshold:
                self.is_open = True
            raise
```

The breaker resets to closed on a **successful** trial, not after a fixed cooldown — the cooldown only gates **whether to try**. This distinction is the load-bearing part of the pattern: it lets the breaker discover the service has recovered without flooding it during the recovery window.

8.8.3. Fallback Actions and Alternate Paths

A cascade of fallbacks: each `except` clause names a **specific** failure mode and the path it triggers.

```
async def workflow_with_fallbacks(data):
    try:
        return await expensive_accurate_agent.run(data)
    except TimeoutError:
        return await fast_agent.run(data)           # quality ↓
    except QuotaExceeded:
        return await backup_provider_agent.run(data) # provider
switch
    except Exception:
        return get_cached_response(data) or "Unable to process
request"
```

The full version with the explanatory comments preserved:

```
async def workflow_with_fallbacks(data):
    # Try primary approach
    try:
        return await expensive_accurate_agent.run(data)
    except TimeoutError:
        # Fallback to faster but less accurate
        return await fast_agent.run(data)
    except QuotaExceeded:
        # Fallback to different provider
        return await backup_provider_agent.run(data)
    except Exception:
        # Last resort: return cached or default response
        return get_cached_response(data) or "Unable to process
request"
```

Naming the exception types is the load-bearing teaching point — `except Exception` alone collapses three different failure modes into one undifferentiated path. Distinct fallbacks per failure mode (timeout → faster model; quota → backup provider; everything else → cache or default) is what makes the pattern resilient rather than just defensive.

8.8.4. Designing Fault-Tolerant Workflows

Best practices for resilient workflows.

Key Principles:

- Make steps idempotent where possible
- Use checkpoints for long-running workflows
- Implement retries with exponential backoff
- Provide fallbacks for critical paths
- Log extensively for debugging
- Monitor failure rates and adjust strategies

8.9. Summary

In this module, you learned how to:

- Design multi-step workflows with sequential, parallel, and conditional execution patterns
- Implement graph-based workflows using DAGs for complex dependency management
- Pass context between workflow steps using structured state objects
- Handle errors gracefully with recovery strategies and partial success patterns
- Implement checkpointing for resumable long-running workflows
- Apply retry mechanisms with exponential backoff and circuit breakers
- Design fallback strategies for fault-tolerant agent systems

These orchestration patterns enable you to build complex, reliable, and maintainable agent workflows that can handle real-world production requirements.

8.10. Reflection Questions

- How do you decide which parts of a workflow can run in parallel versus which must be sequential?
- What are the benefits and risks of using mutable shared state between workflow steps?
- How does checkpointing help in long-running agent workflows, and what are the costs of implementing it?
- When would a circuit breaker pattern be more appropriate than simple retry logic?

8.11. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>

- Python asyncio documentation: <https://docs.python.org/3/library/asyncio.html>
- Pydantic documentation: <https://docs.pydantic.dev>
- Your workflow orchestration platform's official documentation (e.g., Prefect, Temporal)

Multi-Agent Systems

LESSON 9

- ✓ Determine when to employ multiple agents versus a single agent by analyzing use cases
- ✓ Implement agent delegation, escalation, and handoff protocols for task coordination
- ✓ Manage information sharing vs. isolation between agents
- ✓ Design distinct agent roles and responsibilities within a multi-agent system
- ✓ Analyze multi-agent interaction failures and resolve conflicts through structured tracing and protocol refinement

9.1. Module Overview

- Introduction to multi-agent systems
- Single agent vs. multi-agent considerations
- Agent delegation and escalation
- Shared vs. isolated context
- Coordination and handoff protocols
- Designing agent roles and teamwork
- Debugging multi-agent interactions

9.2. Introduction to Multi-Agent Systems

Coordinate multiple specialized AI agents to solve complex problems.

- Division of labor
- Parallel processing
- Modular design

9.2.1. What Are Multi-Agent Systems?

Multiple specialized agents working together on complex tasks.

Key Characteristics:

- **Specialization** – Focused roles
- **Autonomy** – Independent decisions
- **Coordination** – Agents collaborate
- **Distribution** – Work divided across agents

9.2.2. Example: Customer Support System

- Classifier agent → routes requests
- Technical agent → handles tech issues

- Sales agent → handles product questions
- Escalation agent → manages complex cases

9.2.3. Benefits of Multi-Agent Systems

Specialization: Each agent tuned for its task

Parallelism: Independent agents run simultaneously

Modularity: Changes isolated, easier testing

9.2.4. Challenges and Trade-offs

Complexity: More moving parts, complex failure modes

Coordination Overhead: Latency, context passing cost

Accept complexity when: Tasks divide naturally, parallelism helps, or roles need different models

9.3. Single Agent vs. Multi-Agent Considerations

Start simple. Evolve to multi-agent only when benefits justify the cost.

9.3.1. When a Single Agent Is Better

Use one agent for simple or tightly coupled tasks.

Advantages:

- Simpler to build and debug
- Lower latency
- Easier context management

Good Fit:

- Clear-scope queries

- Deep reasoning in one domain
- Real-time conversations

9.3.2. When Multiple Agents Add Value

Use multiple agents when specialization or parallelism improves outcomes.

Advantages:

- Distinct expertise areas
- Parallel execution
- Model choice per role
- Clear escalation paths

Good Fit:

- Research + write + edit pipelines
- Multi-domain queries
- Review workflows
- Complex routing

9.3.3. Decision Framework

Four questions for choosing single-agent vs. multi-agent:

1. Can the task be naturally decomposed into distinct roles?
2. Do different subtasks benefit from different models or prompts?
3. Can subtasks execute in parallel?
4. Is coordination overhead acceptable?

A **yes** on most points argues for multi-agent; a **no** on most argues for keeping it simple.

The full decision flow with both branches called out:

1. Can the task be naturally decomposed into distinct roles?
Yes → Consider multi-agent
No → Stick with single agent
2. Do different subtasks benefit from different models/prompts?
Yes → Multi-agent enables optimization
No → Single agent is simpler
3. Can subtasks execute in parallel?
Yes → Multi-agent improves speed
No → Single agent avoids overhead
4. Is coordination overhead acceptable?
Yes → Multi-agent is viable
No → Single agent is safer

The most common new-builder mistake is reaching for multi-agent before single-agent quality is solid. If you can't articulate which question above is forcing the decomposition, default to a single agent.

9.4. Agent Delegation and Escalation

Route work to specialized agents based on task requirements. Clear routing logic and well-defined capabilities are essential.

9.4.1. Basic Delegation Pattern

One agent routes work to specialized agents based on task type.

```
async def delegate_request(user_input: str):
    route = await router_agent.run(f"Classify: {user_input}")
    if route.output.route == 'technical':
        return await technical_agent.run(user_input)
    elif route.output.route == 'sales':
        return await sales_agent.run(user_input)
    return "Please contact support"
```

The full pattern: a `RouterResult` schema constrains what the router can say, and the three specialist agents are declared once at module scope. The router is sized down (`AI_MODEL_FAST`) because classification is cheap; the specialists use `AI_MODEL_CAPABLE` because their answers ship to users.

```
import os
from pydantic import BaseModel

class RouterResult(BaseModel):
    route: str # 'technical', 'sales', 'general'

router_agent = Agent(
    os.getenv('AI_MODEL_FAST', 'openai:gpt-5.4-mini'),
    output_type=RouterResult,
)
technical_agent = Agent(os.getenv('AI_MODEL_CAPABLE',
'openai:gpt-5.4'))
sales_agent = Agent(os.getenv('AI_MODEL_CAPABLE', 'openai:gpt-5.4'))

async def delegate_request(user_input: str):
    route = await router_agent.run(f"Classify: {user_input}")

    if route.output.route == 'technical':
        return await technical_agent.run(user_input)
    elif route.output.route == 'sales':
        return await sales_agent.run(user_input)
    else:
        return "Please contact support"
```

Constraining `RouterResult.route` to a `Literal['technical', 'sales', 'general']` instead of a free `str` makes the routing decision a typed contract — the router can no longer return a route the dispatch code doesn't handle.

9.4.2. Hierarchical Escalation

Escalate from simple to complex agents based on assessed difficulty.

```
async def escalate_as_needed(task: str):
    assessment = await assessor.run(f"Assess complexity: {task}")
    if assessment.output.complexity == 'simple':
        return await simple_agent.run(task)
    return await expert_agent.run(task)
```

The `TaskAssessment` schema and the three agent instances:

```
class TaskAssessment(BaseModel):
    complexity: str
    reasoning: str

assessor = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    output_type=TaskAssessment,
)
simple_agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))
expert_agent = Agent(os.getenv('AI_MODEL_CAPABLE',
    'openai:gpt-5.4'))
```

This pattern pairs naturally with the cascading-model approach in the **Evaluation and Guardrails** module, but with one important difference: cascading reacts to **output quality**, escalation reacts to **input complexity**. Cascading retries when the cheap model produced a bad answer; escalation routes upstream so the expensive model only sees inputs that actually warrant it.

9.4.3. Chaining Multiple Specialists

Pass work through a sequence of specialized agents.

```
async def content_pipeline(topic: str):  
    research = await research_agent.run(f"Research {topic}")  
    draft    = await writer_agent.run(f"Write article on {topic}.  
Research: {research.output}")  
    final    = await editor_agent.run(f"Edit this article:  
{draft.output}")  
    return final.output
```

The full version with the three agent declarations. Each step takes the previous step's `.output` and folds it into the next prompt — the simplest form of inter-agent context passing.

```
import os

research_agent = Agent(os.getenv('AI_MODEL_CAPABLE',
'openai:gpt-5.4'))
writer_agent   = Agent(os.getenv('AI_MODEL_CAPABLE',
'openai:gpt-5.4'))
editor_agent   = Agent(os.getenv('AI_MODEL_CAPABLE',
'openai:gpt-5.4'))

async def content_pipeline(topic: str):
    # Step 1: Research
    research = await research_agent.run(f"Research {topic}")

    # Step 2: Write
    draft = await writer_agent.run(
        f"Write article on {topic}. Research: {research.output}"
    )

    # Step 3: Edit
    final = await editor_agent.run(
        f"Edit this article: {draft.output}"
    )

    return final.output
```

Three sequential LLM calls means **three times the latency** of a single agent call. Chains earn their cost only when the role separation produces measurably better output than a single agent with a longer prompt — measure before assuming.

9.5. Shared vs. Isolated Context

Context choice affects independence, testing, and coordination overhead.

9.5.1. Context Isolation Strategy

Each agent maintains separate context for independence.

Benefits:

- No cross-agent interference
- Easier reasoning and debugging
- Better for parallel execution

```
agent_a_result = await agent_a.run("Task A", deps=ContextA(...))
agent_b_result = await agent_b.run("Task B", deps=ContextB(...))

combined = combine_results(agent_a_result, agent_b_result)
```

The application-level merge function the slide alludes to. Concatenation is a teaching default; production pipelines typically merge by structured field rather than string-joining `.output` blobs.

```
def combine_results(*results) -> str:
    """Concatenate the .output of each AgentRunResult into a single
    string."""
    return "\n\n".join(r.output for r in results)
```

9.5.2. Shared Context Strategy

Agents access common context for coordination.

Benefits:

- Agents see each other's work
- Reduces redundant computation
- Natural information flow

```
shared = SharedContext(user_id="123")

result = await research_agent.run("Research topic", deps=shared)
shared.results.append(result.output)
draft = await writer_agent.run("Write article", deps=shared)
```

The schema definition for the shared state object:

```
from pydantic import BaseModel

class SharedContext(BaseModel):
    user_id: str
    results: list[str] = []
```

Mutable shared state across concurrent agents needs the same care as any other shared mutable state — if the agents run in parallel rather than sequentially, you need a lock around `shared.results.append(...)` or you'll lose appends under contention. Sequential pipelines like the one above sidestep the problem.

9.5.3. Hybrid Approach

Share specific data, isolate the rest.

```
class TeamContext(BaseModel):
    shared_data: dict[str, Any] # Accessible to all
    agent_private: dict[str, Any] # Agent-specific state
```

The driver function shows how each agent receives the shared data plus its own private slot under its name:

```
from typing import Any
from pydantic import BaseModel
from pydantic_ai import Agent

class TeamContext(BaseModel):
    shared_data: dict[str, Any]
    agent_private: dict[str, Any]

async def run_with_hybrid_context(agent: Agent, agent_name: str,
task: str):
    ctx = TeamContext(
        shared_data={"user_id": "123", "topic": "AI"},
        agent_private={agent_name: {}},
    )
    return await agent.run(task, deps=ctx)
```

The `agent_private` dict keyed by agent name is the simplest implementation — production code often promotes this to a typed sub-model per agent so each role's private state is itself schema-validated.

9.6. Coordination and Handoff Protocols

Handoff protocols define how agents communicate and transfer control. Well-structured protocols ensure reliable information transfer.

9.6.1. Structured Handoff Messages

A typed schema turns agent-to-agent communication into a contract.

```
class Handoff(BaseModel):
    from_agent: str
    to_agent: str
    task: str
    context: dict[str, Any]
    priority: str # 'low', 'medium', 'high'
```

The handoff dispatcher and the registry pattern that backs `get_agent`. The registry is populated at application startup so dispatch is just a dict lookup, not a per-handoff agent construction.

```
from typing import Any
from pydantic import BaseModel
from pydantic_ai import Agent

class Handoff(BaseModel):
    from_agent: str
    to_agent: str
    task: str
    context: dict[str, Any]
    priority: str

# Registry of named agents – populated at application startup.
_AGENTS: dict[str, Agent] = {}

def get_agent(name: str) -> Agent:
    """Look up a registered agent by name."""
    return _AGENTS[name]

async def handoff_task(handoff: Handoff):
    target_agent = get_agent(handoff.to_agent)
    result = await target_agent.run(
        f"Task: {handoff.task}. Context: {handoff.context}"
    )
    return result
```

For tighter type-safety, replace the free `priority: str` with a `Literal['low', 'medium', 'high']` and the free `to_agent: str` with a `Literal[...]` over the registry's keys.

9.6.2. Request-Response Protocol

Pair every request with a typed response that carries the originating `request_id`.

```
class AgentRequest(BaseModel):
    request_id: str
    task: str

class AgentResponse(BaseModel):
    request_id: str
    success: bool
    result: Any
```

The driver that takes a request, runs it through the responder, and returns the typed response. Carrying `request_id` on the response is what lets the caller correlate when many requests are in flight concurrently.

```
from typing import Any
from pydantic import BaseModel

async def agent_communication(responder, request):
    response = await responder.run(request.task)
    return AgentResponse(
        request_id=request.request_id,
        success=True,
        result=response.output,
    )
```

For real systems, expand `AgentResponse` with an `error: str | None` field and set `success=False` when the responder raises — the slide example always returns `success=True`, which masks failures.

9.6.3. Broadcasting to Multiple Agents

Send the same task to multiple agents in parallel, then pick or vote on the best result.

```
async def broadcast_task(agents: list, task: str):
    results = await asyncio.gather(
        *[agent.run(task) for agent in agents],
        return_exceptions=True,
    )
    valid = [r for r in results if not isinstance(r, Exception)]
    return select_best(valid)
```

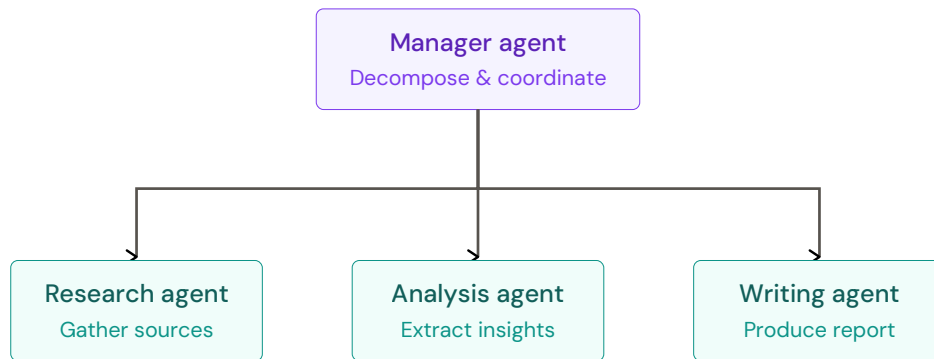
The `select_best` helper. In production, replace this with semantic scoring, majority voting, or an LLM-as-judge call. The default heuristic — longest output as a stand-in for "most thorough" — is a teaching default, not a recommendation.

```
import asyncio

def select_best(results):
    return max(results, key=lambda r: len(r.output))
```

`return_exceptions=True` is the load-bearing part of this pattern — without it, a single failing agent would propagate its exception out of `asyncio.gather` and discard the **other agents'** successful results. With it, the failure becomes one entry in the `results` list that `valid = ...` filters out.

9.7. Designing Agent Roles & Teamwork



9.7.1. Defining Clear Agent Roles

Each agent should have a focused, well-defined responsibility.

Example Roles:

- **Researcher** – Gathers information
- **Analyzer** – Processes data
- **Writer** – Creates content
- **Reviewer** – Checks quality
- **Coordinator** – Manages workflow

9.7.2. Role Definition Example

```
researcher = Agent(  
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),  
    system_prompt="Research specialist: gather info, cite sources,  
focus on accuracy."  
)  
  
writer = Agent(  
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),  
    system_prompt="Content writer: transform research into engaging  
articles."  
)
```

9.7.3. Complementary Agent Skills

Design agents with complementary strengths.

Generalist + Specialist

- Generalist handles routine cases
- Specialist handles complex cases
- Escalation based on difficulty

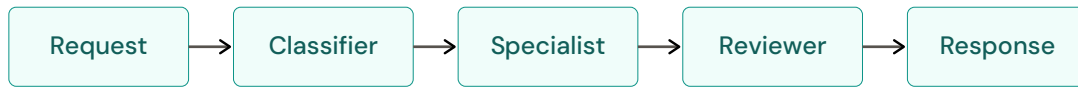
Creator + Critic

- Creator agent generates content
- Critic agent reviews and improves
- Iterative refinement loop

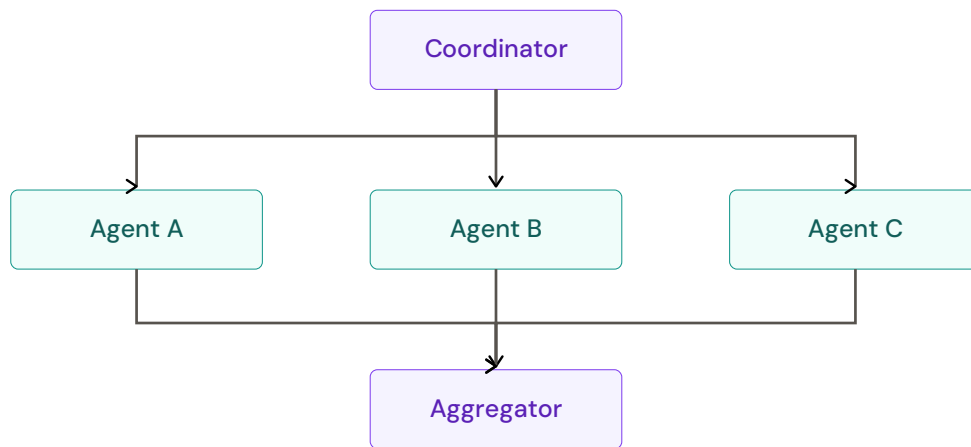
Parallel Specialists

- Multiple domain experts work simultaneously
- Results aggregated by coordinator
- Best for multi-domain tasks

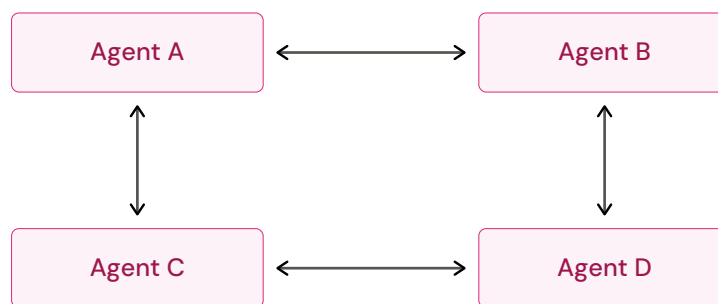
9.7.4. Sequential Pipeline



9.7.5. Hub-and-Spoke



9.7.6. Peer-to-Peer



9.8. Debugging Multi-Agent Interactions

Coordination failures, infinite loops, and unclear error sources create unique challenges. Systematic strategies help.

9.8.1. Common Multi-Agent Issues

- **Infinite Loops** – Agents delegating back and forth → track delegation, detect loops
- **Dropped Context** – Info lost at handoff → use structured protocols
- **Conflicting Decisions** – Contradictory outputs → add conflict resolution
- **Cascading Failures** – One agent breaks downstream → error boundaries, fallbacks

9.8.2. Tracing Multi-Agent Execution

Wrap every agent call so logs always carry the agent name.

```
async def traced_agent_run(agent_name: str, agent, task: str):
    logger.info(f"[{agent_name}] Starting: {task[:50]}...")
    try:
        result = await agent.run(task)
        logger.info(f"[{agent_name}] Success: {result.output[:50]}...")
        return result
    except Exception as e:
        logger.error(f"[{agent_name}] Failed: {e}")
        raise
```

The module-level logger setup and an example call site:

```
import logging

logger = logging.getLogger(__name__)

# Usage
research_result = await traced_agent_run("Researcher",
research_agent, task)
writer_result    = await traced_agent_run("Writer", writer_agent,
task)
```

Tagging every log line with `[agent_name]` is the cheapest possible distributed-tracing primitive: when a multi-agent flow misbehaves, `grep '[Researcher]'` separates that agent's lifecycle from the rest. For real distributed traces, layer the OpenTelemetry pattern from the **Testing and Production Deployment** module on top of this — the agent name becomes a span attribute.

9.8.3. Debugging Techniques

- **Visualize execution** – Sequence diagrams of interactions
- **Unit test agents** – Isolate and verify each role
- **Integration test** – Progressively validate handoffs
- **Monitor metrics** – Utilization, latency, failures

9.9. Summary

In this module, you learned how to:

- Evaluate when to use multiple agents versus a single agent based on task complexity and specialization needs
- Implement delegation patterns with routing and escalation to leverage specialized agents effectively
- Manage shared versus isolated context to balance coordination and independence

- Design structured handoff protocols for reliable agent-to-agent communication
- Define clear agent roles and complementary skills for effective teamwork
- Debug multi-agent interactions by tracing execution, detecting loops, and resolving conflicts

Multi-agent systems enable sophisticated workflows but require careful design of coordination protocols, role definitions, and error handling to be successful.

9.10. Reflection Questions

- How would you decide whether a task requires a multi-agent system or can be handled by a single agent with multiple tools?
- What are the risks of shared context between agents, and how would you mitigate them in a concurrent environment?
- How would you design a debugging strategy for a multi-agent system where agents are producing conflicting outputs?
- What role does structured handoff messaging play in preventing information loss between agents?

9.11. Additional Resources

- Pydantic AI documentation: <https://ai.pydantic.dev>
- Python asyncio documentation: <https://docs.python.org/3/library/asyncio.html>
- OpenAI API documentation: <https://platform.openai.com/docs>
- Anthropic API documentation: <https://docs.anthropic.com>

Testing and Production Deployment

LESSON 10

- ✓ Set up and utilize simulated model backends or stubs to test AI agents in a controlled manner
- ✓ Write effective deterministic tests for agent logic, verifying functionality despite stochastic model outputs
- ✓ Capture and analyze agent execution traces to debug and verify agent behavior step-by-step
- ✓ Integrate observability tools for real-time monitoring of agents and detecting issues in production
- ✓ Deploy AI agents as scalable FastAPI endpoints and implement usage tracking for cost management

10.1. Module Overview

- Introduction to testing and deployment challenges
- Testing agents with simulated backends
- Deterministic testing for non-deterministic systems
- Capturing and inspecting agent runs
- Observability and tracing integration
- Serving agents via FastAPI
- Usage tracking and cost management
- Security and deployment best practices

10.2. Introduction to Testing & Deployment

- Non-determinism defies traditional assertions
- External LLM APIs add latency and cost
- Production demands observability and safeguards

10.2.1. Unique Challenges: Non-Determinism

- Same input can produce different outputs
- Model responses vary across runs
- Difficult to write traditional assertions

10.2.2. Unique Challenges: External Dependencies

- Reliance on third-party LLM APIs
- Network latency and failures
- Rate limits and costs

10.2.3. Unique Challenges: Complex Behavior

- Multi-step reasoning processes

- Tool invocations with side effects
- State management across turns

10.2.4. Testing Strategy: Unit Testing

- Test agent logic with mocked models
- Verify tool definitions and handlers
- Check result validation and parsing

10.2.5. Testing Strategy: Integration Testing

- Test with real models in staging
- Verify end-to-end workflows
- Validate error handling

10.2.6. Testing Strategy: Production Monitoring

- Continuous observability
- Automated anomaly detection
- User feedback collection

10.2.7. Production Concerns: Performance & Reliability

Performance:

- Response latency requirements
- Concurrent request handling

Reliability:

- Error handling and retries
- Graceful degradation

10.2.8. Production Concerns: Cost & Security

Cost Management:

- Token usage tracking
- Budget limits and alerts

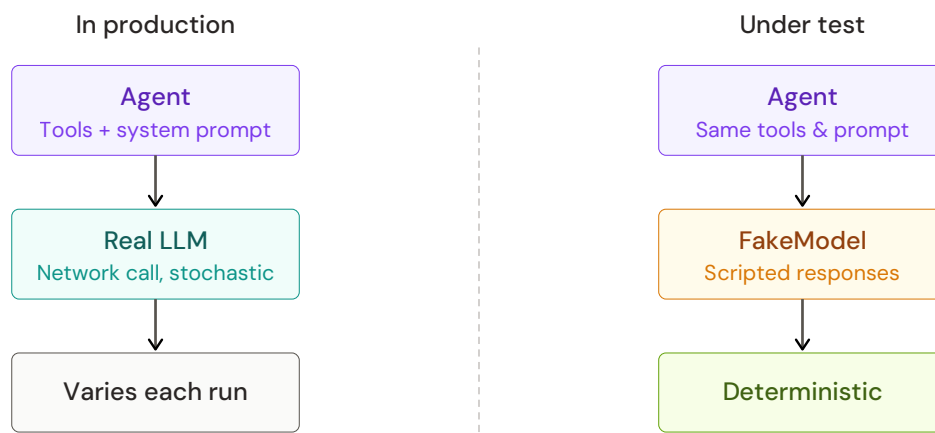
Security:

- API key management
- Input validation and rate limiting

10.3. Testing Agents with Simulated Backends

- Fast feedback without network calls
- Deterministic outputs for assertions
- No API costs during CI

10.3.1. Why Mock LLM Responses?



10.3.2. Benefits of Mocking the LLM

- **Fast** – No network calls
- **Free** – No API costs
- **Deterministic** – Predictable outputs
- **Offline** – No internet required
- **Controlled** – Test specific scenarios

10.3.3. Using TestModel from Pydantic AI

A built-in test model that returns deterministic responses without an API call.

```
test_model = TestModel(custom_output_text="Mocked response")
agent = Agent(test_model)

result = agent.run_sync("Any input")
assert result.output == "Mocked response"
```

Prefer `TestModel` over hand-rolling a `Model` subclass —

`pydantic_ai.models.Model` is an abstract base with multiple required methods (`request`, `model_name`, `system`), so a custom subclass is an advanced pattern not covered in this course.

Two ways to wire `TestModel` in: pass it directly to `Agent(...)` (shown on the slide), or use `Agent.override(model=test_model)` as a `with` block to swap it into an **existing** agent for the scope of a single test — the override pattern is the one the **Testing an AI Agent with Simulated Backends** lab standardizes on.

```

from pydantic_ai import Agent
from pydantic_ai.models.test import TestModel

def test_agent_logic():
    test_model = TestModel(custom_output_text="Mocked response")
    agent = Agent(test_model)
    result = agent.run_sync("Any input")

    assert result.output == "Mocked response"

    # To verify the model was actually consulted, inspect the
    message
    # history rather than a call-counter attribute (TestModel does
    not
    # track calls itself).
    messages = result.all_messages()
    assert len(messages) == 2 # one ModelRequest + one
    ModelResponse

```

10.3.4. Testing Tool Calls

Verify that agents invoke the right tool with the right arguments.

```

async def test_tool_invocation():
    with agent.override(model=TestModel(call_tools='all')):
        result = await agent.run("What's the weather in Paris?")

    tool_calls = [p for m in result.all_messages()
                   for p in getattr(m, 'parts', [])
                   if isinstance(p, ToolCallPart)]
    assert any(c.tool_name == 'get_weather' for c in tool_calls)

```

Tool handlers are routed through Pydantic AI's internal dispatcher, **not** exposed as agent attributes. Assigning `agent.get_weather = AsyncMock(...)` is silently ignored — the assertion would always pass even if the real tool ran. To verify tool calls,

drive the agent with `TestModel(call_tools='all')` (which auto-invokes registered tools) and inspect the messages it produced.

```
from pydantic_ai import Agent
from pydantic_ai.messages import ToolCallPart
from pydantic_ai.models.test import TestModel

agent = Agent(model='openai:gpt-5.4-mini')

@agent.tool_plain
async def get_weather(city: str) -> str:
    """Look up the weather for a city."""
    return f"Weather in {city}: Sunny"

async def test_tool_invocation():
    # Override the real model with TestModel; call_tools='all' tells
    # TestModel to auto-invoke every registered tool.
    with agent.override(model=TestModel(call_tools='all')):
        result = await agent.run("What's the weather in Paris?")

    # Walk the message history to confirm the tool was called.
    tool_calls = [
        part
        for message in result.all_messages()
        for part in getattr(message, 'parts', [])
        if isinstance(part, ToolCallPart)
    ]

    assert any(call.tool_name == 'get_weather' for call in
               tool_calls)
```

When you need to **control** what the tool returns (rather than letting the real implementation run with TestModel-supplied arguments), use `unittest.mock.patch` against the underlying function — for example, `with patch('myapp.weather.lookup', return_value='Mocked: 72F sunny'):`. Patching

at the function level works because the `@agent.tool_plain` decorator wraps the function but still calls it through its original module path.

10.4. Deterministic Testing for Non-Deterministic Systems

- Test behaviors and constraints, not exact outputs
- Isolate deterministic logic from the model
- Detect prompt drift with snapshots

10.4.1. Testing Pattern: Pattern Matching

Focus on structure rather than exact content.

```
import re

def test_agent_output_format():
    result = agent.run_sync("Summarize the text")

    # Test structure rather than exact content
    assert len(result.output) > 10
    assert isinstance(result.output, str)
    assert re.search(r'\w+', result.output) # Contains words
```

10.4.2. Testing Pattern: Property-Based

Assert invariants that must always hold.

```
def test_agent_properties():
    result = agent.run_sync("Generate product name")

    # Test properties that should always hold
    assert result.output is not None
    assert len(result.output) <= 100 # Max length
    assert not result.output.startswith(" ") # No leading spaces
```

10.4.3. Testing Agent Logic in Isolation

Test the non-LLM parts deterministically.

```
def validate_analysis(result: AnalysisResult) -> bool:
    if result.confidence < 0.5:
        return False
    return result.sentiment in ['positive', 'negative', 'neutral']
```

The schema and the unit tests that exercise the validation logic. Because `validate_analysis` is a pure function over `AnalysisResult` instances, it can be tested without invoking the LLM at all — exactly the kind of code that should carry the bulk of an agent's test coverage.

```
from pydantic import BaseModel

class AnalysisResult(BaseModel):
    sentiment: str
    confidence: float

def validate_analysis(result: AnalysisResult) -> bool:
    if result.confidence < 0.5:
        return False
    return result.sentiment in ['positive', 'negative', 'neutral']

def test_validation_logic():
    assert validate_analysis(
        AnalysisResult(sentiment='positive', confidence=0.8)
    ) is True

    assert validate_analysis(
        AnalysisResult(sentiment='invalid', confidence=0.9)
    ) is False
```

10.4.4. Snapshot Testing for Prompts

Track changes to system prompts and detect unintended modifications.

```
SYSTEM_PROMPT = "You are a helpful assistant..."

def test_system_prompt_unchanged(prompt_snapshot):
    prompt_snapshot.assert_match(SYSTEM_PROMPT)
    # First run saves snapshot; subsequent runs detect changes.
```

The full scaffolding includes the pytest fixture and the agent declaration. A snapshot test runs once to capture the current value, then on every subsequent run compares against the saved value — a tripwire for accidental prompt edits.

```
import pytest

@pytest.fixture
def prompt_snapshot(snapshot):
    return snapshot

SYSTEM_PROMPT = "You are a helpful assistant..."

agent = Agent(
    os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'),
    system_prompt=SYSTEM_PROMPT,
)

def test_system_prompt_unchanged(prompt_snapshot):
    prompt_snapshot.assert_match(SYSTEM_PROMPT)
```

In larger projects this pattern extends to **response** snapshots (run a `TestModel` – driven scenario, snapshot the full message history). When the snapshot diff fires, it forces a human to decide whether the change is intentional — the right defense against silent prompt drift across PRs.

10.5. Capturing & Inspecting Agent Runs

- Traces surface agent decision paths
- Replays enable step-by-step debugging
- Framework hooks expose usage and messages

10.5.1. Recording Execution Traces

Capture wall-clock timing + full message history on top of every agent run.

```
async def traced_run(agent, user_input: str) -> ExecutionTrace:
    start = datetime.now()
    result = await agent.run(user_input)
    duration = (datetime.now() - start).total_seconds() * 1000
    return ExecutionTrace(
        timestamp=start, input=user_input, output=result.output,
        duration_ms=duration, messages=result.all_messages(),
    )
```

The full message history (including any tool calls) is already on the run result —

`ExecutionTrace` adds wall-clock timing and the original user input on top of that. Useful for batched performance reports and post-mortem debugging.

```
from dataclasses import dataclass, field
from datetime import datetime
from pydantic_ai.messages import ModelMessage

@dataclass
class ExecutionTrace:
    timestamp: datetime
    input: str
    output: str
    duration_ms: float
    messages: list[ModelMessage] = field(default_factory=list)

async def traced_run(agent, user_input: str) -> ExecutionTrace:
    start = datetime.now()
    result = await agent.run(user_input)
    duration = (datetime.now() - start).total_seconds() * 1000

    return ExecutionTrace(
        timestamp=start,
        input=user_input,
        output=result.output,
        duration_ms=duration,
        messages=result.all_messages(),
    )
```

10.5.2. Step-by-Step Replay and Analysis

Walk the captured `ModelMessage` list to inspect every step of an agent run.

```
for message in trace.messages:
    for part in message.parts:
        if isinstance(part, UserPromptPart):
            print(f"user: {part.content[:80]}")
        elif isinstance(part, TextPart):
            print(f"model: {part.content[:80]}")
        elif isinstance(part, ToolCallPart):
            print(f"tool call: {part.tool_name}({part.args})")
```

The full `replay_trace` walker handles every part type (`UserPromptPart`, `TextPart`, `ToolCallPart`, `ToolReturnPart`) and prints request/response framing plus duration:

```

from pydantic_ai.messages import (
    ModelRequest, ModelResponse, TextPart,
    ToolCallPart, ToolReturnPart, UserPromptPart,
)

def replay_trace(trace: ExecutionTrace) -> None:
    print(f"Execution at {trace.timestamp}")
    print(f"Input: {trace.input}")

    for i, message in enumerate(trace.messages, 1):
        kind = 'request' if isinstance(message, ModelRequest) else
        'response'
        print(f"\nMessage {i} ({kind}):")
        for part in message.parts:
            if isinstance(part, UserPromptPart):
                print(f"  user prompt: {part.content[:80]}")
            elif isinstance(part, TextPart):
                print(f"  model text: {part.content[:80]}")
            elif isinstance(part, ToolCallPart):
                print(f"  tool call: {part.tool_name}
({part.args})")
            elif isinstance(part, ToolReturnPart):
                print(f"  tool return: {part.tool_name} ->
{part.content!r}[:80]")

        print(f"\nFinal Output: {trace.output}")
        print(f"Duration: {trace.duration_ms}ms")

```

10.5.3. Using Pydantic AI's Built-in Tracing

Three accessors on every run result expose what the agent did:

```
result = await agent.run("Analyze this data", message_history=[...])

result.all_messages() # full request/response history
result.new_messages() # only what was added by this call
result.usage()         # token counts (method, not attribute)
```

A common bug to call out: `result.usage()` is a **method**, not an attribute — calling it incorrectly as `result.usage` returns the bound method object (truthy, so naive `if result.usage:` checks pass) instead of the `Usage` instance.

```
import os
from pydantic_ai import Agent

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

# Run with tracing enabled
result = await agent.run(
    "Analyze this data",
    message_history=[...]
)

# Access run details
print(f"Messages: {result.all_messages()}")
print(f"New messages: {result.new_messages()}")
print(f"Usage: {result.usage()}") # Token counts
```

10.6. Observability and Tracing Integration

- Logging reveals live behavior
- Metrics expose trends and SLOs
- Distributed tracing correlates failures

10.6.1. Structured Logging for Agents

Emit JSON-formatted events so logs are machine-queryable in production.

```
async def logged_agent_run(agent, user_input: str, user_id: str):
    logger.info(json.dumps({"event": "agent_request", "user_id":
user_id}))
    try:
        result = await agent.run(user_input)
        logger.info(json.dumps({"event": "agent_success", "usage":
result.usage()}))
        return result
    except Exception as e:
        logger.error(json.dumps({"event": "agent_error", "error":
str(e)}))
        raise
```

The full module-level logger setup that pairs with the function above:

```
import logging
import json

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
```

In production, ship to a structured backend (CloudWatch, Datadog, Honeycomb) so the JSON `event` field becomes a queryable column. Avoid logging the **raw user input** directly — it may contain PII; log a length or hash instead, or run it through the redaction guardrail from the **Evaluation and Guardrails** module first.

10.6.2. Metrics and Monitoring

Track key performance indicators per agent.

```
@dataclass
class AgentMetrics:
    total_requests: int = 0
    successful_requests: int = 0
    total_tokens: int = 0
```

The collector keeps a per-agent `AgentMetrics` instance. In production, export these counters to Prometheus, StatsD, or your APM vendor of choice rather than holding them in process memory.

```
from dataclasses import dataclass
from collections import defaultdict

@dataclass
class AgentMetrics:
    total_requests: int = 0
    successful_requests: int = 0
    total_tokens: int = 0

class MetricsCollector:
    def __init__(self):
        self.metrics = defaultdict(AgentMetrics)

    def record_request(self, agent_name: str, success:
bool, tokens: int):
        m = self.metrics[agent_name]
        m.total_requests += 1
        if success:
            m.successful_requests += 1
        m.total_tokens += tokens
```

10.6.3. OpenTelemetry Integration

Wrap an agent run in a span so traces correlate across services.

```
async def traced_agent_call(agent, user_input: str):
    with tracer.start_as_current_span("agent.run") as span:
        span.set_attribute("agent.input", user_input[:100])
        result = await agent.run(user_input)
        span.set_attribute("agent.tokens",
            result.usage().total_tokens)
    return result
```

The full version handles success/error status and exception recording — the OTel idioms that make a span useful in a real trace UI (Jaeger, Tempo, Honeycomb).

```
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

tracer = trace.get_tracer(__name__)

async def traced_agent_call(agent, user_input: str):
    with tracer.start_as_current_span("agent.run") as span:
        span.set_attribute("agent.input", user_input[:100])

        try:
            result = await agent.run(user_input)
            span.set_attribute("agent.tokens",
                result.usage().total_tokens)
            span.set_status(Status(StatusCode.OK))
            return result
        except Exception as e:
            span.set_status(Status(StatusCode.ERROR, str(e)))
            span.record_exception(e)
            raise
```

The `agent.input` attribute is truncated to 100 chars on the slide; tune that limit to your retention policy. Trace backends typically charge by indexed attribute size, so storing a 10kb prompt in a span attribute on every request is rarely the right move.

10.7. Serving Agents via FastAPI

- REST endpoints integrate with any client
- Async support for concurrent requests
- Auto-generated docs from type hints

10.7.1. Creating Agent Web Service Endpoints

Deploy agents as REST APIs with typed request/response.

```
@app.post("/agent/run")
async def run_agent(request: AgentRequest) -> AgentResponse:
    try:
        result = await agent.run(request.message)
        return AgentResponse(response=result.output)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

The full module-level setup with the request/response schemas — FastAPI auto-generates OpenAPI docs from the type annotations:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import os

app = FastAPI()
agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

class AgentRequest(BaseModel):
    message: str

class AgentResponse(BaseModel):
    response: str

@app.post("/agent/run")
async def run_agent(request: AgentRequest):
    try:
        result = await agent.run(request.message)
        return AgentResponse(response=result.output)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

10.7.2. Handling Concurrency and Errors

A semaphore caps concurrent agent calls; a global handler catches everything else.

```
agent_semaphore = Semaphore(10)

@app.post("/agent/run")
async def run_with_concurrency_control(request: AgentRequest):
    async with agent_semaphore:
        result = await agent.run(request.message)
        return {"response": result.output}
```

The full version with the global exception handler that returns a structured 500 for any unhandled error:

```
from asyncio import Semaphore
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()
agent_semaphore = Semaphore(10)

@app.post("/agent/run")
async def run_with_concurrency_control(request: AgentRequest):
    async with agent_semaphore:
        result = await agent.run(request.message)
        return {"response": result.output}

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc:
Exception):
    return JSONResponse(
        status_code=500,
        content={"error": str(exc)}
    )
```

The semaphore limit (10) caps in-process concurrency only — for multi-instance deployments behind a load balancer, the **real** concurrency limit is `instances × per-instance limit`, and you may also need an upstream rate limit on the LLM provider's API key.

10.7.3. Streaming Responses

Stream agent output as Server-Sent Events for incremental UI rendering.

```
@app.post("/agent/stream")
async def stream_agent_response(request: AgentRequest):
    async def generate():
        async with agent.run_stream(request.message) as stream:
            async for chunk in stream:
                yield f"data: {chunk}\n\n"
    return StreamingResponse(generate(), media_type="text/event-stream")
```

The full version with the import lines:

```
from fastapi.responses import StreamingResponse

@app.post("/agent/stream")
async def stream_agent_response(request: AgentRequest):
    async def generate():
        async with agent.run_stream(request.message) as stream:
            async for chunk in stream:
                yield f"data: {chunk}\n\n"

    return StreamingResponse(
        generate(),
        media_type="text/event-stream",
    )
```

The `data: {chunk}\n\n` framing is the SSE wire protocol — browsers expose it via the `EventSource` API. For unstructured text streams use this pattern; for **structured** (typed) streaming output, see the **Conversations, Memory, and State** module's `agent.run_stream` + `stream_output()` coverage.

10.8. Usage Tracking and Cost Management

- Track tokens per user and request
- Enforce daily or per-tenant budgets

- Plan capacity from historical usage

10.8.1. Request and Token Tracking

Log every request with timestamp, user, tokens, and cost.

```
class UsageTracker:
    def __init__(self):
        self.usage_log = []

    def log_request(self, user_id: str, tokens: int, cost: float):
        self.usage_log.append({
            "timestamp": datetime.now(timezone.utc),
            "user_id": user_id, "tokens": tokens, "cost": cost,
        })
```

The full tracker with the `get_user_usage` aggregator. Note `datetime.utcnow()` was deprecated in Python 3.12; use a timezone-aware UTC instant via `datetime.now(timezone.utc)`.

```
from datetime import datetime, timedelta, timezone

class UsageTracker:
    def __init__(self):
        self.usage_log = []

    def log_request(self, user_id: str, tokens: int, cost: float):
        self.usage_log.append({
            "timestamp": datetime.now(timezone.utc),
            "user_id": user_id,
            "tokens": tokens,
            "cost": cost,
        })

    def get_user_usage(self, user_id: str, days: int = 30):
        cutoff = datetime.now(timezone.utc) - timedelta(days=days)
        user_logs = [
            log for log in self.usage_log
            if log["user_id"] == user_id and log["timestamp"] >
cutoff
        ]
        return {
            "total_tokens": sum(log["tokens"] for log in user_logs),
            "total_cost": sum(log["cost"] for log in user_logs),
        }
```

10.8.2. Cost Control Measures

Enforce a daily budget at the endpoint, returning HTTP 429 when exhausted.

```
@app.post("/agent/run")
async def run_with_budget(request: AgentRequest):
    if not cost_controller.check_budget(0.05):
        raise HTTPException(429, "Daily budget exceeded")
    result = await agent.run(request.message)
    cost_controller.record_usage(calculate_cost(result))
    return {"response": result.output}
```

The full `CostController` class with the daily-rollover logic. The "today" boundary uses UTC consistently — pair both `date()` calls with the same timezone so the rollover fires at the same wall-clock instant for every server instance.

```
class CostController:
    def __init__(self, daily_budget: float):
        self.daily_budget = daily_budget
        self.today_cost = 0.0
        self.last_reset = datetime.now(timezone.utc).date()

    def check_budget(self, estimated_cost: float) -> bool:
        today = datetime.now(timezone.utc).date()
        if today > self.last_reset:
            self.today_cost = 0.0
            self.last_reset = today
        return self.today_cost + estimated_cost <= self.daily_budget

    def record_usage(self, actual_cost: float):
        self.today_cost += actual_cost
```

For multi-instance deployments, hold `today_cost` in Redis or a database so the budget is shared across replicas. An in-process counter only enforces a per-instance limit, which is rarely what you actually want.

10.9. Security & Deployment Best Practices

- Authenticate and rate-limit endpoints
- Validate and sanitize all inputs
- Scale with caching and load balancing

10.9.1. Securing Agent Endpoints

Require a Bearer token on every request via FastAPI's `Depends` injection.

```
@app.post("/agent/run")
async def secured_endpoint(
    request: AgentRequest,
    api_key: str = Depends(verify_api_key),
):
    result = await agent.run(request.message)
    return {"response": result.output}
```

The verifier and the imports it depends on:

```
from fastapi import Depends, HTTPException, Security
from fastapi.security import HTTPBearer,
HTTPAuthorizationCredentials

security = HTTPBearer()

def verify_api_key(credentials: HTTPAuthorizationCredentials =
Security(security)):
    if credentials.credentials != os.getenv("API_KEY"):
        raise HTTPException(401, "Invalid API key")
    return credentials.credentials
```

Comparing the user-supplied credential against `os.getenv("API_KEY")` with `!=` is timing-sensitive — for production, use `secrets.compare_digest(a, b)` to avoid

leaking the comparison via response-time side channels. The course example uses `!=` for readability; flag the production hardening when you teach this slide.

10.9.2. Input Validation and Sanitization

Use Pydantic's `@field_validator` to reject bad inputs before the handler runs.

```
class SafeAgentRequest(BaseModel):
    message: str

    @field_validator('message')
    @classmethod
    def validate_message(cls, v):
        if len(v) > 10000:
            raise ValueError('Message too long')
        return v.strip()
```

The full validator with empty-message and pattern checks:

```
from pydantic import BaseModel, field_validator

class SafeAgentRequest(BaseModel):
    message: str

    @field_validator('message')
    @classmethod
    def validate_message(cls, v):
        if len(v) > 10000:
            raise ValueError('Message too long')

        if len(v.strip()) == 0:
            raise ValueError('Message cannot be empty')

        blocked_patterns = ['<script>', 'javascript:', 'onerror=']
        for pattern in blocked_patterns:
            if pattern.lower() in v.lower():
                raise ValueError('Invalid content detected')

        return v.strip()
```

The `<script>` / `javascript:` / `onerror=` checks are **web-injection** defenses — appropriate when agent output flows into a browser. They do **not** protect against **prompt** injection (instructions hidden in user input that try to redirect the agent); for that, see the input-guardrail patterns in the **Evaluation and Guardrails** module.

10.9.3. Scaling Strategies Overview

Horizontal Scaling:

- Run multiple instances behind load balancer
- Use containerization (Docker, Kubernetes)

Rate Limiting:

- Limit requests per user/IP using Redis for distributed tracking

10.9.4. Scaling Strategies: Caching

Cache repeated prompts to cut latency and cost — but **not** with `@lru_cache`, which doesn't support async functions.

```
@app.post("/agent/run")
async def run_with_cache(request: AgentRequest):
    if (cached := cache.get(request.message)) is not None:
        return {"response": cached, "cached": True}
    result = await agent.run(request.message)
    cache.set(request.message, result.output)
    return {"response": result.output, "cached": False}
```

The full `PromptCache` class — a small bounded LRU-style cache keyed on the prompt string. Because LLM calls are async and the goal is to cache the **result of the call**, you can't simply decorate the agent invocation with `@lru_cache` — that decorator doesn't await coroutines.

```
import os
from collections import OrderedDict
from pydantic_ai import Agent

agent = Agent(os.getenv('AI_MODEL', 'openai:gpt-5.4-mini'))

class PromptCache:
    """Bounded LRU-style cache for prompt -> response strings."""
    def __init__(self, maxsize: int = 1000):
        self._store: OrderedDict[str, str] = OrderedDict()
        self._maxsize = maxsize

    def get(self, key: str) -> str | None:
        if key in self._store:
            self._store.move_to_end(key)
            return self._store[key]
        return None

    def set(self, key: str, value: str) -> None:
        self._store[key] = value
        self._store.move_to_end(key)
        if len(self._store) > self._maxsize:
            self._store.popitem(last=False)

cache = PromptCache(maxsize=1000)
```

For pure synchronous, deterministic helpers — token counters, prompt formatters, schema validators — `@functools.lru_cache` is still the right tool. Reach for an explicit cache (or Redis in production) only when caching async or non-deterministic results.

10.10. Summary

In this module, you learned how to:

- Test AI agents effectively using mock models and deterministic testing strategies
- Implement simulated backends to avoid costly API calls during development
- Capture and analyze agent execution traces for debugging and verification
- Integrate observability tools like structured logging and OpenTelemetry for production monitoring
- Deploy agents as FastAPI web services with proper concurrency control
- Track usage and implement cost management measures to control API expenses
- Secure agent endpoints with authentication, input validation, and rate limiting
- Apply scaling strategies including caching and horizontal scaling for production deployments

These practices ensure your AI agents are reliable, observable, cost-effective, and secure in production environments.

10.11. Reflection Questions

- How does testing AI agents differ from testing traditional deterministic software, and what strategies help bridge the gap?
- What are the trade-offs between using mock models for testing versus testing against real LLM APIs?
- How would you design a cost management strategy that balances budget constraints with response quality?
- What security risks are unique to AI agent endpoints, and how do input validation and rate limiting help address them?

10.12. Additional Resources

- Pydantic AI testing documentation: <https://ai.pydantic.dev>
- FastAPI documentation: <https://fastapi.tiangolo.com>
- Python pytest documentation: <https://docs.python.org/3/library/unittest.html>
- OpenAI API usage and billing documentation: <https://platform.openai.com/docs>

Course Conclusion

LESSON 11

-
- ✓ Describe the key concepts and patterns covered throughout this course
 - ✓ Identify resources for continued learning in agentic AI development
 - ✓ Evaluate next steps for applying agentic AI patterns in real-world projects

11.1. Outline

- Summary of course concepts
- Resources for continued learning
- Where to apply each pattern first
- Reflection and key takeaways

11.2. Summary

Congratulations on completing this course on Practical Development of Agentic AI Applications! Throughout this course, you have learned:

- The fundamental concepts and anatomy of AI agents
- How to create structured outputs with validation using Pydantic models
- Techniques for integrating tools and extending agent capabilities
- Dependency injection and runtime context management
- Managing conversations, memory, and state in multi-turn interactions
- Implementing evaluation metrics and guardrails for safety
- Orchestrating complex multi-step workflows
- Designing and coordinating multi-agent systems
- Testing strategies and production deployment practices

11.3. Resources

- **Pydantic AI documentation** — <https://ai.pydantic.dev> — the primary reference for the framework. Start with the **Concepts** section, then drill into the API page that matches whichever pattern you used most in the labs.
- **Pydantic documentation** — <https://docs.pydantic.dev> — covers `BaseModel`, validators, and the v1→v2 migration notes that come up whenever you encounter older tutorials.

- **OpenAI function-calling guide** — <https://platform.openai.com/docs/guides/function-calling> — the underlying mechanism Pydantic AI's tool layer abstracts; useful when you need to reason about what the model actually receives.
 - **Astral uv documentation** — <https://docs.astral.sh/uv/> — for the per-project venv and lockfile patterns the course labs used.
 - **Anthropic and Google API references** — <https://docs.anthropic.com> and <https://ai.google.dev/> — for swapping providers via the `anthropic:` and `google:` model strings shown in the **Anatomy of an AI Agent** module.
-

The labs use a consistent `AI_MODEL=<provider>:<name>` convention so the same code runs against any supported provider. When you start your own project, pin the model identifier in your `.env` and verify it appears in `pydantic_ai.models.KnownModelName` — Pydantic AI's catalog moves quickly and stale identifiers are a common first-day production failure.

11.4. Next Steps

Where to apply each pattern first — match the architecture to the task, not the other way around.

- **Single-agent designs** are the right default when the task is **well-scoped** (a clear input, a clear output, no branching workflow), the inputs are **clean** (no aggregation across systems), and the latency budget is **interactive** (a few seconds for a response). Most "answer a question" or "extract data from text" use cases fit here.
- **Multi-agent designs** are warranted when **task decomposition is natural** (research → analyze → write are genuinely separate jobs), specialization gives measurable quality gains, and **latency tolerance is high enough** to absorb the additional coordination overhead. If you can't articulate why one agent isn't enough, you don't need more than one.
- **Workflow orchestration** (checkpointing, retries, fallbacks) earns its complexity when failure of any step is **expensive** (loss of work, cost, or user

trust) and the workflow is **long enough** that restarts from scratch are unacceptable. Short pipelines don't need orchestration; multi-hour batch jobs do.

- **Production deployment patterns** (FastAPI, observability, cost controls) come **after** you've validated quality on a single agent. Don't deploy what you haven't measured.
-

The most common new-builder mistake is reaching for multi-agent or workflow orchestration before single-agent quality is solid. Start small — single agent, validated outputs, one tool — measure quality with the evaluation harness from the **Evaluation and Guardrails** module, **then** add complexity only where it pays for itself.

11.5. Reflection Questions

- Which module or pattern from this course do you think will have the most immediate impact on your work?
- How would you approach designing your first production agent system using the patterns covered in this course?
- What are the most important considerations when deciding between a single-agent and multi-agent architecture for a real-world project?
- How has your understanding of AI agent testing and evaluation changed throughout this course?

11.6. Key Takeaways

- Agents are **LLMs plus surrounding machinery** — tools, memory, validation, and a control loop. The framework supplies the machinery; the model supplies the reasoning.
- **Structured outputs are the load-bearing reliability primitive.** A typed schema turns a free-form completion into a contract you can test, validate, and integrate with confidence.

- **Operational concerns scale with system complexity, not with intent.**

Memory patterns, guardrails, evaluation harnesses, and observability earn their cost only when the system reaches the size where they pay back.

Reach for them deliberately.

- **Test first, deploy second.** Deterministic tests against simulated backends catch regressions that production traffic discovers expensively. The patterns from the **Testing and Production Deployment** module apply on day one of any project, not at the end.